Level 8: Real Interview Problems (Questions 181–200)

These twenty questions are styled after real "company round" SQL interview questions. Each one states a business problem first, then the query that solves it, then a short breakdown of how the query actually works. Every query in this level runs against the same sample database, set up once below.

Sample Database Setup

Run this once before trying any of the queries below. It creates the `company` database with seven tables – `employees`, `departments`, `projects`, `orders`, `customers`, `products`, `order_items` – and loads them with the same sample rows used throughout this book (the same 15 employees from the Level 7 org-chart examples, plus orders, products and customers to support the joins in this level).

```
-- Create employee management database
CREATE DATABASE IF NOT EXISTS company;
USE company;

-- Core Tables
CREATE TABLE employees (
    emp_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    age INT,
    gender VARCHAR(10),
    salary DECIMAL(10,2),
    department VARCHAR(50),
    designation VARCHAR(50),
    manager_id INT,
    hire_date DATE,
    city VARCHAR(50),
    bonus DECIMAL(10,2),
    experience_years INT
);

CREATE TABLE departments (
    dept_id INT PRIMARY KEY AUTO_INCREMENT,
    dept_name VARCHAR(100) UNIQUE,
    location VARCHAR(100),
    budget DECIMAL(15,2)
);

CREATE TABLE projects (
    project_id INT PRIMARY KEY AUTO_INCREMENT,
    project_name VARCHAR(100),
    department VARCHAR(50),
    status VARCHAR(20),
    start_date DATE,
    end_date DATE,
    budget DECIMAL(15,2)
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT,
    order_date DATE,
    amount DECIMAL(10,2),
    status VARCHAR(20),
    product_category VARCHAR(50)
);
```

```sql
CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    city VARCHAR(50),
    registration_date DATE,
    email VARCHAR(100)
);

CREATE TABLE products (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    product_name VARCHAR(100),
    category VARCHAR(50),
    price DECIMAL(10,2),
    stock_quantity INT
);

CREATE TABLE order_items (
    order_item_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT,
    product_id INT,
    quantity INT,
    price DECIMAL(10,2)
);

-- Insert Sample Data
INSERT INTO employees VALUES
(1,'Aarav Sharma',28,'M',75000,'IT','Senior Developer',NULL,'2020-01-15','Mumbai',10000,4),
(2,'Priya Patel',32,'F',65000,'HR','HR Manager',1,'2019-03-20','Delhi',8000,5),
(3,'Rahul Singh',45,'M',85000,'Finance','Finance Head',1,'2018-07-10','Mumbai',15000,8),
(4,'Neha Gupta',29,'F',72000,'IT','Developer',1,'2021-02-01','Bangalore',9000,3),
(5,'Vikram Reddy',38,'M',90000,'IT','Team Lead',1,'2017-11-25','Delhi',12000,6),
(6,'Ananya Menon',27,'F',60000,'HR','HR Executive',2,'2022-06-15','Mumbai',5000,2),
(7,'Karthik Nair',51,'M',95000,'IT','Architect',1,'2015-09-30','Bangalore',20000,10),
(8,'Meera Krishnan',33,'F',68000,'Marketing','Marketing Lead',5,'2020-08-12','Delhi',7000,4),
(9,'Arjun Reddy',42,'M',82000,'IT','Senior Developer',5,'2016-12-01','Mumbai',11000,7),
(10,'Sneha Patel',26,'F',58000,'Marketing','Marketing Executive',8,'2023-01-20','Bangalore',4000,1),
(11,'Rajesh Kumar',39,'M',70000,'Finance','Financial Analyst',3,'2019-06-10','Chennai',6000,5),
(12,'Deepa Iyer',34,'F',64000,'HR','HR Specialist',2,'2020-11-15','Hyderabad',5500,4),
(13,'Suresh Babu',47,'M',88000,'IT','Solution Architect',5,'2014-03-01','Chennai',18000,10),
(14,'Kavya Nair',31,'F',67000,'Finance','Accountant',3,'2021-08-20','Hyderabad',5000,3),
(15,'Ravi Teja',44,'M',91000,'IT','Technical Manager',1,'2016-07-15','Mumbai',16000,8);

INSERT INTO departments VALUES
(1,'IT','Mumbai',5000000),
(2,'HR','Delhi',2000000),
(3,'Finance','Mumbai',3000000),
(4,'Marketing','Bangalore',2500000),
(5,'Operations','Pune',3500000);

INSERT INTO projects VALUES
(1,'AI Implementation','IT','Active','2024-01-01','2024-12-31',1500000),
(2,'Employee Portal','IT','Completed','2023-06-01','2023-12-31',800000),
(3,'Financial Audit','Finance','Active','2024-02-01','2024-08-31',500000),
(4,'Market Research','Marketing','Planning','2024-03-01','2024-09-30',300000),
(5,'Cloud Migration','IT','Active','2024-01-15','2024-11-30',2000000),
(6,'Talent Acquisition','HR','Completed','2023-10-01','2024-01-31',400000);

INSERT INTO customers VALUES
```

```
(1,'TechCorp','Mumbai','2023-01-10','tech@corp.com'),
(2,'GlobalSolutions','Delhi','2023-02-15','info@global.com'),
(3,'InnovationLabs','Bangalore','2023-03-20','contact@innovate.com'),
(4,'AlphaInc','Mumbai','2023-04-25','alpha@inc.com'),
(5,'BetaSystems','Pune','2023-05-30','beta@systems.com'),
(6,'GammaTech','Chennai','2023-06-10','gamma@tech.com'),
(7,'DeltaCorp','Hyderabad','2023-07-05','delta@corp.com'),
(8,'EpsilonSoft','Bangalore','2023-08-15','epsilon@soft.com'),
(9,'ThetaSolutions','Delhi','2023-09-20','theta@solutions.com'),
(10,'OmegaLabs','Mumbai','2023-10-01','omega@labs.com');

INSERT INTO orders VALUES
(101,1,'2024-01-10',15000,'Completed','Electronics'),
(102,1,'2024-01-15',23000,'Pending','Software'),
(103,2,'2024-01-20',12000,'Completed','Hardware'),
(104,3,'2024-02-01',50000,'Completed','Electronics'),
(105,3,'2024-02-05',18000,'Cancelled','Software'),
(106,4,'2024-02-10',32000,'Completed','Hardware'),
(107,5,'2024-02-15',45000,'Pending','Electronics'),
(108,6,'2024-03-01',21000,'Completed','Software'),
(109,7,'2024-03-05',67000,'Completed','Electronics'),
(110,8,'2024-03-10',34000,'Completed','Hardware'),
(111,9,'2024-03-15',28000,'Pending','Software'),
(112,10,'2024-04-01',42000,'Completed','Electronics'),
(113,1,'2024-04-05',15000,'Completed','Hardware'),
(114,2,'2024-04-10',33000,'Completed','Software'),
(115,3,'2024-04-15',29000,'Pending','Electronics');

INSERT INTO products VALUES
(1,'Laptop Pro','Electronics',75000,50),
(2,'Wireless Mouse','Accessories',1500,200),
(3,'27" Monitor','Electronics',25000,30),
(4,'Mechanical Keyboard','Accessories',5000,100),
(5,'Enterprise Software','Software',100000,20),
(6,'External SSD','Electronics',12000,60),
(7,'Webcam HD','Accessories',3000,80),
(8,'Cloud Storage Plan','Software',50000,15),
(9,'Printer','Electronics',15000,40),
(10,'USB Hub','Accessories',2000,150);

INSERT INTO order_items VALUES
(1,101,1,2,75000),
(2,101,2,1,1500),
(3,102,5,1,100000),
(4,103,3,1,25000),
(5,104,1,1,75000),
(6,104,6,2,12000),
(7,105,5,1,100000),
(8,106,3,2,25000),
(9,107,1,3,75000),
(10,108,8,1,50000),
(11,109,1,2,75000),
(12,110,6,2,12000),
(13,111,5,1,100000),
(14,112,1,1,75000),
(15,113,4,1,5000);
```

Q181: Amazon orders analysis

PROBLEM

Find products ordered in the last 30 days but with zero sales in the previous 90 days

```
WITH recent_sales AS (
    SELECT DISTINCT product_id
    FROM order_items oi
    JOIN orders o ON oi.order_id = o.order_id
    WHERE o.order_date >= CURDATE() - INTERVAL 30 DAY
),
past_sales AS (
    SELECT DISTINCT product_id
    FROM order_items oi
    JOIN orders o ON oi.order_id = o.order_id
    WHERE o.order_date BETWEEN CURDATE() - INTERVAL 120 DAY AND CURDATE() - INTERVAL 31 DAY
)
SELECT p.product_name, p.price
FROM products p
WHERE p.product_id IN (SELECT product_id FROM recent_sales)
AND p.product_id NOT IN (SELECT product_id FROM past_sales);
```

HOW THIS WORKS

Two CTEs build two separate ID lists – products sold in the last 30 days, and products sold in the 90-day window before that – and the final query keeps only IDs that appear in the first list but not the second. This “IN one set, NOT IN another set” shape is the standard way to spot products that suddenly came back to life after a quiet spell, without needing a join at all.

Q182: Flipkart sales analysis

PROBLEM

Top 3 sellers by month

```
WITH monthly_sales AS (
    SELECT
        e.name AS seller,
        DATE_FORMAT(o.order_date, '%Y-%m') AS month,
        SUM(o.amount) AS total_sales
    FROM employees e
    JOIN orders o ON e.emp_id = o.customer_id
    GROUP BY e.name, DATE_FORMAT(o.order_date, '%Y-%m')
),
ranked_sales AS (
    SELECT *,
           RANK() OVER (PARTITION BY month ORDER BY total_sales DESC) AS rn
    FROM monthly_sales
)
SELECT month, seller, total_sales
FROM ranked_sales
WHERE rn <= 3
ORDER BY month DESC, rn;
```

HOW THIS WORKS

This is the “top-N per group” pattern from Level 6, just wrapped in CTEs for readability: the first CTE rolls sales up by seller and month, the second ranks sellers within each month with RANK() OVER (PARTITION BY month ...), and the outer query keeps only rn <= 3. Splitting it into two named steps makes it obvious that ranking happens after aggregation, not before.

Q183: Swiggy delivery analysis

PROBLEM

Average delivery time by city and delivery partner

```
CREATE TABLE deliveries (
    delivery_id INT,
    restaurant_id INT,
    delivery_partner_id INT,
    order_time DATETIME,
    delivery_time DATETIME,
    city VARCHAR(50)
);

SELECT
    city,
    delivery_partner_id,
    AVG(TIMESTAMPDIFF(MINUTE, order_time, delivery_time)) AS avg_delivery_time,
    COUNT(*) AS total_deliveries
FROM deliveries
WHERE delivery_time IS NOT NULL
GROUP BY city, delivery_partner_id
HAVING total_deliveries >= 10
ORDER BY city, avg_delivery_time;
```

HOW THIS WORKS

A plain GROUP BY – no CTE needed here. TIMESTAMPDIFF(MINUTE, order_time, delivery_time) turns two timestamps into a single “minutes taken” number per row, AVG() rolls that up per city/partner pair, and HAVING total_deliveries >= 10 filters out partners with too few deliveries to trust the average. Filtering delivery_time IS NOT NULL first excludes orders that haven’t been delivered yet.

Q184: Zomato customer segmentation

PROBLEM

Segment customers based on order frequency and amount

```
WITH customer_metrics AS (
    SELECT
        c.customer_id,
        c.name,
        COUNT(o.order_id) AS order_count,
        SUM(o.amount) AS total_spent,
        AVG(o.amount) AS avg_order_value,
        DATEDIFF(CURDATE(), MAX(o.order_date)) AS days_since_last_order
    FROM customers c
    LEFT JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.customer_id, c.name
)
SELECT
    name,
    order_count,
    total_spent,
    CASE
        WHEN order_count >= 10 AND total_spent > 100000 THEN 'Platinum'
        WHEN order_count >= 5 AND total_spent > 50000 THEN 'Gold'
        WHEN order_count >= 2 AND total_spent > 10000 THEN 'Silver'
        WHEN days_since_last_order > 90 THEN 'Churn Risk'
        ELSE 'Bronze'
    END AS customer_segment,
    CASE
        WHEN days_since_last_order < 7 THEN 'Very Active'
        WHEN days_since_last_order < 30 THEN 'Active'
        WHEN days_since_last_order < 90 THEN 'Dormant'
        ELSE 'Inactive'
    END AS activity_status
FROM customer_metrics
ORDER BY total_spent DESC;
```

HOW THIS WORKS

This is a classic RFM-style segmentation: one CTE computes each customer’s order count, total spend, and recency (days_since_last_order) in a single pass, then two separate CASE expressions turn those raw numbers into human-readable labels. Notice the CASE branches are checked top-to-bottom – a customer is only “Gold” if they fail the “Platinum” test first – so branch order matters as much as the conditions themselves.

Q185: Uber trip analysis

PROBLEM

Find drivers with highest acceptance rate and earnings

```
CREATE TABLE trips (
    trip_id INT,
    driver_id INT,
    rider_id INT,
    request_time DATETIME,
    pickup_time DATETIME,
    dropoff_time DATETIME,
    status VARCHAR(20),
    fare DECIMAL(10,2),
    distance_km DECIMAL(5,2)
);

WITH driver_metrics AS (
    SELECT
        driver_id,
        COUNT(*) AS total_requests,
        SUM(CASE WHEN status = 'Completed' THEN 1 ELSE 0 END) AS completed_trips,
        SUM(CASE WHEN status = 'Completed' THEN fare ELSE 0 END) AS total_earnings,
        AVG(CASE WHEN status = 'Completed' THEN
            TIMESTAMPDIFF(MINUTE, pickup_time, dropoff_time)
        END) AS avg_trip_time
    FROM trips
    GROUP BY driver_id
)
SELECT
    driver_id,
    completed_trips,
    total_earnings,
    ROUND(completed_trips * 100.0 / total_requests, 2) AS acceptance_rate,
    RANK() OVER (ORDER BY total_earnings DESC) AS earnings_rank
FROM driver_metrics
WHERE total_requests > 50
ORDER BY acceptance_rate DESC, total_earnings DESC;
```

HOW THIS WORKS

SUM(CASE WHEN status = ‘Completed’ THEN 1 ELSE 0 END) is the standard trick for “counting only rows that match a condition” inside a single GROUP BY, used twice here – once to count completed trips, once to total their fares. total_requests (every row, completed or not) stays as a plain COUNT(*) so the acceptance-rate division afterward compares completed trips against all requests, not just completed ones.

Q186: Ola ride cancellation analysis

PROBLEM

Find reasons for high cancellation rate by time of day

```
WITH trip_cancellations AS (
    SELECT
        CASE
            WHEN HOUR(request_time) BETWEEN 6 AND 11 THEN 'Morning'
            WHEN HOUR(request_time) BETWEEN 12 AND 17 THEN 'Afternoon'
            WHEN HOUR(request_time) BETWEEN 18 AND 23 THEN 'Evening'
            ELSE 'Night'
        END AS time_of_day,
        status,
        COUNT(*) AS trip_count
    FROM trips
    WHERE status = 'Cancelled'
    GROUP BY time_of_day, status
),
total_by_time AS (
    SELECT
        CASE
            WHEN HOUR(request_time) BETWEEN 6 AND 11 THEN 'Morning'
            WHEN HOUR(request_time) BETWEEN 12 AND 17 THEN 'Afternoon'
            WHEN HOUR(request_time) BETWEEN 18 AND 23 THEN 'Evening'
            ELSE 'Night'
        END AS time_of_day,
        COUNT(*) AS total_trips
    FROM trips
    GROUP BY time_of_day
)
SELECT
    tc.time_of_day,
    tc.trip_count AS cancellations,
    tt.total_trips,
    ROUND(tc.trip_count * 100.0 / tt.total_trips, 2) AS cancellation_rate
FROM trip_cancellations tc
JOIN total_by_time tt ON tc.time_of_day = tt.time_of_day
ORDER BY cancellation_rate DESC;
```

HOW THIS WORKS

The same CASE expression buckets HOUR(request_time) into four labels in both CTEs – one counts only cancelled trips per bucket, the other counts all trips per bucket – and the join lines the two counts up so the rate can be divided out. Whenever you see the identical CASE logic repeated across two CTEs like this, it's because one side needs a filtered count and the other needs the unfiltered total.

Q187: Netflix user watch patterns

PROBLEM

Find binge-watching sessions (3+ episodes in 24 hours)

```
CREATE TABLE viewing_history (
    user_id INT,
    episode_id INT,
    series_id INT,
    watch_date DATETIME,
    duration INT
);

WITH binge_patterns AS (
    SELECT
        user_id,
        series_id,
        watch_date,
        LAG(watch_date) OVER (PARTITION BY user_id, series_id ORDER BY watch_date) AS prev_watch,
        COUNT(*) OVER (PARTITION BY user_id, series_id
                       ORDER BY watch_date
                       RANGE BETWEEN INTERVAL 24 HOUR PRECEDING AND CURRENT ROW) AS episodes_in_24h
    FROM viewing_history
)
SELECT
    user_id,
    series_id,
    MIN(watch_date) AS binge_start,
    MAX(watch_date) AS binge_end,
    episodes_in_24h
FROM binge_patterns
WHERE episodes_in_24h >= 3
GROUP BY user_id, series_id, episodes_in_24h
ORDER BY episodes_in_24h DESC;
```

HOW THIS WORKS

The interesting part is the window frame: RANGE BETWEEN INTERVAL 24 HOUR PRECEDING AND CURRENT ROW is a time-based window rather than a row-count-based one – instead of “the previous 2 rows” it means “every row whose watch_date falls within the last 24 hours of this row’s watch_date”. COUNT(*) over that frame gives a rolling episode count per user/series, and keeping only episodes_in_24h >= 3 surfaces genuine binge sessions. (LAG here is computed but unused in the final SELECT – in practice you’d drop it or use it to also report the gap between episodes.)

Q188: Spotify song popularity analysis

PROBLEM

Find songs that increased in popularity month-over-month

```
CREATE TABLE song_plays (
    song_id INT,
    play_date DATE,
    plays_count INT,
    unique_listeners INT
);

WITH monthly_plays AS (
    SELECT
        song_id,
        DATE_FORMAT(play_date, '%Y-%m') AS month,
        SUM(plays_count) AS total_plays,
        SUM(unique_listeners) AS total_listeners
    FROM song_plays
    GROUP BY song_id, DATE_FORMAT(play_date, '%Y-%m')
),
play_growth AS (
    SELECT
        song_id,
        month,
        total_plays,
        LAG(total_plays) OVER (PARTITION BY song_id ORDER BY month) AS prev_month_plays,
        LAG(total_listeners) OVER (PARTITION BY song_id ORDER BY month) AS prev_month_listeners
    FROM monthly_plays
)
SELECT
    song_id,
    month,
    total_plays,
    prev_month_plays,
    ROUND((total_plays - prev_month_plays) * 100.0 / prev_month_plays, 2) AS growth_percent
FROM play_growth
WHERE prev_month_plays IS NOT NULL
AND total_plays > prev_month_plays
ORDER BY growth_percent DESC
LIMIT 10;
```

HOW THIS WORKS

LAG(total_plays) OVER (PARTITION BY song_id ORDER BY month) fetches “last month’s total_plays for this same song” onto the current row, without any self-join. Once that value sits alongside this month’s number, a plain percentage-change formula does the rest. prev_month_plays IS NOT NULL drops each song’s first month, which has no prior month to compare against.

Q189: YouTube video performance

PROBLEM

Identify videos with high engagement ratio (likes + comments / views)

```
CREATE TABLE videos (
    video_id INT,
    upload_date DATE,
    views INT,
    likes INT,
    comments INT,
    shares INT,
    category VARCHAR(50)
);

SELECT
    video_id,
    category,
    views,
    likes,
    comments,
    shares,
    ROUND((likes + comments + shares) * 100.0 / views, 2) AS engagement_rate,
    CASE
        WHEN views > 1000000 THEN 'Viral'
        WHEN views > 100000 THEN 'Popular'
        WHEN views > 10000 THEN 'Rising'
        ELSE 'New'
    END AS video_tier
FROM videos
WHERE views > 1000
ORDER BY engagement_rate DESC
LIMIT 20;
```

HOW THIS WORKS

No CTE or window function needed – just row-level arithmetic. engagement_rate is computed directly per row as (likes + comments + shares) / views, and video_tier buckets the raw view count with a CASE expression. Multiplying by 100.0 (not 100) forces the division into floating-point math before ROUND(), which is what keeps the result from truncating to a whole number.

Q190: Instagram follower growth

PROBLEM

Calculate follower growth rate by month

```
CREATE TABLE follower_history (
    account_id INT,
    date DATE,
    followers INT,
    following INT,
    posts_count INT
);

WITH monthly_growth AS (
    SELECT
        account_id,
        DATE_FORMAT(date, '%Y-%m') AS month,
        MIN(followers) AS start_followers,
        MAX(followers) AS end_followers,
        MIN(posts_count) AS start_posts,
        MAX(posts_count) AS end_posts
    FROM follower_history
    GROUP BY account_id, DATE_FORMAT(date, '%Y-%m')
)
SELECT
    account_id,
    month,
    start_followers,
    end_followers,
    end_followers - start_followers AS net_gain,
    ROUND((end_followers - start_followers) * 100.0 / start_followers, 2) AS growth_rate,
    (end_posts - start_posts) AS new_posts
FROM monthly_growth
WHERE start_followers > 0
ORDER BY growth_rate DESC;
```

HOW THIS WORKS

Rather than LAG-ing a previous row, this squeezes “start of month” and “end of month” out of a single GROUP BY: since follower_history logs one row per day, MIN(followers) within a month is (assuming followers only grow) the earliest reading and MAX(followers) the latest. It’s a lighter-weight alternative to LAG() that works whenever a metric only moves in one direction across the days being grouped.

Q191: Banking transaction analysis

PROBLEM

Identify suspicious transactions (> 3x average, > 5 transactions/day)

```
CREATE TABLE bank_transactions (
    transaction_id INT,
    customer_id INT,
    transaction_date DATETIME,
    amount DECIMAL(10,2),
    type VARCHAR(20),
    status VARCHAR(20)
);

WITH customer_avg AS (
    SELECT
        customer_id,
        AVG(amount) AS avg_transaction,
        STDDEV(amount) AS stddev_transaction
    FROM bank_transactions
    WHERE status = 'Completed'
    GROUP BY customer_id
),
daily_counts AS (
    SELECT
        customer_id,
        DATE(transaction_date) AS day,
        COUNT(*) AS daily_count
    FROM bank_transactions
    GROUP BY customer_id, DATE(transaction_date)
)
SELECT
    t.transaction_id,
    t.customer_id,
    t.amount,
    c.avg_transaction,
    t.amount / c.avg_transaction AS multiplier,
    CASE
        WHEN t.amount > c.avg_transaction + 3 * c.stddev_transaction THEN 'Extreme'
        WHEN t.amount > c.avg_transaction * 2 THEN 'High'
        ELSE 'Normal'
    END AS risk_level
FROM bank_transactions t
JOIN customer_avg c ON t.customer_id = c.customer_id
WHERE t.amount > c.avg_transaction * 2
OR (t.customer_id, DATE(t.transaction_date)) IN (
    SELECT customer_id, day FROM daily_counts WHERE daily_count > 5
)
ORDER BY t.amount DESC;
```

HOW THIS WORKS

Two independent CTEs each answer one half of “what does normal look like for this customer” – customer_avg gives a per-customer baseline (mean and standard deviation), daily_counts gives a per-day transaction frequency. The WHERE clause then flags a transaction if either condition trips: its amount is more than double that customer’s average, OR it belongs to a (customer, day) pair with more than 5 transactions. The tuple form (t.customer_id, DATE(t.transaction_date)) IN (...) is a compact way to check two columns against a subquery at once, instead of writing a second join.

Q192: ATM withdrawal patterns

PROBLEM

Find ATMs with most withdrawals and average amount by time

```
CREATE TABLE atm_transactions (
    atm_id INT,
    transaction_time DATETIME,
    amount DECIMAL(10,2),
    customer_id INT,
    status VARCHAR(20)
);

SELECT
    atm_id,
    DATE(transaction_time) AS day,
    HOUR(transaction_time) AS hour,
    COUNT(*) AS withdrawals,
    AVG(amount) AS avg_amount,
    SUM(amount) AS total_amount,
    RANK() OVER (PARTITION BY DATE(transaction_time) ORDER BY COUNT(*) DESC) AS withdrawal_rank
FROM atm_transactions
WHERE status = 'Successful'
AND transaction_time >= CURDATE() - INTERVAL 30 DAY
GROUP BY atm_id, DATE(transaction_time), HOUR(transaction_time)
HAVING withdrawals >= 10
ORDER BY day DESC, withdrawal_rank;
```

HOW THIS WORKS

This mixes an aggregate window function directly into a GROUP BY query: COUNT*() OVER (PARTITION BY DATE(…) …) is allowed to reference the same COUNT()* that GROUP BY just computed, because window functions always run after grouping. The result is each atm/day/hour bucket ranked against every other bucket from that same day, so you can see which ATM was busiest on any given day without a separate query.

Q193: Hospital patient readmission analysis

PROBLEM

Calculate 30-day readmission rate by department

```
CREATE TABLE admissions (
    admission_id INT,
    patient_id INT,
    admission_date DATE,
    discharge_date DATE,
    department VARCHAR(50),
    diagnosis VARCHAR(100)
);

WITH readmissions AS (
    SELECT
        a1.patient_id,
        a1.department,
        a1.admission_date,
        a1.discharge_date,
        a2.admission_date AS readmission_date,
        DATEDIFF(a2.admission_date, a1.discharge_date) AS days_between
    FROM admissions a1
    LEFT JOIN admissions a2 ON a1.patient_id = a2.patient_id
        AND a2.admission_date > a1.admission_date
        AND DATEDIFF(a2.admission_date, a1.discharge_date) <= 30
),
department_stats AS (
    SELECT
        department,
        COUNT(DISTINCT patient_id) AS unique_patients,
        COUNT(DISTINCT CASE WHEN readmission_date IS NOT NULL THEN patient_id END) AS readmitted_patients
    FROM readmissions
    GROUP BY department
)
SELECT
    department,
    unique_patients,
    readmitted_patients,
    ROUND(readmitted_patients * 100.0 / unique_patients, 2) AS readmission_rate
FROM department_stats
WHERE unique_patients >= 10
ORDER BY readmission_rate DESC;
```

HOW THIS WORKS

The readmissions CTE is a self-join on the same admissions table (a1 = the original stay, a2 = any later stay by the same patient within 30 days of discharge) – structurally similar to the self-referencing joins used for hierarchies earlier, but joined by patient_id and a date condition instead of a manager_id. LEFT JOIN keeps patients with no readmission too (their readmission_date is NULL), which is exactly what COUNT(DISTINCT CASE WHEN readmission_date IS NOT NULL THEN patient_id END) needs in order to count only the patients who really did come back.

Q194: Student marks analysis

PROBLEM

Find top 3 students in each subject with average > 80

```
CREATE TABLE marks (
    student_id INT,
    subject VARCHAR(50),
    marks INT,
    exam_date DATE
);

WITH student_subject_scores AS (
    SELECT
        student_id,
        subject,
        AVG(marks) AS avg_marks,
        COUNT(*) AS exams_taken
    FROM marks
    GROUP BY student_id, subject
    HAVING avg_marks > 80
),
ranked_students AS (
    SELECT
        student_id,
        subject,
        avg_marks,
        RANK() OVER (PARTITION BY subject ORDER BY avg_marks DESC) AS subject_rank
    FROM student_subject_scores
)
SELECT
    student_id,
    subject,
    ROUND(avg_marks, 2) AS avg_score,
    subject_rank
FROM ranked_students
WHERE subject_rank <= 3
ORDER BY subject, subject_rank;
```

HOW THIS WORKS

Two filters happen at two different stages, which is why this needs two CTEs rather than one: HAVING avg_marks > 80 filters out weak students before ranking even starts, then RANK() OVER (PARTITION BY subject ...) ranks only the students who survived that filter, and subject_rank <= 3 keeps the podium. Doing the HAVING filter and the rank filter in the same WHERE clause wouldn't work, since a WHERE clause can't reference a window function's own output.

Q195: E-commerce revenue analysis by product category

PROBLEM

Find categories with highest growth in revenue

```
WITH monthly_category_revenue AS (
    SELECT
        p.category,
        DATE_FORMAT(o.order_date, '%Y-%m') AS month,
        SUM(oi.quantity * oi.price) AS revenue
    FROM products p
    JOIN order_items oi ON p.product_id = oi.product_id
    JOIN orders o ON oi.order_id = o.order_id
    WHERE o.status = 'Completed'
    GROUP BY p.category, DATE_FORMAT(o.order_date, '%Y-%m')
),
category_growth AS (
    SELECT
        category,
        month,
        revenue,
        LAG(revenue) OVER (PARTITION BY category ORDER BY month) AS prev_month_revenue
    FROM monthly_category_revenue
)
SELECT
    category,
    month,
    revenue,
    prev_month_revenue,
    ROUND((revenue - prev_month_revenue) * 100.0 / prev_month_revenue, 2) AS growth_percent,
    CASE
        WHEN revenue > prev_month_revenue THEN 'Growing'
        WHEN revenue < prev_month_revenue THEN 'Declining'
        ELSE 'Stable'
    END AS trend
FROM category_growth
WHERE prev_month_revenue IS NOT NULL
ORDER BY growth_percent DESC;
```

HOW THIS WORKS

Same LAG-based month-over-month shape as Q188's Spotify question and Q190's Instagram question – by this point in the level the pattern should be recognizable on sight: aggregate to one row per group-per-month, LAG() that revenue within each group, then compare current vs. previous. The only new piece here is the three-way join (products, order_items, orders) needed just to get category and revenue onto the same row in the first place.

Q196: Warehouse inventory optimization

PROBLEM

Identify products with low turnover (aged inventory)

```
CREATE TABLE inventory_items (
    item_id INT,
    product_id INT,
    warehouse_id INT,
    quantity INT,
    last_restock_date DATE,
    sales_last_30_days INT
);

WITH inventory_metrics AS (
    SELECT
        product_id,
        warehouse_id,
        quantity,
        DATEDIFF(CURDATE(), last_restock_date) AS days_since_restock,
        sales_last_30_days,
        CASE
            WHEN sales_last_30_days = 0 AND DATEDIFF(CURDATE(), last_restock_date) > 90 THEN 'Critical'
            WHEN sales_last_30_days < 5 AND DATEDIFF(CURDATE(), last_restock_date) > 60 THEN 'Excess'
            WHEN quantity < 10 AND sales_last_30_days > 0 THEN 'Low Stock'
            ELSE 'Normal'
        END AS inventory_status
    FROM inventory_items
)
SELECT
    product_id,
    warehouse_id,
    quantity,
    days_since_restock,
    sales_last_30_days,
    inventory_status,
    ROUND(quantity * 1.0 / NULLIF(sales_last_30_days, 0), 2) AS days_of_inventory
FROM inventory_metrics
WHERE inventory_status != 'Normal'
ORDER BY FIELD(inventory_status, 'Critical', 'Low Stock', 'Excess'), days_since_restock DESC;
```

HOW THIS WORKS

NULLIF(sales_last_30_days, 0) swaps a 0 for NULL right before the division, which is what stops quantity / 0 from throwing a divide-by-zero error – dividing by NULL just quietly produces NULL instead. FIELD(inventory_status, ‘Critical’, ‘Low Stock’, ‘Excess’) is a handy way to sort text values into a custom order (not alphabetical) by giving each string a position number to sort by, so the most urgent status prints first.

Q197: Flight booking analysis

PROBLEM

Find optimal pricing by analyzing booking patterns

```
CREATE TABLE flight_bookings (
    booking_id INT,
    flight_id INT,
    booking_date DATE,
    travel_date DATE,
```

```
    price DECIMAL(10,2),
    seats_booked INT,
    customer_id INT
);

WITH booking_metrics AS (
    SELECT
        flight_id,
        booking_date,
        travel_date,
        price,
        seats_booked,
        DATEDIFF(travel_date, booking_date) AS days_in_advance,
        CASE
            WHEN DATEDIFF(travel_date, booking_date) > 60 THEN 'Very Early'
            WHEN DATEDIFF(travel_date, booking_date) > 30 THEN 'Early'
            WHEN DATEDIFF(travel_date, booking_date) > 14 THEN 'Mid'
            WHEN DATEDIFF(travel_date, booking_date) > 7 THEN 'Late'
            ELSE 'Last Minute'
        END AS booking_window
    FROM flight_bookings
),
price_analysis AS (
    SELECT
        flight_id,
        booking_window,
        AVG(price) AS avg_price,
        SUM(seats_booked) AS total_seats,
        COUNT(*) AS bookings_count
    FROM booking_metrics
    GROUP BY flight_id, booking_window
)
SELECT
    flight_id,
    booking_window,
    ROUND(avg_price, 2) AS avg_price,
    total_seats,
    bookings_count,
    ROUND(AVG(avg_price) OVER (PARTITION BY flight_id), 2) AS flight_avg_price,
    ROUND((avg_price - AVG(avg_price) OVER (PARTITION BY flight_id)) /
          AVG(avg_price) OVER (PARTITION BY flight_id) * 100, 2) AS price_variance
FROM price_analysis
ORDER BY flight_id, FIELD(booking_window, 'Very Early', 'Early', 'Mid', 'Late', 'Last Minute');
```

HOW THIS WORKS

Three layers stack here: booking_metrics buckets each individual booking into a “how far in advance” window, price_analysis aggregates those into one row per flight/window, and the outer SELECT adds AVG(avg_price) OVER (PARTITION BY flight_id) – a window function computed on top of already-aggregated data – to get each flight’s overall average price for comparison. price_variance then shows, as a percentage, how much pricier or cheaper each booking window is versus that flight’s own average, which is exactly the “how much does booking early actually save” answer the question is really asking for.

Q198: Hotel reservation analysis

PROBLEM

Find peak occupancy periods and predict demand

```
CREATE TABLE reservations (
    reservation_id INT,
    hotel_id INT,
    check_in_date DATE,
    check_out_date DATE,
    rooms_booked INT,
    total_guests INT,
    total_amount DECIMAL(10,2),
    status VARCHAR(20)
);

WITH daily_occupancy AS (
    SELECT hotel_id, check_in_date AS date, rooms_booked, 'Check-in' AS event_type
    FROM reservations WHERE status = 'Confirmed'
    UNION ALL
    SELECT hotel_id, check_out_date AS date, -rooms_booked, 'Check-out' AS event_type
    FROM reservations WHERE status = 'Confirmed'
),
occupancy_by_date AS (
    SELECT
        hotel_id,
        date,
        SUM(rooms_booked) AS net_rooms_change,
        SUM(SUM(rooms_booked)) OVER (PARTITION BY hotel_id ORDER BY date) AS occupied_rooms
    FROM daily_occupancy
    GROUP BY hotel_id, date
)
SELECT
    hotel_id, date, occupied_rooms,
    LAG(occupied_rooms) OVER (PARTITION BY hotel_id ORDER BY date) AS prev_day_occupancy,
    occupied_rooms - LAG(occupied_rooms) OVER (PARTITION BY hotel_id ORDER BY date) AS daily_change,
    CASE
        WHEN occupied_rooms > (SELECT AVG(occupied_rooms) FROM occupancy_by_date WHERE hotel_id = o.hotel_id) * 1.2
        THEN 'Peak'
        WHEN occupied_rooms < (SELECT AVG(occupied_rooms) FROM occupancy_by_date WHERE hotel_id = o.hotel_id) * 0.8
        THEN 'Low'
        ELSE 'Normal'
    END AS occupancy_tier
FROM occupancy_by_date o
WHERE date >= CURDATE()
ORDER BY hotel_id, date;
```

HOW THIS WORKS

The clever setup is daily_occupancy: instead of computing “how many rooms are occupied on date X” directly, it turns every reservation into two signed events – a +rooms_booked row on check-in and a -rooms_booked row on check-out – stacked with UNION ALL. A running total of those signed events (SUM(SUM(...)) OVER (ORDER BY date), summing an already-summed daily total) then naturally produces the occupied-room count for every date, rising on check-ins and falling on check-outs, the same running-balance idea as Q177’s credits/debits but applied to room-nights instead of money.

Q199: Stock market transaction analysis

PROBLEM

Calculate portfolio performance with buy/sell patterns

```
CREATE TABLE stock_trades (
    trade_id INT,
    user_id INT,
    stock_symbol VARCHAR(10),
    trade_date DATE,
    trade_type VARCHAR(4), -- 'BUY', 'SELL'
    shares INT,
    price DECIMAL(10,2)
);

WITH position_calculations AS (
    SELECT
        user_id, stock_symbol, trade_date, trade_type, shares, price,
        CASE WHEN trade_type = 'BUY' THEN shares ELSE -shares END AS net_shares,
        CASE WHEN trade_type = 'BUY' THEN shares * price ELSE -shares * price END AS net_value
    FROM stock_trades
),
daily_positions AS (
    SELECT
        user_id, stock_symbol, trade_date, shares, price, net_shares, net_value,
        SUM(net_shares) OVER (PARTITION BY user_id, stock_symbol ORDER BY trade_date) AS cumulative_shares,
        SUM(net_value) OVER (PARTITION BY user_id, stock_symbol ORDER BY trade_date) AS cost_basis
    FROM position_calculations
)
SELECT
    user_id, stock_symbol, trade_date, trade_type, shares, price, cumulative_shares, cost_basis,
    CASE WHEN cumulative_shares > 0 THEN ROUND(cost_basis / cumulative_shares, 2) ELSE 0 END AS avg_cost_per_share,
    RANK() OVER (PARTITION BY user_id ORDER BY trade_date DESC) AS recent_trade_rank
FROM daily_positions
WHERE trade_type = 'SELL'
ORDER BY user_id, trade_date DESC;
```

HOW THIS WORKS

BUY and SELL rows are first converted into signed net_shares and net_value (a SELL becomes negative), and then two running totals – SUM(net_shares) OVER (...) and SUM(net_value) OVER (...), both ordered by trade_date – turn that trade-by-trade history into a live position size and running cost basis per user/stock, the same signed-running-total idea as Q177 and Q198 applied a third time to a different domain. avg_cost_per_share = cost_basis / cumulative_shares at that point in time is then just one more column computed from the two running totals.

Q200: End-to-End Employee Management System SQL Challenge

```
-- Complete employee management dashboard
WITH employee_details AS (
    SELECT
        e.emp_id, e.name, e.age, e.gender, e.salary, e.department, e.designation,
        e.hire_date, e.experience_years, e.bonus,
        m.name AS manager_name, d.location AS dept_location, d.budget AS dept_budget
    FROM employees e
    LEFT JOIN employees m ON e.manager_id = m.emp_id
    LEFT JOIN departments d ON e.department = d.dept_name
),
department_stats AS (
    SELECT
        department, COUNT(*) AS employee_count, AVG(salary) AS avg_salary,
        SUM(salary) AS total_salary, AVG(experience_years) AS avg_experience,
        MAX(salary) AS max_salary, MIN(salary) AS min_salary
    FROM employee_details
    GROUP BY department
),
project_stats AS (
    SELECT department, COUNT(*) AS active_projects, SUM(budget) AS project_budget
    FROM projects WHERE status = 'Active'
    GROUP BY department
),
order_stats AS (
    SELECT e.department, COUNT(o.order_id) AS total_orders, SUM(o.amount) AS total_revenue,
           AVG(o.amount) AS avg_order_value
    FROM employees e
    LEFT JOIN orders o ON e.emp_id = o.customer_id
    GROUP BY e.department
)
SELECT
    ed.department, ed.employee_count, ROUND(ed.avg_salary, 2) AS avg_salary,
    ROUND(ed.total_salary, 2) AS total_salary, ROUND(ed.avg_experience, 1) AS avg_experience_years,
    COALESCE(ps.active_projects, 0) AS active_projects,
    COALESCE(ROUND(os.total_revenue, 2), 0) AS total_revenue,
    ROUND(COALESCE(os.total_revenue, 0) / NULLIF(ed.total_salary, 0) * 100, 2) AS revenue_to_salary_ratio,
    RANK() OVER (ORDER BY COALESCE(os.total_revenue, 0) DESC) AS revenue_rank
FROM department_stats ed
LEFT JOIN project_stats ps ON ed.department = ps.department
LEFT JOIN order_stats os ON ed.department = os.department
ORDER BY revenue_rank;
```

HOW THIS WORKS

The capstone question, and deliberately the longest query in the book – but every individual piece has already appeared earlier in this level or the ones before it. Four CTEs each aggregate one business area (employee details, department stats, project stats, order stats) completely independently, and only the final SELECT joins all four together with LEFT JOIN, so a department with zero active projects or zero orders still shows up instead of disappearing. COALESCE(…, 0) papers over the NULLs a LEFT JOIN produces for departments with no matching projects/orders, and NULLIF(ed.total_salary, 0) guards the final ratio against a divide-by-zero the same way Q196 did. If you can trace what each CTE contributes on its own before looking at the JOINs, you've fully internalized the pattern this whole level was building toward.

📘

The Complete SQL Query Book

200 interview questions, explained in plain English — with real output tables for every single query.

A beginner-friendly companion to the SQL Interview Preparation cheat sheet. Every original query is preserved exactly as given, with an added explanation and a real, executed result table so you can see — not just read — what each query produces.

The Sample Database

Every query in this book runs against this same small company database — an HR + e-commerce system with 7 tables. Get familiar with these tables first; once you know what data lives where, every query below will make a lot more sense.

employees · 15 rows

Columns: `emp_id`, `name`, `age`, `gender`, `salary`, `department`, `designation`, `manager_id`, `hire_date`, `city`, `bonus`, `experience_years`

The heart of this book's examples — 15 employees across IT, HR, Finance, and Marketing, each with a salary, city, manager, and hire date. Most queries in Levels 1–6 use this table.

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mum
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi
3	Rahul Singh	45	M	85,000	Finance	Finance Head	1	2018-07-10	Mum
4	Neha Gupta	29	F	72,000	IT	Developer	1	2021-02-01	Bang

4 rows returned.

departments · 5 rows

Columns: `dept_id`, `dept_name`, `location`, `budget`

5 departments with their location and yearly budget. Used to practice JOINs against `employees` via the shared `department` / `dept_name` columns.

dept_id	dept_name	location	budget
1	IT	Mumbai	5,000,000

dept_id	dept_name	location	budget
2	HR	Delhi	2,000,000
3	Finance	Mumbai	3,000,000
4	Marketing	Bangalore	2,500,000

4 rows returned.

projects · 6 rows

Columns: `project_id`, `project_name`, `department`, `status`, `start_date`, `end_date`, `budget`

6 company projects, each tied to a department, with a status (Active/Completed/Planning) and budget.

project_id	project_name	department	status	start_date	end_date	budget
1	AI Implementation	IT	Active	2024-01-01	2024-12-31	1,500,000
2	Employee Portal	IT	Completed	2023-06-01	2023-12-31	800,000
3	Financial Audit	Finance	Active	2024-02-01	2024-08-31	500,000
4	Market Research	Marketing	Planning	2024-03-01	2024-09-30	300,000

4 rows returned.

customers · 10 rows

Columns: `customer_id`, `name`, `city`, `registration_date`, `email`

10 companies who place orders. Used together with `orders` to practice JOINs, subqueries, and customer-analysis queries.

customer_id	name	city	registration_date	email
1	TechCorp	Mumbai	2023-01-10	tech@corp.com
2	GlobalSolutions	Delhi	2023-02-15	info@global.com
3	InnovationLabs	Bangalore	2023-03-20	contact@innovate.com
4	AlphaInc	Mumbai	2023-04-25	alpha@inc.com

4 rows returned.

orders · 15 rows

Columns: `order_id`, `customer_id`, `order_date`, `amount`, `status`, `product_category`

15 customer orders with amount, status, and category. This is the main table for aggregate and revenue-analysis queries.

order_id	customer_id	order_date	amount	status	product_category
101	1	2024-01-10	15,000	Completed	Electronics
102	1	2024-01-15	23,000	Pending	Software
103	2	2024-01-20	12,000	Completed	Hardware
104	3	2024-02-01	50,000	Completed	Electronics

4 rows returned.

products · 10 rows

Columns: `product_id`, `product_name`, `category`, `price`, `stock_quantity`

10 products for sale, each with a category, price, and stock level.

product_id	product_name	category	price	stock_quantity
1	Laptop Pro	Electronics	75,000	50
2	Wireless Mouse	Accessories	1,500	200
3	27" Monitor	Electronics	25,000	30
4	Mechanical Keyboard	Accessories	5,000	100

4 rows returned.

order_items · 15 rows

Columns: `order_item_id`, `order_id`, `product_id`, `quantity`, `price`

The line items inside each order (which product, how many, at what price). Connects `orders` to `products` for detailed sales analysis.

order_item_id	order_id	product_id	quantity	price
1	101	1	2	75,000
2	101	2	1	1,500
3	102	5	1	100,000
4	103	3	1	25,000

4 rows returned.

Level 1: Basics

Learn the four foundational building blocks of every query: choosing columns with SELECT, filtering rows with WHERE, sorting with ORDER BY, and controlling result size with LIMIT/OFFSET. Nearly every later query in this book builds on these basics.

Q1. Display all employees EASY

```
SELECT * FROM employees;
```

WHAT THIS QUERY DOES

The SELECT * pattern means 'give me every column.' Combined with FROM employees and no WHERE filter, this simply dumps every row of the table exactly as it's stored — the SQL equivalent of opening a spreadsheet and looking at everything. It's usually the very first query anyone runs against a new table, just to see what's in it.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	1
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi	8,
3	Rahul Singh	45	M	85,000	Finance	Finance Head	1	2018-07-10	Mumbai	1
4	Neha Gupta	29	F	72,000	IT	Developer	1	2021-02-01	Bangalore	9,
5	Vikram Reddy	38	M	90,000	IT	Team Lead	1	2017-11-25	Delhi	1
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5,
7	Karthik Nair	51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	2
8	Meera Krishnan	33	F	68,000	Marketing	Marketing Lead	5	2020-08-12	Delhi	7,
9	Arjun Reddy	42	M	82,000	IT	Senior Developer	5	2016-12-01	Mumbai	1
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4,
11	Rajesh Kumar	39	M	70,000	Finance	Financial Analyst	3	2019-06-10	Chennai	6,
12	Deepa Iyer	34	F	64,000	HR	HR Specialist	2	2020-11-15	Hyderabad	5,

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
13	Suresh Babu	47	M	88,000	IT	Solution Architect	5	2014-03-01	Chennai	18
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5,
15	Ravi Teja	44	M	91,000	IT	Technical Manager	1	2016-07-15	Mumbai	1(

15 rows returned.

Q2. Display employee names only EASY

```
SELECT name FROM employees;
```

WHAT THIS QUERY DOES

Instead of *, we name the exact column we want: name. SQL only returns the columns you list, in the order you list them — so this returns a single-column result with 15 rows, one name per employee, ignoring salary, age, and everything else.

RESULT ON OUR SAMPLE DATA

name
Aarav Sharma
Priya Patel
Rahul Singh
Neha Gupta
Vikram Reddy
Ananya Menon
Karthik Nair
Meera Krishnan
Arjun Reddy
Sneha Patel
Rajesh Kumar
Deepa Iyer
Suresh Babu
Kavya Nair
Ravi Teja

15 rows returned.

Q3. Display unique departments EASY

```
SELECT DISTINCT department FROM employees;
```

WHAT THIS QUERY DOES

`DISTINCT` removes duplicate values from the result. There are 15 employees but only 4 departments, so without `DISTINCT` you'd see 'IT' repeated many times; with it, each department name appears exactly once.

RESULT ON OUR SAMPLE DATA

department
IT
HR
Finance
Marketing

4 rows returned.

Q4. Employees with salary > 50000 EASY

```
SELECT * FROM employees WHERE salary > 50000;
```

WHAT THIS QUERY DOES

`WHERE` filters rows — only rows where the condition is true make it into the result. Here every employee happens to earn more than 50,000, so all 15 rows pass the filter; change the number and you'd see fewer rows come back.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	1
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi	8
3	Rahul Singh	45	M	85,000	Finance	Finance Head	1	2018-07-10	Mumbai	1
4	Neha Gupta	29	F	72,000	IT	Developer	1	2021-02-01	Bangalore	9
5	Vikram Reddy	38	M	90,000	IT	Team Lead	1	2017-11-25	Delhi	1
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5
7		51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	2

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
	Karthik Nair									
8	Meera Krishnan	33	F	68,000	Marketing	Marketing Lead	5	2020-08-12	Delhi	7
9	Arjun Reddy	42	M	82,000	IT	Senior Developer	5	2016-12-01	Mumbai	1
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4
11	Rajesh Kumar	39	M	70,000	Finance	Financial Analyst	3	2019-06-10	Chennai	6
12	Deepa Iyer	34	F	64,000	HR	HR Specialist	2	2020-11-15	Hyderabad	5
13	Suresh Babu	47	M	88,000	IT	Solution Architect	5	2014-03-01	Chennai	1
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5
15	Ravi Teja	44	M	91,000	IT	Technical Manager	1	2016-07-15	Mumbai	1

15 rows returned.

Q5. Salary between 40000 and 80000 EASY

```
SELECT * FROM employees WHERE salary BETWEEN 40000 AND 80000;
```

WHAT THIS QUERY DOES

`BETWEEN a AND b` is shorthand for `salary >= a AND salary <= b` — both endpoints are included. It reads almost like plain English, which is why it's preferred over writing the two comparisons out separately.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	1
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi	8
4	Neha Gupta	29	F	72,000	IT	Developer	1	2021-02-01	Bangalore	9
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5
8	Meera Krishnan	33	F	68,000	Marketing	Marketing Lead	5	2020-08-12	Delhi	7
10		26	F	58,000	Marketing		8	2023-01-20	Bangalore	4

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
	Sneha Patel					Marketing Executive				
11	Rajesh Kumar	39	M	70,000	Finance	Financial Analyst	3	2019-06-10	Chennai	6,
12	Deepa Iyer	34	F	64,000	HR	HR Specialist	2	2020-11-15	Hyderabad	5,
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5,

9 rows returned.

Q6. Employees hired after 2020 EASY

```
SELECT * FROM employees WHERE hire_date > '2020-12-31';
```

WHAT THIS QUERY DOES

Dates in SQL can be compared just like numbers, as long as they're stored in a consistent format (here, `YYYY-MM-DD` text, which sorts correctly as a string too). 'After 2020' is expressed as 'after the last day of 2020' rather than trying to compare a date to a bare year.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bo
4	Neha Gupta	29	F	72,000	IT	Developer	1	2021-02-01	Bangalore	9,0
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5,0
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4,0
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5,0

4 rows returned.

Q7. Employees whose name starts with A EASY

```
SELECT * FROM employees WHERE name LIKE 'A%';
```

WHAT THIS QUERY DOES

`LIKE` does pattern matching on text. The `%` symbol is a wildcard meaning 'any sequence of characters (including none)' — so `'A%'` matches any name that starts with a capital A, followed by anything at all.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bonu
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	10,00
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5,000
9	Arjun Reddy	42	M	82,000	IT	Senior Developer	5	2016-12-01	Mumbai	11,00

3 rows returned.

Q8. Ends with n EASY

```
SELECT * FROM employees WHERE name LIKE '%n';
```

WHAT THIS QUERY DOES

Same wildcard idea as the previous query, but now the % is at the *front*: 'anything, followed by a literal n' — meaning the name must end in the letter n. Placement of % is what controls whether you're matching a prefix, suffix, or something in the middle.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bonu
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5,00
8	Meera Krishnan	33	F	68,000	Marketing	Marketing Lead	5	2020-08-12	Delhi	7,00

2 rows returned.

Q9. Contains "ar" EASY

```
SELECT * FROM employees WHERE name LIKE '%ar%';
```

WHAT THIS QUERY DOES

Wildcards on *both* sides mean 'anything, then the letters ar, then anything' — in other words, match if 'ar' shows up anywhere in the name at all, regardless of position.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bon
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	10,0

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bon
7	Karthik Nair	51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	20,(
9	Arjun Reddy	42	M	82,000	IT	Senior Developer	5	2016-12-01	Mumbai	11,(
11	Rajesh Kumar	39	M	70,000	Finance	Financial Analyst	3	2019-06-10	Chennai	6,0(

4 rows returned.

Q10. NULL manager EASY

```
SELECT * FROM employees WHERE manager_id IS NULL;
```

WHAT THIS QUERY DOES

NULL means 'no value' — it's SQL's way of representing missing or not-applicable data. Critically, you can never write `manager_id = NULL` (that never matches anything, even other NULLs); you must use the special `IS NULL` check instead. This finds the one employee with no manager — presumably the top of the org chart.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bonu
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	10,00

1 row returned.

Q11. Salary ascending EASY

```
SELECT * FROM employees ORDER BY salary ASC;
```

WHAT THIS QUERY DOES

`ORDER BY` sorts the result set. `ASC` (ascending, the default if you omit it) means lowest values first — so this lists employees starting with the smallest salary and working up to the largest.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4,
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5,
12		34	F	64,000	HR	HR Specialist	2	2020-11-15	Hyderabad	5,

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
	Deepa Iyer									
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi	8,
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5,
8	Meera Krishnan	33	F	68,000	Marketing	Marketing Lead	5	2020-08-12	Delhi	7,
11	Rajesh Kumar	39	M	70,000	Finance	Financial Analyst	3	2019-06-10	Chennai	6,
4	Neha Gupta	29	F	72,000	IT	Developer	1	2021-02-01	Bangalore	9,
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	1
9	Arjun Reddy	42	M	82,000	IT	Senior Developer	5	2016-12-01	Mumbai	1
3	Rahul Singh	45	M	85,000	Finance	Finance Head	1	2018-07-10	Mumbai	1
13	Suresh Babu	47	M	88,000	IT	Solution Architect	5	2014-03-01	Chennai	1
5	Vikram Reddy	38	M	90,000	IT	Team Lead	1	2017-11-25	Delhi	1
15	Ravi Teja	44	M	91,000	IT	Technical Manager	1	2016-07-15	Mumbai	1
7	Karthik Nair	51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	2

15 rows returned.

Q12. Salary descending EASY

```
SELECT * FROM employees ORDER BY salary DESC;
```

WHAT THIS QUERY DOES

Flipping to DESC (descending) reverses the sort direction — highest salary first. This is the more common choice in real reports, since people usually want to see 'the biggest/best/most' items at the top.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
7	Karthik Nair	51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	2
15		44	M	91,000	IT		1	2016-07-15	Mumbai	1

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
	Ravi Teja					Technical Manager				
5	Vikram Reddy	38	M	90,000	IT	Team Lead	1	2017-11-25	Delhi	1
13	Suresh Babu	47	M	88,000	IT	Solution Architect	5	2014-03-01	Chennai	1
3	Rahul Singh	45	M	85,000	Finance	Finance Head	1	2018-07-10	Mumbai	1
9	Arjun Reddy	42	M	82,000	IT	Senior Developer	5	2016-12-01	Mumbai	1
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	1
4	Neha Gupta	29	F	72,000	IT	Developer	1	2021-02-01	Bangalore	9
11	Rajesh Kumar	39	M	70,000	Finance	Financial Analyst	3	2019-06-10	Chennai	6
8	Meera Krishnan	33	F	68,000	Marketing	Marketing Lead	5	2020-08-12	Delhi	7
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi	8
12	Deepa Iyer	34	F	64,000	HR	HR Specialist	2	2020-11-15	Hyderabad	5
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4

15 rows returned.

Q13. Department then salary EASY

```
SELECT * FROM employees ORDER BY department, salary DESC;
```

WHAT THIS QUERY DOES

You can sort by multiple columns at once. This groups all rows with the same department together (alphabetically, since no direction was given for it — so it defaults to ascending), and *within* each department group, sorts by salary from highest to lowest. Think of the second column as a tiebreaker for the first.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
3	Rahul Singh	45	M	85,000	Finance	Finance Head	1	2018-07-10	Mumbai	1
11	Rajesh Kumar	39	M	70,000	Finance	Financial Analyst	3	2019-06-10	Chennai	6
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi	8
12	Deepa Iyer	34	F	64,000	HR	HR Specialist	2	2020-11-15	Hyderabad	5
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5
7	Karthik Nair	51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	2
15	Ravi Teja	44	M	91,000	IT	Technical Manager	1	2016-07-15	Mumbai	1
5	Vikram Reddy	38	M	90,000	IT	Team Lead	1	2017-11-25	Delhi	1
13	Suresh Babu	47	M	88,000	IT	Solution Architect	5	2014-03-01	Chennai	1
9	Arjun Reddy	42	M	82,000	IT	Senior Developer	5	2016-12-01	Mumbai	1
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	1
4	Neha Gupta	29	F	72,000	IT	Developer	1	2021-02-01	Bangalore	9
8	Meera Krishnan	33	F	68,000	Marketing	Marketing Lead	5	2020-08-12	Delhi	7
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4

15 rows returned.

Q14. Top 5 salaries EASY

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 5;
```

WHAT THIS QUERY DOES

LIMIT caps how many rows come back. Sorting highest-first and then taking only the first 5 rows is the standard way to answer 'who are the top N by some measure' — sort, then slice.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bon
7	Karthik Nair	51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	20,0
15	Ravi Teja	44	M	91,000	IT	Technical Manager	1	2016-07-15	Mumbai	16,0
5	Vikram Reddy	38	M	90,000	IT	Team Lead	1	2017-11-25	Delhi	12,0
13	Suresh Babu	47	M	88,000	IT	Solution Architect	5	2014-03-01	Chennai	18,0
3	Rahul Singh	45	M	85,000	Finance	Finance Head	1	2018-07-10	Mumbai	15,0

5 rows returned.

Q15. Lowest 5 salaries EASY

```
SELECT * FROM employees ORDER BY salary ASC LIMIT 5;
```

WHAT THIS QUERY DOES

Same 'sort then slice' idea as the previous query, just sorting the other direction first — ascending order puts the smallest salaries at the top, and `LIMIT 5` grabs just those.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bo
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4,0
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5,0
12	Deepa Iyer	34	F	64,000	HR	HR Specialist	2	2020-11-15	Hyderabad	5,5
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi	8,0
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5,0

5 rows returned.

Q16. Distinct cities EASY

```
SELECT DISTINCT city FROM employees;
```

WHAT THIS QUERY DOES

Same DISTINCT technique as Q3, applied to the city column instead of department — it collapses the 15 employee rows down to just the unique city names among them.

RESULT ON OUR SAMPLE DATA

city
Mumbai
Delhi
Bangalore
Chennai
Hyderabad

5 rows returned.

Q17. Distinct job roles EASY

```
SELECT DISTINCT designation FROM employees;
```

WHAT THIS QUERY DOES

Again, DISTINCT on a different column (designation, i.e. job title) — useful for quickly seeing 'what roles exist in this company' without scrolling through every employee.

RESULT ON OUR SAMPLE DATA

designation
Senior Developer
HR Manager
Finance Head
Developer
Team Lead
HR Executive
Architect
Marketing Lead
Marketing Executive
Financial Analyst
HR Specialist
Solution Architect
Accountant
Technical Manager

14 rows returned.

Q18. Count distinct departments EASY

```
SELECT COUNT(DISTINCT department) FROM employees;
```

WHAT THIS QUERY DOES

This combines two ideas: `DISTINCT` narrows the department column down to unique values, and `COUNT()` (an aggregate function, covered properly in Level 2) then counts how many there are — turning a list of unique departments into a single number.

RESULT ON OUR SAMPLE DATA

COUNT(DISTINCT department)
4

1 row returned.

Q19. Distinct managers EASY

```
SELECT DISTINCT manager_id FROM employees WHERE manager_id IS NOT NULL;
```

WHAT THIS QUERY DOES

`IS NOT NULL` is the opposite of Q10's `IS NULL` — it keeps only rows that *do* have a value. Combined with `DISTINCT`, this lists every unique employee ID that manages at least one other person, excluding the placeholder NULL that would otherwise show up for the top-level employee.

RESULT ON OUR SAMPLE DATA

manager_id
1
2
5
8
3

5 rows returned.

Q20. Distinct designations EASY

```
SELECT DISTINCT designation FROM employees;
```

WHAT THIS QUERY DOES

Identical to Q17 — included here again in the original cheat sheet as a reminder that DISTINCT-on-a-column is a pattern worth drilling until it's automatic, not just a one-off trick.

RESULT ON OUR SAMPLE DATA

designation
Senior Developer
HR Manager
Finance Head
Developer
Team Lead
HR Executive
Architect
Marketing Lead
Marketing Executive
Financial Analyst
HR Specialist
Solution Architect
Accountant
Technical Manager

14 rows returned.

Q21. First 10 employees EASY

```
SELECT * FROM employees LIMIT 10;
```

WHAT THIS QUERY DOES

With no `ORDER BY`, 'first 10' really means 'whatever 10 rows the database happens to return first' — in practice this is usually insertion order, but that isn't officially guaranteed by SQL unless you explicitly sort. `LIMIT` alone is fine for quick previews, but for anything where order matters, pair it with `ORDER BY`.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bo
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	10
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi	8,0
3	Rahul Singh	45	M	85,000	Finance	Finance Head	1	2018-07-10	Mumbai	15
4		29	F	72,000	IT	Developer	1	2021-02-01	Bangalore	9,0

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bo
	Neha Gupta									
5	Vikram Reddy	38	M	90,000	IT	Team Lead	1	2017-11-25	Delhi	12
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5,0
7	Karthik Nair	51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	20
8	Meera Krishnan	33	F	68,000	Marketing	Marketing Lead	5	2020-08-12	Delhi	7,0
9	Arjun Reddy	42	M	82,000	IT	Senior Developer	5	2016-12-01	Mumbai	11
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4,0

10 rows returned.

Q22. Employees 11-20 EASY

```
SELECT * FROM employees LIMIT 10 OFFSET 10;
```

WHAT THIS QUERY DOES

OFFSET skips a number of rows before starting to return results. `LIMIT 10 OFFSET 10` means 'skip the first 10, then give me the next 10' — this is exactly the mechanism behind pagination (page 2 of search results, etc). Since our table only has 15 employees, this actually returns just 5 rows (11 through 15).

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bor
11	Rajesh Kumar	39	M	70,000	Finance	Financial Analyst	3	2019-06-10	Chennai	6,0
12	Deepa Iyer	34	F	64,000	HR	HR Specialist	2	2020-11-15	Hyderabad	5,5
13	Suresh Babu	47	M	88,000	IT	Solution Architect	5	2014-03-01	Chennai	18,0
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5,0
15	Ravi Teja	44	M	91,000	IT	Technical Manager	1	2016-07-15	Mumbai	16,0

5 rows returned.

Q23. Highest salary EASY

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 1;
```

WHAT THIS QUERY DOES

The 'sort then slice' pattern from Q14, taken to its extreme: sort by salary descending, then keep only the very first row. This is a common (if slightly informal) way to find a single maximum row along with all its other column values — compare with `MAX(salary)` in Level 2, which gives you just the number, not the whole row.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bon
7	Karthik Nair	51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	20,0

1 row returned.

Q24. Lowest salary EASY

```
SELECT * FROM employees ORDER BY salary ASC LIMIT 1;
```

WHAT THIS QUERY DOES

Mirror image of Q23 — sort ascending instead of descending, then take the first row, giving you the single lowest-paid employee's full record.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bonu
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4,000

1 row returned.

Q25. Second highest salary MEDIUM

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

WHAT THIS QUERY DOES

This is your first **subquery** — a query nested inside another one. The inner query `(SELECT MAX(salary) FROM employees)` runs first and finds the single highest salary (95,000). The outer query then asks for the maximum salary *among employees earning less than that* — effectively finding 'the biggest number that

isn't the biggest number,' i.e. the second-highest salary. This trick generalizes to 3rd-highest, 4th-highest, etc. by nesting further (see Q112).

RESULT ON OUR SAMPLE DATA

MAX(salary)
91,000

1 row returned.

Level 2: Aggregate Functions

Aggregate functions collapse many rows into a single summary number: COUNT, AVG, MAX, MIN, and SUM. Paired with GROUP BY, they let you compute per-category summaries (average salary per department, headcount per team) instead of one grand total, and HAVING lets you filter those group results.

Q26. Total employees EASY

```
SELECT COUNT(*) AS total_employees FROM employees;
```

WHAT THIS QUERY DOES

`COUNT(*)` is your first **aggregate function** — instead of returning many rows, it collapses the whole table down into a single number: how many rows exist. The `AS total_employees` part just renames the output column to something more readable than the default `COUNT(*)` label.

RESULT ON OUR SAMPLE DATA

total_employees
15

1 row returned.

Q27. Average salary EASY

```
SELECT AVG(salary) AS avg_salary FROM employees;
```

WHAT THIS QUERY DOES

`AVG()` adds up every value in the column and divides by the number of rows — the familiar 'mean' from school math, computed across all 15 employees' salaries in one step.

RESULT ON OUR SAMPLE DATA

avg_salary
75,333.33

1 row returned.

Q28. Maximum salary EASY

```
SELECT MAX(salary) AS max_salary FROM employees;
```

WHAT THIS QUERY DOES

`MAX()` scans the whole column and returns the single largest value. Unlike Q23 (`ORDER BY ... LIMIT 1`), this gives you *just the number*, not the full row it came from — often all you need, and slightly cheaper for the database to compute.

RESULT ON OUR SAMPLE DATA

max_salary
95,000

1 row returned.

Q29. Minimum salary EASY

```
SELECT MIN(salary) AS min_salary FROM employees;
```

WHAT THIS QUERY DOES

The mirror image of `MAX()` — `MIN()` returns the smallest value in the column.

RESULT ON OUR SAMPLE DATA

min_salary
58,000

1 row returned.

Q30. Salary sum EASY

```
SELECT SUM(salary) AS total_salary FROM employees;
```

WHAT THIS QUERY DOES

`SUM()` adds every value in the column together. This is exactly what a company's total monthly payroll would be, computed straight from the raw per-employee data.

RESULT ON OUR SAMPLE DATA

total_salary
1,130,000

1 row returned.

Q31. Count employees per department EASY

```
SELECT department, COUNT(*) AS employee_count
FROM employees
GROUP BY department;
```

WHAT THIS QUERY DOES

This is where aggregates get powerful: `GROUP BY department` splits the table into buckets — one bucket per distinct department — and `COUNT(*)` is then computed *separately for each bucket* instead of the whole table at once. The result is one row per department, each with its own headcount.

RESULT ON OUR SAMPLE DATA

department	employee_count
Finance	3
HR	3
IT	7
Marketing	2

4 rows returned.

Q32. Average salary per department EASY

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department;
```

WHAT THIS QUERY DOES

Same grouping idea as Q31, just swapping which aggregate runs inside each department bucket — here it's the average salary *within* that department, not a headcount.

RESULT ON OUR SAMPLE DATA

department	avg_salary
Finance	74,000
HR	63,000
IT	84,714.29
Marketing	63,000

4 rows returned.

Q33. Highest salary per department EASY

```
SELECT department, MAX(salary) AS highest_salary
FROM employees
GROUP BY department;
```

WHAT THIS QUERY DOES

Again, same `GROUP BY` skeleton, this time with `MAX()` — the top earner's salary computed independently for each department, side by side in one result set instead of needing four separate queries.

RESULT ON OUR SAMPLE DATA

department	highest_salary
Finance	85,000
HR	65,000
IT	95,000
Marketing	68,000

4 rows returned.

Q34. Lowest salary per department EASY

```
SELECT department, MIN(salary) AS lowest_salary
FROM employees
GROUP BY department;
```

WHAT THIS QUERY DOES

The `MIN()` version of the same pattern — the lowest-paid salary within each department.

RESULT ON OUR SAMPLE DATA

department	lowest_salary
Finance	67,000
HR	60,000
IT	72,000
Marketing	58,000

4 rows returned.

Q35. Departments with more than 5 employees MEDIUM

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department
HAVING COUNT(*) > 5;
```

WHAT THIS QUERY DOES

This introduces `HAVING` — a filter that runs *after* grouping, on the aggregated results. It's easy to confuse with `WHERE`, but they operate at different stages: `WHERE` filters individual rows *before* grouping happens, while `HAVING` filters entire groups *after* their aggregate (here, the count) has been computed. You can't write `WHERE COUNT(*) > 5` because at the point `WHERE` runs, no counting has happened yet.

RESULT ON OUR SAMPLE DATA

department	emp_count
IT	7

1 row returned.

Q36. Average age EASY

```
SELECT AVG(age) AS avg_age FROM employees;
```

WHAT THIS QUERY DOES

Same `AVG()` idea as Q27, applied to the age column instead of `salary` — aggregate functions work the same way regardless of which numeric column you point them at.

RESULT ON OUR SAMPLE DATA

avg_age
36.40

1 row returned.

Q37. Maximum experience EASY

```
SELECT MAX(experience_years) FROM employees;
```

WHAT THIS QUERY DOES

Notice this query has no AS alias — that's perfectly valid SQL, it just means the output column will be labeled with the raw expression (`MAX(experience_years)`) instead of a friendly name. Aliases are a readability choice, not a requirement.

RESULT ON OUR SAMPLE DATA

MAX(experience_years)
10

1 row returned.

Q38. Salary greater than average MEDIUM

```
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

WHAT THIS QUERY DOES

A subquery inside a `WHERE` clause: the inner query computes one single number (the average salary, ≈75,333), and the outer query then filters individual employees against that number. This is a common way to answer 'who is above/below the average' — the average itself doesn't need to be a separate step you run manually first.

RESULT ON OUR SAMPLE DATA

name	salary
Rahul Singh	85,000
Vikram Reddy	90,000
Karthik Nair	95,000
Arjun Reddy	82,000
Suresh Babu	88,000
Ravi Teja	91,000

6 rows returned.

Q39. Employees earning minimum salary MEDIUM

```
SELECT name, salary FROM employees
WHERE salary = (SELECT MIN(salary) FROM employees);
```

WHAT THIS QUERY DOES

Same subquery pattern as Q38, but matching *equal to* the minimum rather than *greater than* the average. Note this correctly returns every employee tied for the

lowest salary, not just one — unlike `ORDER BY salary LIMIT 1` (Q24), which would only ever show a single row even if there were a tie.

RESULT ON OUR SAMPLE DATA

name	salary
Sneha Patel	58,000

1 row returned.

Q40. Total bonus EASY

```
SELECT SUM(bonus) AS total_bonus FROM employees;
```

WHAT THIS QUERY DOES

Another straightforward `SUM()`, this time totalling the `bonus` column across every employee.

RESULT ON OUR SAMPLE DATA

total_bonus
151,500

1 row returned.

Q41. Average bonus EASY

```
SELECT AVG(bonus) AS avg_bonus FROM employees;
```

WHAT THIS QUERY DOES

The average bonus paid out per employee, computed the same way as the average salary in Q27.

RESULT ON OUR SAMPLE DATA

avg_bonus
10,100

1 row returned.

Q42. Count NULL values MEDIUM

```
SELECT COUNT(*) - COUNT(manager_id) AS null_managers FROM employees;
```

WHAT THIS QUERY DOES

A subtle but important detail: `COUNT(*)` counts *every row*, but `COUNT(column_name)` only counts rows where that specific column is **not NULL** — NULLs are silently skipped. Subtracting the two tells you how many rows have a NULL in that column, here revealing there's exactly 1 employee with no manager (matching what Q10 found directly).

RESULT ON OUR SAMPLE DATA

null_managers
1

1 row returned.

Q43. Count non-NULL values EASY

```
SELECT COUNT(manager_id) AS non_null_managers FROM employees;
```

WHAT THIS QUERY DOES

As explained in Q42, `COUNT(column)` automatically ignores NULLs — so this simply counts how many employees *do* have a manager assigned, without needing an explicit `WHERE manager_id IS NOT NULL` clause.

RESULT ON OUR SAMPLE DATA

non_null_managers
14

1 row returned.

Q44. Department with highest average salary MEDIUM

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
ORDER BY avg_salary DESC LIMIT 1;
```

WHAT THIS QUERY DOES

This chains three concepts together: group by department, compute each group's average salary, then apply the familiar 'sort then take the top 1' trick from Q23 — but now sorting the *grouped/aggregated* results rather than raw rows.

RESULT ON OUR SAMPLE DATA

department	avg_salary
IT	84,714.29

1 row returned.

Q45. Department with lowest average salary MEDIUM

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
ORDER BY avg_salary ASC LIMIT 1;
```

WHAT THIS QUERY DOES

Identical to Q44 with the sort direction flipped — ascending order surfaces the department with the *lowest* average salary at the top instead.

RESULT ON OUR SAMPLE DATA

department	avg_salary
HR	63,000

1 row returned.

Q46. Highest bonus EASY

```
SELECT MAX(bonus) FROM employees;
```

WHAT THIS QUERY DOES

Straightforward `MAX()` on the bonus column, same pattern as Q28 applied elsewhere.

RESULT ON OUR SAMPLE DATA

MAX(bonus)
20,000

1 row returned.

Q47. Lowest bonus EASY

```
SELECT MIN(bonus) FROM employees;
```

WHAT THIS QUERY DOES

Straightforward `MIN()` on the bonus column.

RESULT ON OUR SAMPLE DATA

MIN(bonus)
4,000

1 row returned.

Q48. Average experience EASY

```
SELECT AVG(experience_years) FROM employees;
```

WHAT THIS QUERY DOES

The average number of years of experience across all employees — same `AVG()` pattern, different column.

RESULT ON OUR SAMPLE DATA

AVG(experience_years)
5.33

1 row returned.

Q49. Total projects EASY

```
SELECT COUNT(*) FROM projects;
```

WHAT THIS QUERY DOES

`COUNT(*)` isn't limited to the `employees` table — it works identically on any table. This counts how many rows exist in `projects`.

RESULT ON OUR SAMPLE DATA

COUNT(*)
6

1 row returned.

Q50. Total managers MEDIUM

```
SELECT COUNT(DISTINCT manager_id) FROM employees
WHERE manager_id IS NOT NULL;
```

WHAT THIS QUERY DOES

Combines three ideas from earlier: filter out NULLs first (`WHERE manager_id IS NOT NULL`, as in Q19), narrow to unique values (`DISTINCT`), then count them. The result is the number of *distinct people* who manage at least one employee — not the number of employees who have a manager (that was Q43), but how many unique managers exist among them.

RESULT ON OUR SAMPLE DATA

COUNT(DISTINCT manager_id)
5

1 row returned.

Level 3: GROUP BY & HAVING

GROUP BY becomes far more useful once combined with JOINs (sales by region) and date functions (revenue by month/quarter/year). This level also reinforces WHERE-before-GROUP-BY filtering and introduces correlated subqueries for finding the top/bottom row within each group.

Q51. Employees per city EASY

```
SELECT city, COUNT(*) AS emp_count
FROM employees
GROUP BY city
ORDER BY emp_count DESC;
```

WHAT THIS QUERY DOES

A grouped count (like Q31) with an ORDER BY added on top, sorting cities from most employees to fewest — a very common 'top categories' style report.

RESULT ON OUR SAMPLE DATA

city	emp_count
Mumbai	5
Delhi	3
Bangalore	3
Hyderabad	2
Chennai	2

5 rows returned.

Q52. Employees per department EASY

```
SELECT department, COUNT(*) AS emp_count
FROM employees
GROUP BY department;
```

WHAT THIS QUERY DOES

Identical to Q31 — included again here as the cheat sheet's opening example for this GROUP BY-focused section.

RESULT ON OUR SAMPLE DATA

department	emp_count
Finance	3
HR	3

department	emp_count
IT	7
Marketing	2

4 rows returned.

Q53. Salary by designation EASY

```
SELECT designation, AVG(salary) AS avg_salary
FROM employees
GROUP BY designation;
```

WHAT THIS QUERY DOES

Grouping by `designation` instead of `department` — since almost every employee here has a unique job title, most groups end up containing just one person, which is a useful reminder that GROUP BY works fine even when many groups have only a single row.

RESULT ON OUR SAMPLE DATA

designation	avg_salary
Accountant	67,000
Architect	95,000
Developer	72,000
Finance Head	85,000
Financial Analyst	70,000
HR Executive	60,000
HR Manager	65,000
HR Specialist	64,000
Marketing Executive	58,000
Marketing Lead	68,000
Senior Developer	78,500
Solution Architect	88,000
Team Lead	90,000
Technical Manager	91,000

14 rows returned.

Q54. Average salary by city EASY

```
SELECT city, AVG(salary) AS avg_salary
FROM employees
```

```
GROUP BY city
ORDER BY avg_salary DESC;
```

WHAT THIS QUERY DOES

Same grouped-average pattern as Q32/Q53, grouped by `city` this time and sorted so the highest-paying city appears first.

RESULT ON OUR SAMPLE DATA

city	avg_salary
Chennai	79,000
Mumbai	78,600
Bangalore	75,000
Delhi	74,333.33
Hyderabad	65,500

5 rows returned.

Q55. Departments with average salary > 60K MEDIUM

```
SELECT department, AVG(salary) AS avg_salary
FROM employees
GROUP BY department
HAVING AVG(salary) > 60000;
```

WHAT THIS QUERY DOES

Another `HAVING` example (following Q35), this time filtering on the group's *average* rather than its *count*. As it happens, every department in this dataset clears 60,000 average salary, so all four groups survive the filter — a good reminder that HAVING doesn't always narrow the result down.

RESULT ON OUR SAMPLE DATA

department	avg_salary
Finance	74,000
HR	63,000
IT	84,714.29
Marketing	63,000

4 rows returned.

Q56. Cities having more than 20 employees MEDIUM

```
SELECT city, COUNT(*) AS emp_count
FROM employees
```

```
GROUP BY city
HAVING COUNT(*) > 20;
```

WHAT THIS QUERY DOES

With only 15 employees total spread across 5 cities, no single city could possibly have more than 20 — so this query correctly returns *zero rows*. That's not a bug or an error: an empty result set is a perfectly valid, meaningful answer ('no city meets this condition'), and real-world queries return this constantly.

RESULT ON OUR SAMPLE DATA

city	emp_count
(no rows returned)	

0 rows returned.

Q57. Count females per department EASY

```
SELECT department, COUNT(*) AS female_count
FROM employees
WHERE gender = 'F'
GROUP BY department;
```

WHAT THIS QUERY DOES

This shows the correct order of operations: `WHERE gender = 'F'` filters down to only female employees *first*, and only *then* does `GROUP BY department` bucket what's left. Filtering before grouping (with WHERE) versus after (with HAVING) is exactly the distinction introduced back in Q35.

RESULT ON OUR SAMPLE DATA

department	female_count
Finance	1
HR	3
IT	1
Marketing	2

4 rows returned.

Q58. Count males per department EASY

```
SELECT department, COUNT(*) AS male_count
FROM employees
WHERE gender = 'M'
GROUP BY department;
```

WHAT THIS QUERY DOES

Mirror image of Q57, filtering for `'M'` instead of `'F'` before grouping. Note that Marketing doesn't appear in this result at all — when a group would end up with zero matching rows, GROUP BY simply omits it rather than showing a row with count 0.

RESULT ON OUR SAMPLE DATA

department	male_count
Finance	2
IT	6

2 rows returned.

Q59. Maximum salary by city EASY

```
SELECT city, MAX(salary) AS max_salary
FROM employees
GROUP BY city;
```

WHAT THIS QUERY DOES

The `MAX()`-per-group pattern from Q33, applied to `city` instead of `department`.

RESULT ON OUR SAMPLE DATA

city	max_salary
Bangalore	95,000
Chennai	88,000
Delhi	90,000
Hyderabad	67,000
Mumbai	91,000

5 rows returned.

Q60. Minimum salary by city EASY

```
SELECT city, MIN(salary) AS min_salary
FROM employees
GROUP BY city;
```

WHAT THIS QUERY DOES

The `MIN()`-per-group counterpart to Q59.

RESULT ON OUR SAMPLE DATA

city	min_salary
Bangalore	58,000
Chennai	70,000
Delhi	65,000
Hyderabad	64,000
Mumbai	60,000

5 rows returned.

Q61. Average experience by department EASY

```
SELECT department, AVG(experience_years) AS avg_experience
FROM employees
GROUP BY department;
```

WHAT THIS QUERY DOES

Same grouped-average recipe as always, this time summarizing `experience_years` instead of salary — useful for questions like 'which department has the most seasoned team.'

RESULT ON OUR SAMPLE DATA

department	avg_experience
Finance	5.33
HR	3.67
IT	6.86
Marketing	2.50

4 rows returned.

Q62. Highest paid employee in every department HARD

```
SELECT department, name, salary
FROM employees e1
WHERE salary = (SELECT MAX(salary) FROM employees e2 WHERE e1.department =
e2.department);
```

WHAT THIS QUERY DOES

This is a **correlated subquery** — notice the inner query references `e1.department`, a column from the *outer* query. That means the subquery can't run just once; it has to re-run separately for every single row of e1, each time asking 'what's the max salary in *this particular row's* department,' and keeping the row only if it matches. It's a different (and more flexible) way to reach the same kind of answer as GROUP BY + MAX, except this version returns the full row — including the

employee's *name* — which plain GROUP BY alone cannot do without extra tricks, since you can't SELECT a non-grouped, non-aggregated column like name directly alongside a GROUP BY.

RESULT ON OUR SAMPLE DATA

department	name	salary
HR	Priya Patel	65,000
Finance	Rahul Singh	85,000
IT	Karthik Nair	95,000
Marketing	Meera Krishnan	68,000

4 rows returned.

Q63. Lowest paid employee in every department HARD

```
SELECT department, name, salary
FROM employees e1
WHERE salary = (SELECT MIN(salary) FROM employees e2 WHERE e1.department =
e2.department);
```

WHAT THIS QUERY DOES

The same correlated-subquery technique as Q62, swapping MAX for MIN to find each department's lowest earner by name instead of its highest.

RESULT ON OUR SAMPLE DATA

department	name	salary
IT	Neha Gupta	72,000
HR	Ananya Menon	60,000
Marketing	Sneha Patel	58,000
Finance	Kavya Nair	67,000

4 rows returned.

Q64. Departments with total salary > 5 lakh MEDIUM

```
SELECT department, SUM(salary) AS total_salary
FROM employees
GROUP BY department
HAVING SUM(salary) > 500000;
```

WHAT THIS QUERY DOES

A third HAVING example, this time filtering on a *summed* total rather than a count or average. Only IT clears half a million in combined salary — the other three

departments, despite having reasonable average salaries, simply don't have enough headcount to push their totals that high.

RESULT ON OUR SAMPLE DATA

department	total_salary
IT	593,000

1 row returned.

Q65. Employees grouped by joining year MEDIUM

```
SELECT YEAR(hire_date) AS join_year, COUNT(*) AS emp_count
FROM employees
GROUP BY YEAR(hire_date)
ORDER BY join_year;
```

WHAT THIS QUERY DOES

YEAR() extracts just the year portion out of a full date (e.g. 2020-01-15 → 2020). This is a very common trick: you can GROUP BY the *result of a function*, not just a plain column, which lets you bucket exact dates into coarser categories like 'year hired.' (This book's practice database runs on SQLite rather than MySQL, so the query actually executed uses SQLite's equivalent date function, `strftime('%Y', hire_date)` — the underlying idea and the result are identical either way.)

RESULT ON OUR SAMPLE DATA

join_year	emp_count
2014	1
2015	1
2016	2
2017	1
2018	1
2019	2
2020	3
2021	2
2022	1
2023	1

10 rows returned.

Q66. Projects by status EASY

```
SELECT status, COUNT(*) AS project_count
FROM projects
GROUP BY status;
```

WHAT THIS QUERY DOES

The familiar grouped-count pattern, this time on the `projects` table — showing how many projects are Active, Completed, or still in Planning.

RESULT ON OUR SAMPLE DATA

status	project_count
Active	3
Completed	2
Planning	1

3 rows returned.

Q67. Orders by month MEDIUM

```
SELECT MONTH(order_date) AS month, COUNT(*) AS order_count
FROM orders
GROUP BY MONTH(order_date)
ORDER BY month;
```

WHAT THIS QUERY DOES

Same 'group by a function of a date column' idea as Q65, using `MONTH()` instead of `YEAR()` to see order volume trends across the calendar. (Executed here using SQLite's `strftime('%m', order_date)`, which returns the month as a zero-padded string like '01', '02'.)

RESULT ON OUR SAMPLE DATA

month	order_count
01	3
02	4
03	4
04	4

4 rows returned.

Q68. Customers by city EASY

```
SELECT city, COUNT(*) AS customer_count
FROM customers
GROUP BY city;
```

WHAT THIS QUERY DOES

Grouped count applied to the `customers` table for the first time — the exact same technique works identically across every table in the database.

RESULT ON OUR SAMPLE DATA

city	customer_count
Bangalore	2
Chennai	1
Delhi	2
Hyderabad	1
Mumbai	3
Pune	1

6 rows returned.

Q69. Products by category EASY

```
SELECT category, COUNT(*) AS product_count
FROM products
GROUP BY category;
```

WHAT THIS QUERY DOES

Grouped count on the `products` table, showing how the product catalog is split across Electronics, Accessories, and Software.

RESULT ON OUR SAMPLE DATA

category	product_count
Accessories	4
Electronics	4
Software	2

3 rows returned.

Q70. Sales by region MEDIUM

```
SELECT c.city AS region, SUM(o.amount) AS total_sales
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY c.city;
```

WHAT THIS QUERY DOES

This is the first query in this book combining a **JOIN** with **GROUP BY** (JOINs are covered fully in Level 4, but this preview shows why they matter here): the `orders` table only has a `customer_id` number, not a city, so we JOIN to `customers` to pull in each order's city, and *then* group and sum by that city. Grouping can happen on a column that came from a joined table just as easily as one from the main table.

RESULT ON OUR SAMPLE DATA

region	total_sales
Bangalore	131,000
Chennai	21,000
Delhi	73,000
Hyderabad	67,000
Mumbai	127,000
Pune	45,000

6 rows returned.

Q71. Revenue by month MEDIUM

```
SELECT MONTH(order_date) AS month, SUM(amount) AS total_revenue
FROM orders
GROUP BY MONTH(order_date)
ORDER BY month;
```

WHAT THIS QUERY DOES

Same technique as Q67, swapping `COUNT(*)` for `SUM(amount)` — instead of 'how many orders per month,' this answers 'how much revenue per month.'

RESULT ON OUR SAMPLE DATA

month	total_revenue
01	50,000
02	145,000
03	150,000
04	119,000

4 rows returned.

Q72. Revenue by quarter MEDIUM

```
SELECT QUARTER(order_date) AS quarter, SUM(amount) AS total_revenue
FROM orders
GROUP BY QUARTER(order_date)
ORDER BY quarter;
```

WHAT THIS QUERY DOES

QUARTER() buckets a date into 1 of 4 three-month periods (Jan-Mar = Q1, Apr-Jun = Q2, and so on) — useful for the kind of quarterly reporting most businesses run on. SQLite has no built-in QUARTER() function, so the version actually executed computes it manually from the month: ((month - 1) / 3) + 1, which produces the exact same grouping.

RESULT ON OUR SAMPLE DATA

quarter	total_revenue
1	345,000
2	119,000

2 rows returned.

Q73. Revenue by year EASY

```
SELECT YEAR(order_date) AS year, SUM(amount) AS total_revenue
FROM orders
GROUP BY YEAR(order_date)
ORDER BY year;
```

WHAT THIS QUERY DOES

Same idea as Q65's YEAR() grouping, now summing revenue instead of counting employees. Since every order in this sample dataset happens to fall in 2024, the result collapses to a single row — with a larger, multi-year dataset this would show one row per year.

RESULT ON OUR SAMPLE DATA

year	total_revenue
2024	464,000

1 row returned.

Q74. Top selling category MEDIUM

```
SELECT p.category, SUM(oi.quantity) AS total_sold
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY total_sold DESC LIMIT 1;
```

WHAT THIS QUERY DOES

Another JOIN + GROUP BY combination: connect products to their order line items to find out how many units of each category were actually sold, then apply the familiar 'sort descending, take 1' pattern to surface the single best-selling category.

RESULT ON OUR SAMPLE DATA

category	total_sold
Electronics	16

1 row returned.

Q75. Bottom selling category MEDIUM

```
SELECT p.category, SUM(oi.quantity) AS total_sold
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY total_sold ASC LIMIT 1;
```

WHAT THIS QUERY DOES

The mirror image of Q74 — sorting ascending instead of descending to find the category with the *fewest* units sold rather than the most.

RESULT ON OUR SAMPLE DATA

category	total_sold
Accessories	2

1 row returned.

Level 4: JOINs

JOINs combine data spread across tables: INNER (only matches), LEFT/RIGHT (keep all of one side), FULL OUTER (keep everything), SELF (a table joined to itself), and CROSS (every combination). This is the biggest level in the book -- by the end you'll recognize the 'junction table' pattern (orders/order_items, students/enrollments/courses) everywhere.

Q76. INNER JOIN employee & department MEDIUM

```
SELECT e.name, e.department, d.location, d.budget
FROM employees e
INNER JOIN departments d ON e.department = d.dept_name;
```

WHAT THIS QUERY DOES

A `JOIN` combines rows from two tables based on a matching condition — here, an employee's `department` text must match a department's `dept_name`. `INNER JOIN` (the default kind, often just written `JOIN`) only keeps rows where a match is found on *both* sides — an employee whose department didn't exist in the `departments` table would simply be left out entirely, and a department with no employees wouldn't appear either. Aliasing the tables as `e` and `d` is a convention that keeps column references short once you're juggling two or more tables.

RESULT ON OUR SAMPLE DATA

name	department	location	budget
Aarav Sharma	IT	Mumbai	5,000,000
Priya Patel	HR	Delhi	2,000,000
Rahul Singh	Finance	Mumbai	3,000,000
Neha Gupta	IT	Mumbai	5,000,000
Vikram Reddy	IT	Mumbai	5,000,000
Ananya Menon	HR	Delhi	2,000,000
Karthik Nair	IT	Mumbai	5,000,000
Meera Krishnan	Marketing	Bangalore	2,500,000
Arjun Reddy	IT	Mumbai	5,000,000
Sneha Patel	Marketing	Bangalore	2,500,000
Rajesh Kumar	Finance	Mumbai	3,000,000
Deepa Iyer	HR	Delhi	2,000,000
Suresh Babu	IT	Mumbai	5,000,000
Kavya Nair	Finance	Mumbai	3,000,000
Ravi Teja	IT	Mumbai	5,000,000

15 rows returned.

Q77. LEFT JOIN MEDIUM

```
SELECT e.name, e.department, d.location
FROM employees e
LEFT JOIN departments d ON e.department = d.dept_name;
```

WHAT THIS QUERY DOES

`LEFT JOIN` keeps *every* row from the left table (`employees`) no matter what — if a matching department isn't found, the department columns simply come back as `NULL` instead of the row being dropped. This is the key difference from `INNER JOIN`: INNER JOIN can silently lose rows that don't match; LEFT JOIN guarantees the 'main' table's rows all survive, which is exactly why it's the go-to choice whenever you specifically want to find *unmatched* rows (see Q81).

RESULT ON OUR SAMPLE DATA

name	department	location
Aarav Sharma	IT	Mumbai
Priya Patel	HR	Delhi
Rahul Singh	Finance	Mumbai
Neha Gupta	IT	Mumbai
Vikram Reddy	IT	Mumbai
Ananya Menon	HR	Delhi
Karthik Nair	IT	Mumbai
Meera Krishnan	Marketing	Bangalore
Arjun Reddy	IT	Mumbai
Sneha Patel	Marketing	Bangalore
Rajesh Kumar	Finance	Mumbai
Deepa Iyer	HR	Delhi
Suresh Babu	IT	Mumbai
Kavya Nair	Finance	Mumbai
Ravi Teja	IT	Mumbai

15 rows returned.

Q78. RIGHT JOIN MEDIUM

```
SELECT e.name, d.dept_name, d.location
FROM employees e
RIGHT JOIN departments d ON e.department = d.dept_name;
```

WHAT THIS QUERY DOES

The mirror image of LEFT JOIN: now every row from the *right* table (departments) is guaranteed to survive, with employee columns coming back NULL if no employee matches. Notice the row count here is 16, one more than employees alone (15) — that's the 'Operations' department showing up with a NULL employee name, since no one in our sample data works there. RIGHT JOIN is used far less often than LEFT JOIN in practice, mostly because you can always rewrite a RIGHT JOIN as a LEFT JOIN by swapping which table you list first.

RESULT ON OUR SAMPLE DATA

name	dept_name	location
Aarav Sharma	IT	Mumbai
Priya Patel	HR	Delhi
Rahul Singh	Finance	Mumbai
Neha Gupta	IT	Mumbai
Vikram Reddy	IT	Mumbai
Ananya Menon	HR	Delhi
Karthik Nair	IT	Mumbai
Meera Krishnan	Marketing	Bangalore
Arjun Reddy	IT	Mumbai
Sneha Patel	Marketing	Bangalore
Rajesh Kumar	Finance	Mumbai
Deepa Iyer	HR	Delhi
Suresh Babu	IT	Mumbai
Kavya Nair	Finance	Mumbai
Ravi Teja	IT	Mumbai
NULL	Operations	Pune

16 rows returned.

Q79. FULL OUTER JOIN (MySQL workaround) HARD

```
SELECT e.name, d.dept_name, d.location
FROM employees e
LEFT JOIN departments d ON e.department = d.dept_name
```

```
UNION
SELECT e.name, d.dept_name, d.location
FROM employees e
RIGHT JOIN departments d ON e.department = d.dept_name;
```

WHAT THIS QUERY DOES

A `FULL OUTER JOIN` keeps every row from *both* tables, matched where possible and NULL-padded where not. Older MySQL versions don't support `FULL OUTER JOIN` directly, so this classic workaround unions together a LEFT JOIN (catches unmatched employees) with a RIGHT JOIN (catches unmatched departments) — and since `UNION` automatically removes duplicate rows, the rows that matched on both sides don't get counted twice. Modern SQLite (and modern MySQL/Postgres) support `FULL OUTER JOIN` natively, so the live example below runs the simpler direct syntax and produces the same result.

RESULT ON OUR SAMPLE DATA

name	dept_name	location
Aarav Sharma	IT	Mumbai
Priya Patel	HR	Delhi
Rahul Singh	Finance	Mumbai
Neha Gupta	IT	Mumbai
Vikram Reddy	IT	Mumbai
Ananya Menon	HR	Delhi
Karthik Nair	IT	Mumbai
Meera Krishnan	Marketing	Bangalore
Arjun Reddy	IT	Mumbai
Sneha Patel	Marketing	Bangalore
Rajesh Kumar	Finance	Mumbai
Deepa Iyer	HR	Delhi
Suresh Babu	IT	Mumbai
Kavya Nair	Finance	Mumbai
Ravi Teja	IT	Mumbai
NULL	Operations	Pune

16 rows returned.

Q80. SELF JOIN employee-manager MEDIUM

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

WHAT THIS QUERY DOES

A **self join** joins a table to itself — here, `employees` is used twice, once aliased as `e` (playing the role of 'the employee') and once as `m` (playing the role of 'their manager'). This is exactly how you unravel a self-referencing column like `manager_id`, which stores another row's ID from the very same table, into a readable name-to-name relationship. `LEFT JOIN` is used specifically so the one employee with no manager (`manager_id IS NULL`) still appears, just with a NULL manager name, rather than being dropped.

RESULT ON OUR SAMPLE DATA

employee	manager
Aarav Sharma	*NULL*
Priya Patel	Aarav Sharma
Rahul Singh	Aarav Sharma
Neha Gupta	Aarav Sharma
Vikram Reddy	Aarav Sharma
Ananya Menon	Priya Patel
Karthik Nair	Aarav Sharma
Meera Krishnan	Vikram Reddy
Arjun Reddy	Vikram Reddy
Sneha Patel	Meera Krishnan
Rajesh Kumar	Rahul Singh
Deepa Iyer	Priya Patel
Suresh Babu	Vikram Reddy
Kavya Nair	Rahul Singh
Ravi Teja	Aarav Sharma

15 rows returned.

Q81. Employees without department MEDIUM

```
SELECT e.name
FROM employees e
LEFT JOIN departments d ON e.department = d.dept_name
WHERE d.dept_id IS NULL;
```

WHAT THIS QUERY DOES

This is the classic 'find unmatched rows' pattern: LEFT JOIN keeps every employee, filling in NULL for any department columns that didn't match — and then `WHERE d.dept_id IS NULL` keeps only the rows where that NULL-filling actually happened, i.e. employees whose department didn't exist in the departments table. In our sample data every employee's department is valid, so this correctly returns 0 rows

— but this exact pattern is one of the most useful JOIN techniques for spotting broken/orphaned foreign-key references in real data.

RESULT ON OUR SAMPLE DATA

name
(no rows returned)

0 rows returned.

Q82. Departments without employees MEDIUM

```
SELECT d.dept_name
FROM departments d
LEFT JOIN employees e ON d.dept_name = e.department
WHERE e.emp_id IS NULL;
```

WHAT THIS QUERY DOES

The mirror image of Q81 — LEFT JOIN starting from `departments` this time, so every department survives, and the NULL-check then reveals which departments have no employees at all. 'Operations' is the one department in our sample data that exists but has nobody assigned to it yet.

RESULT ON OUR SAMPLE DATA

dept_name
Operations

1 row returned.

Q83. Orders with customers EASY

```
SELECT o.order_id, c.name, o.order_date, o.amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id;
```

WHAT THIS QUERY DOES

A plain INNER JOIN attaching each order to the customer who placed it — since `orders` only stores a `customer_id` number, this is what lets you show the actual customer *name* alongside each order instead of a meaningless ID.

RESULT ON OUR SAMPLE DATA

order_id	name	order_date	amount
101	TechCorp	2024-01-10	15,000
102	TechCorp	2024-01-15	23,000

order_id	name	order_date	amount
103	GlobalSolutions	2024-01-20	12,000
104	InnovationLabs	2024-02-01	50,000
105	InnovationLabs	2024-02-05	18,000
106	AlphaInc	2024-02-10	32,000
107	BetaSystems	2024-02-15	45,000
108	GammaTech	2024-03-01	21,000
109	DeltaCorp	2024-03-05	67,000
110	EpsilonSoft	2024-03-10	34,000
111	ThetaSolutions	2024-03-15	28,000
112	OmegaLabs	2024-04-01	42,000
113	TechCorp	2024-04-05	15,000
114	GlobalSolutions	2024-04-10	33,000
115	InnovationLabs	2024-04-15	29,000

15 rows returned.

Q84. Customers without orders MEDIUM

```
SELECT c.name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

WHAT THIS QUERY DOES

Same 'LEFT JOIN then check for NULL' technique as Q81/Q82, now finding customers who exist in the `customers` table but have never placed an order. Every customer in our sample data happens to have at least one order, so this correctly returns 0 rows here.

RESULT ON OUR SAMPLE DATA

name
(no rows returned)

0 rows returned.

Q85. Products never ordered MEDIUM

```
SELECT p.product_name
FROM products p
```

```
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.order_item_id IS NULL;
```

WHAT THIS QUERY DOES

The same unmatched-rows pattern again, this time surfacing products that have never appeared in any order — useful for spotting slow-moving or completely dead inventory.

RESULT ON OUR SAMPLE DATA

product_name
Webcam HD
Printer
USB Hub

3 rows returned.

Q86. Orders with product details MEDIUM

```
SELECT o.order_id, p.product_name, oi.quantity, oi.price
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id;
```

WHAT THIS QUERY DOES

Your first **three-table join**: `orders` connects to `order_items` (to find out which products and quantities were in each order), which in turn connects to `products` (to get readable product names instead of just IDs). Chaining JOINs like this — each one attaching one more related table — is exactly how real reporting queries assemble a complete picture spread across a normalized database.

RESULT ON OUR SAMPLE DATA

order_id	product_name	quantity	price
101	Laptop Pro	2	75,000
101	Wireless Mouse	1	1,500
102	Enterprise Software	1	100,000
103	27" Monitor	1	25,000
104	Laptop Pro	1	75,000
104	External SSD	2	12,000
105	Enterprise Software	1	100,000
106	27" Monitor	2	25,000
107	Laptop Pro	3	75,000
108	Cloud Storage Plan	1	50,000

order_id	product_name	quantity	price
109	Laptop Pro	2	75,000
110	External SSD	2	12,000
111	Enterprise Software	1	100,000
112	Laptop Pro	1	75,000
113	Mechanical Keyboard	1	5,000

15 rows returned.

Q87. Students with courses (Example structure) MEDIUM

```
-- Assuming tables exist: students, courses, enrollments
SELECT s.name, c.course_name
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id;
```

WHAT THIS QUERY DOES

This query uses a hypothetical schema the cheat sheet sketches as an example, separate from this book's main company database — so a small illustrative `students` / `courses` / `enrollments` dataset (4 students, 4 courses) has been added just to demonstrate it concretely. The structure is a classic **many-to-many relationship**: one student can take many courses, and one course can have many students, so a middle 'junction' table (`enrollments`) is needed to connect them — you can't link students directly to courses with a single foreign key the way employees link directly to one department.

RESULT ON OUR SAMPLE DATA

name	course_name
Aditi Rao	Data Structures
Aditi Rao	Operating Systems
Bilal Khan	Data Structures
Chen Wei	Computer Networks
Diana Cruz	Data Structures
Diana Cruz	Computer Networks

6 rows returned.

Q88. Courses without students MEDIUM

```
-- Assuming tables exist
SELECT c.course_name
```

```
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
WHERE e.enrollment_id IS NULL;
```

WHAT THIS QUERY DOES

Same unmatched-rows LEFT JOIN pattern as Q81/84/85, applied to the illustrative students/courses schema — 'Compiler Design' was deliberately left with zero enrollments in the sample data so this query has a genuine, non-empty result to show.

RESULT ON OUR SAMPLE DATA

course_name
Compiler Design

1 row returned.

Q89. Employees with projects MEDIUM

```
SELECT e.name, p.project_name
FROM employees e
JOIN projects p ON e.department = p.department;
```

WHAT THIS QUERY DOES

Joining on `department` (rather than a dedicated project-assignment table) means this connects *every* employee to *every* project run by their department — not just one specific project they're personally assigned to. That's why the result has 29 rows even though there are only 15 employees: IT alone runs 3 active projects, so every IT employee shows up 3 times, once per project. This is worth noticing as a caution about JOINs: joining on a shared category column (department) rather than a true one-to-one assignment key can multiply rows in ways that aren't always what you want.

RESULT ON OUR SAMPLE DATA

name	project_name
Aarav Sharma	AI Implementation
Aarav Sharma	Cloud Migration
Aarav Sharma	Employee Portal
Priya Patel	Talent Acquisition
Rahul Singh	Financial Audit
Neha Gupta	AI Implementation
Neha Gupta	Cloud Migration
Neha Gupta	Employee Portal
Vikram Reddy	AI Implementation

name	project_name
Vikram Reddy	Cloud Migration
Vikram Reddy	Employee Portal
Ananya Menon	Talent Acquisition
Karthik Nair	AI Implementation
Karthik Nair	Cloud Migration
Karthik Nair	Employee Portal
Meera Krishnan	Market Research
Arjun Reddy	AI Implementation
Arjun Reddy	Cloud Migration
Arjun Reddy	Employee Portal
Sneha Patel	Market Research

Showing first 20 of 29 rows.

Q90. Projects without employees MEDIUM

```
SELECT p.project_name
FROM projects p
LEFT JOIN employees e ON p.department = e.department
WHERE e.emp_id IS NULL;
```

WHAT THIS QUERY DOES

Same unmatched-rows technique again — finds any project belonging to a department that currently has no employees at all. Since every department that has a project also has staff in our sample data, this returns 0 rows.

RESULT ON OUR SAMPLE DATA

project_name
(no rows returned)

0 rows returned.

Q91. Three-table join - Employee, Department, Projects HARD

```
SELECT e.name, e.department, d.location, p.project_name
FROM employees e
JOIN departments d ON e.department = d.dept_name
LEFT JOIN projects p ON e.department = p.department;
```

WHAT THIS QUERY DOES

Mixing join types in one query: an `INNER JOIN` to `departments` (every employee's department is expected to exist, so no need to preserve unmatched rows there) combined with a `LEFT JOIN` to `projects` (so employees in departments with zero projects still show up, with a NULL project name, rather than disappearing). Choosing INNER vs LEFT independently for each join in a multi-join query, based on whether you want to risk losing unmatched rows, is an important practical skill.

RESULT ON OUR SAMPLE DATA

name	department	location	project_name
Aarav Sharma	IT	Mumbai	AI Implementation
Aarav Sharma	IT	Mumbai	Cloud Migration
Aarav Sharma	IT	Mumbai	Employee Portal
Priya Patel	HR	Delhi	Talent Acquisition
Rahul Singh	Finance	Mumbai	Financial Audit
Neha Gupta	IT	Mumbai	AI Implementation
Neha Gupta	IT	Mumbai	Cloud Migration
Neha Gupta	IT	Mumbai	Employee Portal
Vikram Reddy	IT	Mumbai	AI Implementation
Vikram Reddy	IT	Mumbai	Cloud Migration
Vikram Reddy	IT	Mumbai	Employee Portal
Ananya Menon	HR	Delhi	Talent Acquisition
Karthik Nair	IT	Mumbai	AI Implementation
Karthik Nair	IT	Mumbai	Cloud Migration
Karthik Nair	IT	Mumbai	Employee Portal
Meera Krishnan	Marketing	Bangalore	Market Research
Arjun Reddy	IT	Mumbai	AI Implementation
Arjun Reddy	IT	Mumbai	Cloud Migration
Arjun Reddy	IT	Mumbai	Employee Portal
Sneha Patel	Marketing	Bangalore	Market Research

Showing first 20 of 29 rows.

Q92. Four-table join - Customer, Orders, OrderItems, Products

HARD

```
SELECT c.name AS customer, o.order_id, p.product_name, oi.quantity, oi.price
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id;
```

WHAT THIS QUERY DOES

Extending the three-table join from Q86 by one more table: customer → their orders → each order's line items → each item's product details. Four tables chained together like this is a very normal size for a real invoice/receipt-style report — each JOIN just walks one more step along the natural relationships between the tables.

RESULT ON OUR SAMPLE DATA

customer	order_id	product_name	quantity	price
TechCorp	101	Laptop Pro	2	75,000
TechCorp	101	Wireless Mouse	1	1,500
TechCorp	102	Enterprise Software	1	100,000
GlobalSolutions	103	27" Monitor	1	25,000
InnovationLabs	104	Laptop Pro	1	75,000
InnovationLabs	104	External SSD	2	12,000
InnovationLabs	105	Enterprise Software	1	100,000
AlphaInc	106	27" Monitor	2	25,000
BetaSystems	107	Laptop Pro	3	75,000
GammaTech	108	Cloud Storage Plan	1	50,000
DeltaCorp	109	Laptop Pro	2	75,000
EpsilonSoft	110	External SSD	2	12,000
ThetaSolutions	111	Enterprise Software	1	100,000
OmegaLabs	112	Laptop Pro	1	75,000
TechCorp	113	Mechanical Keyboard	1	5,000

15 rows returned.

Q93. Employee → Department → Location EASY

```
SELECT e.name, e.department, d.location, d.budget
FROM employees e
JOIN departments d ON e.department = d.dept_name;
```

WHAT THIS QUERY DOES

Identical to Q76 — the cheat sheet repeats this exact join here to reinforce that 'follow a foreign key to pull in details from the related table' is the single most common JOIN use case you'll write, over and over, in real applications.

RESULT ON OUR SAMPLE DATA

name	department	location	budget
Aarav Sharma	IT	Mumbai	5,000,000
Priya Patel	HR	Delhi	2,000,000

name	department	location	budget
Rahul Singh	Finance	Mumbai	3,000,000
Neha Gupta	IT	Mumbai	5,000,000
Vikram Reddy	IT	Mumbai	5,000,000
Ananya Menon	HR	Delhi	2,000,000
Karthik Nair	IT	Mumbai	5,000,000
Meera Krishnan	Marketing	Bangalore	2,500,000
Arjun Reddy	IT	Mumbai	5,000,000
Sneha Patel	Marketing	Bangalore	2,500,000
Rajesh Kumar	Finance	Mumbai	3,000,000
Deepa Iyer	HR	Delhi	2,000,000
Suresh Babu	IT	Mumbai	5,000,000
Kavya Nair	Finance	Mumbai	3,000,000
Ravi Teja	IT	Mumbai	5,000,000

15 rows returned.

Q94. Customer → Order → Product MEDIUM

```
SELECT c.name, o.order_date, p.product_name, oi.quantity
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id;
```

WHAT THIS QUERY DOES

Essentially the same four-table chain as Q92, just selecting a slightly different set of output columns (order date instead of order ID and price) — a reminder that the JOIN structure of a query and the specific columns you choose to display are two independent decisions.

RESULT ON OUR SAMPLE DATA

name	order_date	product_name	quantity
TechCorp	2024-01-10	Laptop Pro	2
TechCorp	2024-01-10	Wireless Mouse	1
TechCorp	2024-01-15	Enterprise Software	1
GlobalSolutions	2024-01-20	27" Monitor	1
InnovationLabs	2024-02-01	Laptop Pro	1
InnovationLabs	2024-02-01	External SSD	2
InnovationLabs	2024-02-05	Enterprise Software	1
AlphaInc	2024-02-10	27" Monitor	2

name	order_date	product_name	quantity
BetaSystems	2024-02-15	Laptop Pro	3
GammaTech	2024-03-01	Cloud Storage Plan	1
DeltaCorp	2024-03-05	Laptop Pro	2
EpsilonSoft	2024-03-10	External SSD	2
ThetaSolutions	2024-03-15	Enterprise Software	1
OmegaLabs	2024-04-01	Laptop Pro	1
TechCorp	2024-04-05	Mechanical Keyboard	1

15 rows returned.

Q95. Library joins (Example) MEDIUM

```
-- Members, Books, Loans
SELECT m.name, b.title, l.loan_date
FROM members m
JOIN loans l ON m.member_id = l.member_id
JOIN books b ON l.book_id = b.book_id;
```

WHAT THIS QUERY DOES

Another illustrative hypothetical schema — a small library dataset (3 members, 3 books, 4 loans) was added to demonstrate this concretely. Structurally, this is the same three-table chain as Q86 (orders → order_items → products): a middle table (`loans`) records *which* member borrowed *which* book and when, connecting two tables that have no direct relationship to each other.

RESULT ON OUR SAMPLE DATA

name	title	loan_date
Aarav Sharma	Clean Code	2024-05-01
Aarav Sharma	Design Patterns	2024-05-10
Priya Patel	The Pragmatic Programmer	2024-05-15
Rahul Singh	Clean Code	2024-06-01

4 rows returned.

Q96. Hospital joins (Example) MEDIUM

```
-- Patients, Doctors, Appointments
SELECT p.name, d.name, a.appointment_date
FROM patients p
```

```
JOIN appointments a ON p.patient_id = a.patient_id
JOIN doctors d ON a.doctor_id = d.doctor_id;
```

WHAT THIS QUERY DOES

Illustrative hospital schema (3 patients, 2 doctors, 4 appointments) added for a concrete demonstration. The `appointments` table plays the same 'junction table' role as `loans` did in Q95 or `order_items` in Q86 — it's what links two otherwise-unrelated tables (patients, doctors) together, recording who saw whom and when.

RESULT ON OUR SAMPLE DATA

name	name	appointment_date
Meera Krishnan	Dr. Kapoor	2024-03-01
Arjun Reddy	Dr. Iyer	2024-03-03
Sneha Patel	Dr. Kapoor	2024-03-05
Meera Krishnan	Dr. Iyer	2024-04-02

4 rows returned.

Q97. Banking joins (Example) MEDIUM

```
-- Customers, Accounts, Transactions
SELECT c.name, a.account_number, t.amount, t.transaction_date
FROM customers c
JOIN accounts a ON c.customer_id = a.customer_id
JOIN transactions t ON a.account_id = t.account_id;
```

WHAT THIS QUERY DOES

Illustrative banking schema added (2 bank customers, 2 accounts, 3 transactions) — kept in separate tables named `bank_customers/accounts/transactions` so they don't collide with this book's main `customers` table. Note this is a slightly deeper chain than a simple two-hop join: customer → their account(s) → that account's transactions, a very standard shape for financial data where a customer's money movements are always tied to a specific account, not to the customer directly.

RESULT ON OUR SAMPLE DATA

name	account_number	amount	transaction_date
TechCorp	AC1001	5,000	2024-01-05
TechCorp	AC1001	-1,200	2024-01-10
GlobalSolutions	AC1002	8,000	2024-01-12

3 rows returned.

Q98. Airline joins (Example) MEDIUM

```
-- Passengers, Flights, Bookings
SELECT p.name, f.flight_number, b.booking_date
FROM passengers p
JOIN bookings b ON p.passenger_id = b.passenger_id
JOIN flights f ON b.flight_id = f.flight_id;
```

WHAT THIS QUERY DOES

Illustrative airline schema added (2 passengers, 2 flights, 2 bookings). Same junction-table shape once more — `bookings` connects passengers to flights, since one passenger can book many flights and one flight carries many passengers (a many-to-many relationship, just like Q87's students/courses).

RESULT ON OUR SAMPLE DATA

name	flight_number	booking_date
Karthik Nair	AI-202	2024-02-01
Kavya Nair	6E-501	2024-02-03

2 rows returned.

Q99. Hotel joins (Example) MEDIUM

```
-- Guests, Rooms, Reservations
SELECT g.name, r.room_number, res.check_in_date
FROM guests g
JOIN reservations res ON g.guest_id = res.guest_id
JOIN rooms r ON res.room_id = r.room_id;
```

WHAT THIS QUERY DOES

Illustrative hotel schema added (2 guests, 2 rooms, 2 reservations). By now the pattern should feel very familiar: whenever two real-world entities (guests, rooms) can each relate to many of the other, a junction table (`reservations`) in the middle is what SQL uses to record and query those specific pairings.

RESULT ON OUR SAMPLE DATA

name	room_number	check_in_date
Suresh Babu	101	2024-03-10
Ravi Teja	102	2024-03-12

2 rows returned.

Q100. Movie booking joins (Example) HARD

```
-- Users, Movies, Showtimes, Bookings
SELECT u.name, m.title, s.show_time, b.booking_date
FROM users u
JOIN bookings b ON u.user_id = b.user_id
JOIN showtimes s ON b.showtime_id = s.showtime_id
JOIN movies m ON s.movie_id = m.movie_id;
```

WHAT THIS QUERY DOES

Illustrative movie-booking schema added (2 users, 2 movies, 2 showtimes, 2 bookings; tables named `app_users` and `movie_bookings` in the live example to avoid clashing with SQLite's reserved words and this book's other tables). This is a four-table chain with *two* junction-style hops: a movie has many showtimes, and a showtime has many bookings from different users — so reaching from 'user' to 'movie title' takes two intermediate steps (showtimes, then bookings) rather than one.

RESULT ON OUR SAMPLE DATA

name	title	show_time	booking_date
Neha Gupta	Interstellar Redux	2024-04-01 18:00	2024-03-28
Rajesh Kumar	The Last Algorithm	2024-04-01 21:00	2024-03-29

2 rows returned.

Q101. Employees and their project count MEDIUM

```
SELECT e.name, COUNT(p.project_id) AS project_count
FROM employees e
LEFT JOIN projects p ON e.department = p.department
GROUP BY e.emp_id, e.name;
```

WHAT THIS QUERY DOES

Combining JOIN with GROUP BY and COUNT, all in one query: LEFT JOIN attaches every project run by an employee's department (so employees in project-less departments still appear, with a count of 0 rather than vanishing), and grouping by employee then counts how many project rows each one picked up. Grouping by `e.emp_id` as well as `e.name` avoids accidentally merging two different employees who happen to share the same name.

RESULT ON OUR SAMPLE DATA

name	project_count
Aarav Sharma	3
Priya Patel	1

name	project_count
Rahul Singh	1
Neha Gupta	3
Vikram Reddy	3
Ananya Menon	1
Karthik Nair	3
Meera Krishnan	1
Arjun Reddy	3
Sneha Patel	1
Rajesh Kumar	1
Deepa Iyer	1
Suresh Babu	3
Kavya Nair	1
Ravi Teja	3

15 rows returned.

Q102. Departments with total project budget MEDIUM

```
SELECT d.dept_name, SUM(p.budget) AS total_budget
FROM departments d
LEFT JOIN projects p ON d.dept_name = p.department
GROUP BY d.dept_id, d.dept_name;
```

WHAT THIS QUERY DOES

Same JOIN + GROUP BY + aggregate combination as Q101, now summing project budgets per department instead of counting rows per employee. LEFT JOIN ensures 'Operations' (which has no projects) still shows up in the result, with a NULL total budget rather than being silently dropped.

RESULT ON OUR SAMPLE DATA

dept_name	total_budget
Finance	500,000
HR	400,000
IT	4,300,000
Marketing	300,000
Operations	*NULL*

5 rows returned.

Q103. Customers with highest order amount MEDIUM

```
SELECT c.name, MAX(o.amount) AS max_order
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
ORDER BY max_order DESC LIMIT 3;
```

WHAT THIS QUERY DOES

JOIN to attach each customer's orders, GROUP BY to find each customer's single biggest order, then the familiar 'sort descending, take top N' pattern (Q14) to surface the 3 customers whose largest single order was the biggest of all.

RESULT ON OUR SAMPLE DATA

name	max_order
DeltaCorp	67,000
InnovationLabs	50,000
BetaSystems	45,000

3 rows returned.

Q104. Products with total quantity sold MEDIUM

```
SELECT p.product_name, SUM(oi.quantity) AS total_sold
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name
ORDER BY total_sold DESC;
```

WHAT THIS QUERY DOES

LEFT JOIN here matters for a subtle reason: it guarantees products that have *never* been ordered still appear in the result (with a total of NULL/0), rather than silently disappearing the way an INNER JOIN would make them — useful for a true 'sales by product' report where you want to see the zero-sellers too, not just a list of what happened to sell.

RESULT ON OUR SAMPLE DATA

product_name	total_sold
Laptop Pro	9
External SSD	4
27" Monitor	3
Enterprise Software	3
Wireless Mouse	1

product_name	total_sold
Mechanical Keyboard	1
Cloud Storage Plan	1
Webcam HD	*NULL*
Printer	*NULL*
USB Hub	*NULL*

10 rows returned.

Q105. Employee salary vs department average HARD

```
SELECT e.name, e.salary, d.avg_salary
FROM employees e
JOIN (SELECT department, AVG(salary) AS avg_salary FROM employees GROUP BY
department) d
ON e.department = d.department;
```

WHAT THIS QUERY DOES

This introduces joining against a **subquery acting as a temporary table** (sometimes called a derived table): the parenthesized `SELECT ... GROUP BY department` first computes each department's average salary as its own mini result set, which is then joined back to `employees` just like any real table would be. This is a very common technique whenever you need to compare each individual row against an aggregate computed from its own group — and is a clean, JOIN-based alternative to the correlated subquery approach used back in Q62/Q123.

RESULT ON OUR SAMPLE DATA

name	salary	avg_salary
Aarav Sharma	75,000	84,714.29
Priya Patel	65,000	63,000
Rahul Singh	85,000	74,000
Neha Gupta	72,000	84,714.29
Vikram Reddy	90,000	84,714.29
Ananya Menon	60,000	63,000
Karthik Nair	95,000	84,714.29
Meera Krishnan	68,000	63,000
Arjun Reddy	82,000	84,714.29
Sneha Patel	58,000	63,000
Rajesh Kumar	70,000	74,000
Deepa Iyer	64,000	63,000
Suresh Babu	88,000	84,714.29

name	salary	avg_salary
Kavya Nair	67,000	74,000
Ravi Teja	91,000	84,714.29

15 rows returned.

Q106. Multiple join with conditions HARD

```
SELECT c.name, o.order_date, SUM(oi.quantity * oi.price) AS total
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.status = 'Completed'
AND o.order_date > '2024-01-01'
GROUP BY c.customer_id, c.name, o.order_date;
```

WHAT THIS QUERY DOES

A realistic 'everything at once' query: three tables joined together, filtered down with WHERE (only completed orders after a certain date), and grouped to compute each qualifying order's true total value (`quantity × price`, summed across every line item in that order). This is close to what an actual sales report query looks like in production — JOINs, filters, and aggregation all working together.

RESULT ON OUR SAMPLE DATA

name	order_date	total
TechCorp	2024-01-10	151,500
TechCorp	2024-04-05	5,000
GlobalSolutions	2024-01-20	25,000
InnovationLabs	2024-02-01	99,000
AlphaInc	2024-02-10	50,000
GammaTech	2024-03-01	50,000
DeltaCorp	2024-03-05	150,000
EpsilonSoft	2024-03-10	24,000
OmegaLabs	2024-04-01	75,000

9 rows returned.

Q107. Self join for same city employees MEDIUM

```
SELECT e1.name AS emp1, e2.name AS emp2, e1.city
FROM employees e1
JOIN employees e2 ON e1.city = e2.city AND e1.emp_id < e2.emp_id;
```

WHAT THIS QUERY DOES

Another self join (see Q80), but this time joining the table to itself to find *pairs* of employees who share something in common — here, living in the same city. The condition `e1.emp_id < e2.emp_id` is a deliberate trick to avoid two kinds of redundancy: without it, you'd get both ('Aarav','Rahul') and ('Rahul','Aarav') as separate rows (the same pair, twice), and you'd also incorrectly match every employee with themself. Requiring the first ID to be strictly less than the second guarantees each pair appears exactly once.

RESULT ON OUR SAMPLE DATA

emp1	emp2	city
Aarav Sharma	Ananya Menon	Mumbai
Aarav Sharma	Arjun Reddy	Mumbai
Aarav Sharma	Rahul Singh	Mumbai
Aarav Sharma	Ravi Teja	Mumbai
Priya Patel	Meera Krishnan	Delhi
Priya Patel	Vikram Reddy	Delhi
Rahul Singh	Ananya Menon	Mumbai
Rahul Singh	Arjun Reddy	Mumbai
Rahul Singh	Ravi Teja	Mumbai
Neha Gupta	Karthik Nair	Bangalore
Neha Gupta	Sneha Patel	Bangalore
Vikram Reddy	Meera Krishnan	Delhi
Ananya Menon	Arjun Reddy	Mumbai
Ananya Menon	Ravi Teja	Mumbai
Karthik Nair	Sneha Patel	Bangalore
Arjun Reddy	Ravi Teja	Mumbai
Rajesh Kumar	Suresh Babu	Chennai
Deepa Iyer	Kavya Nair	Hyderabad

18 rows returned.

Q108. Employees with managers in same department MEDIUM

```
SELECT e.name AS employee, m.name AS manager, e.department
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.department = m.department;
```

WHAT THIS QUERY DOES

Builds on the employee-manager self join from Q80, adding a filter for cases where the employee and their manager happen to be recorded in the exact same department — useful for spotting normal, expected reporting-line structures (versus, say, a cross-department dotted-line manager, which this filter would exclude).

RESULT ON OUR SAMPLE DATA

employee	manager	department
Neha Gupta	Aarav Sharma	IT
Vikram Reddy	Aarav Sharma	IT
Ananya Menon	Priya Patel	HR
Karthik Nair	Aarav Sharma	IT
Arjun Reddy	Vikram Reddy	IT
Sneha Patel	Meera Krishnan	Marketing
Rajesh Kumar	Rahul Singh	Finance
Deepa Iyer	Priya Patel	HR
Suresh Babu	Vikram Reddy	IT
Kavya Nair	Rahul Singh	Finance
Ravi Teja	Aarav Sharma	IT

11 rows returned.

Q109. Cross join for department-location combinations HARD

```
SELECT d.dept_name, l.location
FROM departments d
CROSS JOIN (SELECT DISTINCT city AS location FROM employees) l;
```

WHAT THIS QUERY DOES

A `CROSS JOIN` is different from every other join in this level: it doesn't match on any condition at all — it simply pairs *every* row from the first table with *every* row from the second, producing all possible combinations (5 departments × 5 distinct cities = 25 rows here). This is useful for generating a complete grid of possibilities (e.g.

'which department/city combinations exist, and which don't') that you might later compare against actual data to spot gaps.

RESULT ON OUR SAMPLE DATA

dept_name	location
Finance	Mumbai
Finance	Delhi
Finance	Bangalore
Finance	Chennai
Finance	Hyderabad
HR	Mumbai
HR	Delhi
HR	Bangalore
HR	Chennai
HR	Hyderabad
IT	Mumbai
IT	Delhi
IT	Bangalore
IT	Chennai
IT	Hyderabad
Marketing	Mumbai
Marketing	Delhi
Marketing	Bangalore
Marketing	Chennai
Marketing	Hyderabad

Showing first 20 of 25 rows.

Q110. Complex join with subquery HARD

```
SELECT e.name, e.salary, d.dept_name
FROM employees e
JOIN departments d ON e.department = d.dept_name
WHERE e.salary > (SELECT AVG(salary) FROM employees WHERE department =
e.department);
```

WHAT THIS QUERY DOES

Combines a plain JOIN (to pull in the department's location details via d) with a **correlated subquery** in the WHERE clause (the same 'above your own department's average' check from Q123) — showing that JOINs and correlated

subqueries aren't an either/or choice; real queries frequently use both together, each solving a different part of the problem.

RESULT ON OUR SAMPLE DATA

name	salary	dept_name
Priya Patel	65,000	HR
Rahul Singh	85,000	Finance
Vikram Reddy	90,000	IT
Karthik Nair	95,000	IT
Meera Krishnan	68,000	Marketing
Deepa Iyer	64,000	HR
Suresh Babu	88,000	IT
Ravi Teja	91,000	IT

8 rows returned.

Level 5: Subqueries

A subquery is a query nested inside another one. This level covers the full toolkit: simple aggregate subqueries, correlated subqueries (which re-run per outer row), EXISTS/NOT EXISTS, IN/NOT IN, ANY/ALL, and the classic 'Nth highest value' and 'find the argmax group' patterns that come up constantly in interviews.

Q111. Second highest salary MEDIUM

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

WHAT THIS QUERY DOES

Identical to Q25 — included again here as this level's opening example of the nested-subquery technique, since the whole rest of Level 5 builds on being comfortable with a query inside a query.

RESULT ON OUR SAMPLE DATA

MAX(salary)
91,000

1 row returned.

Q112. Third highest salary MEDIUM

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees
                WHERE salary < (SELECT MAX(salary) FROM employees));
```

WHAT THIS QUERY DOES

Nesting the same trick from Q111 one level deeper: the innermost subquery finds the highest salary, the middle layer finds the highest salary *below* that (the 2nd highest), and the outer query finds the highest salary below *that* (the 3rd highest). Each layer peels off one more 'top' value — but as you can see, this gets unwieldy fast, which is exactly why Q113 introduces a cleaner, more general approach for 'Nth highest' that doesn't require manually nesting a nother layer for every N.

RESULT ON OUR SAMPLE DATA

MAX(salary)
90,000

1 row returned.

Q113. Nth highest salary (generic) HARD

```
-- For N=4
SELECT DISTINCT salary
FROM employees e1
WHERE 4 = (SELECT COUNT(DISTINCT salary)
           FROM employees e2
           WHERE e2.salary >= e1.salary);
```

WHAT THIS QUERY DOES

A cleaner, reusable alternative to the nested-MAX approach: for each distinct salary in the table, count how many *distinct* salaries are greater than or equal to it. A salary that's the highest will have exactly 1 salary ≥ itself (only itself); the 2nd highest will have exactly 2 (itself and the highest); and so on. So 'find the salary where this count equals 4' directly gives you the 4th-highest salary, no matter how large N is — just change the number, without adding another layer of nesting. This correlated subquery pattern is a genuinely important one to memorize for interviews, since 'Nth highest X' is an extremely common question.

RESULT ON OUR SAMPLE DATA

salary
88,000

1 row returned.

Q114. Salary above average EASY

```
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

WHAT THIS QUERY DOES

Identical to Q38 — the single-value subquery pattern (compute one number, then filter rows against it) shown again here as this level's canonical simple example.

RESULT ON OUR SAMPLE DATA

name	salary
Rahul Singh	85,000
Vikram Reddy	90,000
Karthik Nair	95,000
Arjun Reddy	82,000
Suresh Babu	88,000
Ravi Teja	91,000

6 rows returned.

Q115. Salary below average EASY

```
SELECT name, salary FROM employees
WHERE salary < (SELECT AVG(salary) FROM employees);
```

WHAT THIS QUERY DOES

The mirror image of Q114 — same subquery, opposite comparison direction, to find who's earning below the company-wide average instead of above it.

RESULT ON OUR SAMPLE DATA

name	salary
Aarav Sharma	75,000
Priya Patel	65,000
Neha Gupta	72,000
Ananya Menon	60,000
Meera Krishnan	68,000
Sneha Patel	58,000
Rajesh Kumar	70,000
Deepa Iyer	64,000
Kavya Nair	67,000

9 rows returned.

Q116. Department average comparison MEDIUM

```
SELECT name, salary, department
FROM employees e
WHERE salary > (SELECT AVG(salary) FROM employees WHERE department =
e.department);
```

WHAT THIS QUERY DOES

Identical to Q123 (they're the same query, included twice by the original cheat sheet) — this is a **correlated subquery**: the inner query's `WHERE department = e.department` reaches out to the outer query's current row, so the 'average' being compared against is recalculated fresh for each employee's own department, not one single company-wide number. See Q123 for the fuller explanation of how correlated subqueries differ from the simple kind in Q114/115.

RESULT ON OUR SAMPLE DATA

name	salary	department
Priya Patel	65,000	HR
Rahul Singh	85,000	Finance
Vikram Reddy	90,000	IT
Karthik Nair	95,000	IT
Meera Krishnan	68,000	Marketing
Deepa Iyer	64,000	HR
Suresh Babu	88,000	IT
Ravi Teja	91,000	IT

8 rows returned.

Q117. EXISTS - Employees with orders MEDIUM

```
SELECT name FROM employees e
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = e.emp_id);
```

WHAT THIS QUERY DOES

EXISTS is a true/false check: it doesn't care *what* the subquery returns (notice it just selects the literal 1, a throwaway placeholder), only *whether it returns any rows at all* for the current outer row. This keeps an employee only if at least one order exists with a matching `customer_id`. In our sample schema, `orders.customer_id` actually refers to `customers`, not `employees` — so this query is really testing 'does this employee's ID number happen to coincidentally match some customer's order,' a coincidental/illustrative overlap in the ID numbering rather than a real business relationship, which is worth noticing since it's a good reminder to always double check what a foreign-key-like column is actually supposed to reference.

RESULT ON OUR SAMPLE DATA

name
Aarav Sharma
Priya Patel
Rahul Singh
Neha Gupta
Vikram Reddy
Ananya Menon
Karthik Nair
Meera Krishnan
Arjun Reddy
Sneha Patel

10 rows returned.

Q118. NOT EXISTS - Employees without orders MEDIUM

```
SELECT name FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = e.emp_id);
```

WHAT THIS QUERY DOES

The negation of Q117 — keeps an employee only if the subquery finds *zero* matching rows. EXISTS/NOT EXISTS are often the most efficient way to write these existence-checks, since the database can stop searching the instant it finds (or confirms the absence of) a single matching row, without needing to count or collect every match the way IN sometimes implies.

RESULT ON OUR SAMPLE DATA

name
Rajesh Kumar
Deepa Iyer
Suresh Babu
Kavya Nair
Ravi Teja

5 rows returned.

Q119. IN - Employees in IT MEDIUM

```
SELECT name FROM employees
WHERE department IN (SELECT dept_name FROM departments WHERE location =
'Mumbai');
```

WHAT THIS QUERY DOES

IN checks whether a value matches *any* value in a list — and that list can come from a subquery instead of being typed out by hand. Here, the inner query first finds every department located in Mumbai, and the outer query then keeps employees whose department name appears anywhere in that resulting list. Despite the question's title mentioning 'IT,' the actual query filters by *location*, not department name — IT just happens to be one of the departments based in Mumbai in our sample data.

RESULT ON OUR SAMPLE DATA

name
Aarav Sharma
Rahul Singh
Neha Gupta

name
Vikram Reddy
Karthik Nair
Arjun Reddy
Rajesh Kumar
Suresh Babu
Kavya Nair
Ravi Teja

10 rows returned.

Q120. NOT IN - Employees not in IT MEDIUM

```
SELECT name FROM employees
WHERE department NOT IN (SELECT dept_name FROM departments WHERE location =
'Mumbai');
```

WHAT THIS QUERY DOES

The negation of Q119 — keeps employees whose department is *not* among the Mumbai-based ones. One important gotcha worth knowing for interviews: `NOT IN` behaves unexpectedly if the subquery's result list contains a `NULL` — it can cause the whole query to return zero rows unexpectedly. `NOT EXISTS` doesn't have this problem, which is why many experienced SQL developers prefer `NOT EXISTS` over `NOT IN` as a matter of habit.

RESULT ON OUR SAMPLE DATA

name
Priya Patel
Ananya Menon
Meera Krishnan
Sneha Patel
Deepa Iyer

5 rows returned.

Q121. ANY - Salary greater than any IT employee HARD

```
SELECT name, salary FROM employees
WHERE salary > ANY (SELECT salary FROM employees WHERE department = 'IT');
```

WHAT THIS QUERY DOES

> ANY means 'greater than *at least one* value in the list' — which is logically the same as being greater than the *smallest* value in that list. So this keeps anyone earning more than the lowest-paid IT employee. *Note: SQLite doesn't support the ANY/ALL keywords directly (they're a MySQL/PostgreSQL/SQL Server feature), so the live example below runs the logically equivalent `salary > (SELECT MIN(salary) FROM employees WHERE department = 'IT')` — same meaning, different syntax.*

RESULT ON OUR SAMPLE DATA

name	salary
Aarav Sharma	75,000
Rahul Singh	85,000
Vikram Reddy	90,000
Karthik Nair	95,000
Arjun Reddy	82,000
Suresh Babu	88,000
Ravi Teja	91,000

7 rows returned.

Q122. ALL - Salary greater than all Marketing employees HARD

```
SELECT name, salary FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE department = 'Marketing');
```

WHAT THIS QUERY DOES

> ALL means 'greater than *every single* value in the list' — equivalent to being greater than the *largest* value in that list. This keeps only employees who out-earn even the highest-paid Marketing employee. *(Again executed here as `salary > (SELECT MAX(salary) FROM employees WHERE department = 'Marketing')` since SQLite has no native ALL keyword.)* A useful memory trick: ANY is the 'easy to satisfy' one (beat just one value = beat the minimum), while ALL is the 'hard to satisfy' one (beat every value = beat the maximum).

RESULT ON OUR SAMPLE DATA

name	salary
Aarav Sharma	75,000
Rahul Singh	85,000
Neha Gupta	72,000
Vikram Reddy	90,000
Karthik Nair	95,000
Arjun Reddy	82,000
Rajesh Kumar	70,000

name	salary
Suresh Babu	88,000
Ravi Teja	91,000

9 rows returned.

Q123. Correlated subquery - Salary above department average

MEDIUM

```
SELECT name, salary, department
FROM employees e1
WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE e1.department =
e2.department);
```

WHAT THIS QUERY DOES

This is the fullest explanation of the correlated subquery pattern seen a few times already (Q62/63, Q116): unlike Q114's subquery (which computes one single number, totally independent of the outer query), this inner query's condition `e1.department = e2.department` reaches back out and references the *current outer row* — meaning conceptually, the subquery has to be re-evaluated separately for every single employee, each time computing 'the average salary within *this employee's own* department' rather than one company-wide figure. It's the single most powerful subquery pattern in this book precisely because it lets each row be judged against a standard computed just for its own group.

RESULT ON OUR SAMPLE DATA

name	salary	department
Priya Patel	65,000	HR
Rahul Singh	85,000	Finance
Vikram Reddy	90,000	IT
Karthik Nair	95,000	IT
Meera Krishnan	68,000	Marketing
Deepa Iyer	64,000	HR
Suresh Babu	88,000	IT
Ravi Teja	91,000	IT

8 rows returned.

Q124. Employees earning more than manager MEDIUM

```
SELECT e.name AS employee, e.salary AS emp_salary,
       m.name AS manager, m.salary AS mgr_salary
```

```
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

WHAT THIS QUERY DOES

No subquery here at all — this reuses the employee-manager self join from Q80/108, then simply filters with a plain `WHERE` comparing two columns from the joined result. It's included in the Subqueries level as a useful contrast: some 'compare against a related value' questions are better solved with a JOIN than a subquery, and recognizing which tool fits is as important as knowing the syntax for both.

RESULT ON OUR SAMPLE DATA

employee	emp_salary	manager	mgr_salary
Rahul Singh	85,000	Aarav Sharma	75,000
Vikram Reddy	90,000	Aarav Sharma	75,000
Karthik Nair	95,000	Aarav Sharma	75,000
Ravi Teja	91,000	Aarav Sharma	75,000

4 rows returned.

Q125. Products never ordered MEDIUM

```
SELECT product_name FROM products
WHERE product_id NOT IN (SELECT DISTINCT product_id FROM order_items);
```

WHAT THIS QUERY DOES

The same underlying question as Q85, now solved with `NOT IN` instead of a LEFT JOIN + NULL check — a good demonstration that many 'find unmatched rows' problems can be written either way. The subquery collects every product ID that appears anywhere in `order_items`, and the outer query keeps products whose ID isn't in that list.

RESULT ON OUR SAMPLE DATA

product_name
Webcam HD
Printer
USB Hub

3 rows returned.

Q126. Customers ordering all products HARD

```
SELECT c.name
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_id, c.name
HAVING COUNT(DISTINCT oi.product_id) = (SELECT COUNT(*) FROM products);
```

WHAT THIS QUERY DOES

A 'relational division' style query: join everything together, group by customer, and count how many *distinct* products each customer has ever ordered — then use `HAVING` to keep only customers whose distinct-product count matches the *total* number of products that exist (computed by a subquery). In plain English: 'find customers who have, between all their orders, bought one of literally everything.' With 10 products in our catalog and no single customer having ordered that broadly, this correctly returns 0 rows here, but the technique itself is a classic interview pattern worth knowing.

RESULT ON OUR SAMPLE DATA

name
(no rows returned)

0 rows returned.

Q127. Department maximum salary with employee name MEDIUM

```
SELECT name, department, salary
FROM employees e
WHERE salary = (SELECT MAX(salary) FROM employees WHERE department =
e.department);
```

WHAT THIS QUERY DOES

Identical to Q62 — the correlated-subquery technique for finding the top earner's full row in every group, shown again here as this level's canonical 'match against a per-group MAX' example.

RESULT ON OUR SAMPLE DATA

name	department	salary
Priya Patel	HR	65,000
Rahul Singh	Finance	85,000
Karthik Nair	IT	95,000
Meera Krishnan	Marketing	68,000

4 rows returned.

Q128. Department minimum salary with employee name MEDIUM

```
SELECT name, department, salary
FROM employees e
WHERE salary = (SELECT MIN(salary) FROM employees WHERE department =
e.department);
```

WHAT THIS QUERY DOES

Identical to Q63 — the MIN version of Q127's pattern.

RESULT ON OUR SAMPLE DATA

name	department	salary
Neha Gupta	IT	72,000
Ananya Menon	HR	60,000
Sneha Patel	Marketing	58,000
Kavya Nair	Finance	67,000

4 rows returned.

Q129. Latest order for each customer MEDIUM

```
SELECT customer_id, order_date, amount
FROM orders o1
WHERE order_date = (SELECT MAX(order_date) FROM orders o2 WHERE o1.customer_id =
o2.customer_id);
```

WHAT THIS QUERY DOES

The correlated-subquery pattern applied to `orders` instead of `employees`: for each order, check whether its date matches the *most recent* date among all orders placed by that same customer. Rows that pass are exactly each customer's single latest order.

RESULT ON OUR SAMPLE DATA

customer_id	order_date	amount
4	2024-02-10	32,000
5	2024-02-15	45,000
6	2024-03-01	21,000
7	2024-03-05	67,000
8	2024-03-10	34,000
9	2024-03-15	28,000

customer_id	order_date	amount
10	2024-04-01	42,000
1	2024-04-05	15,000
2	2024-04-10	33,000
3	2024-04-15	29,000

10 rows returned.

Q130. Oldest order for each customer MEDIUM

```
SELECT customer_id, order_date, amount
FROM orders o1
WHERE order_date = (SELECT MIN(order_date) FROM orders o2 WHERE o1.customer_id =
o2.customer_id);
```

WHAT THIS QUERY DOES

The mirror image of Q129, using `MIN` instead of `MAX` to find each customer's very first order instead of their latest.

RESULT ON OUR SAMPLE DATA

customer_id	order_date	amount
1	2024-01-10	15,000
2	2024-01-20	12,000
3	2024-02-01	50,000
4	2024-02-10	32,000
5	2024-02-15	45,000
6	2024-03-01	21,000
7	2024-03-05	67,000
8	2024-03-10	34,000
9	2024-03-15	28,000
10	2024-04-01	42,000

10 rows returned.

Q131. Employees hired after average hire date MEDIUM

```
SELECT name, hire_date FROM employees
WHERE hire_date > (SELECT AVG(hire_date) FROM employees);
```

WHAT THIS QUERY DOES

A worthwhile pitfall to understand rather than a query to copy as-is: `AVG()` is built for numbers, and a date like `'2020-01-15'` is really just text. Different databases handle `AVG()` on a date/text column differently — some will error out, others will silently coerce it into some numeric interpretation that doesn't correspond to any real calendar date. In this book's SQLite engine, it happens to extract a rough numeric value and the resulting comparison doesn't reliably mean 'hired after the true average date' — the result shown below should be treated as illustrative of the *syntax*, not as a reliable answer. To do this correctly, you'd first convert each date to a comparable number of days (e.g. SQLite's `julianday(hire_date)`, or MySQL's `TO_DAYS(hire_date)`), average *that*, and then convert back — a good reminder that not every query that runs without an error is actually computing something meaningful.

RESULT ON OUR SAMPLE DATA

name	hire_date
Aarav Sharma	2020-01-15
Priya Patel	2019-03-20
Neha Gupta	2021-02-01
Ananya Menon	2022-06-15
Meera Krishnan	2020-08-12
Sneha Patel	2023-01-20
Rajesh Kumar	2019-06-10
Deepa Iyer	2020-11-15
Kavya Nair	2021-08-20

9 rows returned.

Q132. Department with most employees HARD

```
SELECT department
FROM employees
GROUP BY department
HAVING COUNT(*) = (SELECT MAX(emp_count)
                   FROM (SELECT COUNT(*) AS emp_count FROM employees GROUP BY
department) AS t);
```

WHAT THIS QUERY DOES

A subquery *inside a subquery*: the innermost query groups employees by department and counts each group's size; the middle layer wraps that as a mini result-table (aliased `t`) and takes the overall maximum count from it; the outer query then keeps whichever department(s) match that maximum. This is essentially 'find the argmax' — the group with the biggest count — done by first computing all

the counts, then finding the biggest one, then matching back to find which group(s) achieved it.

RESULT ON OUR SAMPLE DATA

department
IT

1 row returned.

Q133. Department with highest total salary HARD

```
SELECT department
FROM employees
GROUP BY department
HAVING SUM(salary) = (SELECT MAX(total_salary)
                      FROM (SELECT SUM(salary) AS total_salary FROM employees
GROUP BY department) AS t);
```

WHAT THIS QUERY DOES

The exact same triple-nested 'find the argmax' structure as Q132, just aggregating with `SUM(salary)` instead of `COUNT(*)` — a good example of how the same query *shape* answers 'which group has the most people' and 'which group costs the most in salary' with only the aggregate function swapped.

RESULT ON OUR SAMPLE DATA

department
IT

1 row returned.

Q134. Employees with salary in top 3 MEDIUM

```
SELECT name, salary
FROM employees
WHERE salary IN (SELECT DISTINCT salary FROM employees ORDER BY salary DESC LIMIT
3);
```

WHAT THIS QUERY DOES

The subquery finds the 3 highest distinct salary *values* (not employees — if two people tied for 3rd place, this still only picks 3 salary values), and the outer query then finds every employee earning one of those values. Worth noting this can return *more than 3 employees* if there are salary ties among the top 3 values — a subtly different result than a strict 'top 3 people' query.

RESULT ON OUR SAMPLE DATA

name	salary
Vikram Reddy	90,000
Karthik Nair	95,000
Ravi Teja	91,000

3 rows returned.

Q135. Employees with salary in bottom 3 MEDIUM

```
SELECT name, salary
FROM employees
WHERE salary IN (SELECT DISTINCT salary FROM employees ORDER BY salary ASC LIMIT
3);
```

WHAT THIS QUERY DOES

The mirror image of Q134 — the 3 lowest distinct salary values, and everyone earning one of them.

RESULT ON OUR SAMPLE DATA

name	salary
Ananya Menon	60,000
Sneha Patel	58,000
Deepa Iyer	64,000

3 rows returned.

Q136. Customers with at least 2 orders MEDIUM

```
SELECT name FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders GROUP BY customer_id HAVING
COUNT(*) >= 2);
```

WHAT THIS QUERY DOES

The subquery does its own grouping and filtering internally (GROUP BY + HAVING, from Level 3) to find customer IDs with 2 or more orders, and the outer query then looks up those customers' names. This shows subqueries aren't limited to single-value results — a subquery can itself be a fully grouped, filtered mini-query, as long as its final output is a single column that the outer `IN` can check against.

RESULT ON OUR SAMPLE DATA

name
TechCorp
GlobalSolutions

name
InnovationLabs

3 rows returned.

Q137. Products above average price EASY

```
SELECT product_name, price FROM products
WHERE price > (SELECT AVG(price) FROM products);
```

WHAT THIS QUERY DOES

The same simple 'compare against one aggregate value' subquery from Q114/115, applied to the `products` table's `price` column instead of employee salaries.

RESULT ON OUR SAMPLE DATA

product_name	price
Laptop Pro	75,000
Enterprise Software	100,000
Cloud Storage Plan	50,000

3 rows returned.

Q138. Employees working on active projects MEDIUM

```
SELECT DISTINCT e.name
FROM employees e
WHERE e.department IN (SELECT department FROM projects WHERE status = 'Active');
```

WHAT THIS QUERY DOES

The inner query finds every department currently running at least one Active project; the outer query then lists employees belonging to any of those departments. `DISTINCT` matters here because the inner subquery could list the same department multiple times (once per active project in it), which would otherwise risk showing an employee's name more than once if `IN` were somehow sensitive to duplicate list entries — though in practice `IN` already de-duplicates its comparison list, so DISTINCT here is really about making sure each employee's name appears only once in the final output, not about the subquery itself.

RESULT ON OUR SAMPLE DATA

name
Aarav Sharma
Rahul Singh
Neha Gupta

name
Vikram Reddy
Karthik Nair
Arjun Reddy
Rajesh Kumar
Suresh Babu
Kavya Nair
Ravi Teja

10 rows returned.

Q139. Customers with high value orders EASY

```
SELECT name FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders WHERE amount > 50000);
```

WHAT THIS QUERY DOES

A simple `IN` subquery: find every order over 50,000, collect their customer IDs, and look up which customers those belong to. Note a customer with *multiple* qualifying orders would still only appear once in the final list, since `customers.customer_id` is unique per customer regardless of how many matching order IDs led to it being included.

RESULT ON OUR SAMPLE DATA

name
DeltaCorp

1 row returned.

Q140. Employees with bonus above average EASY

```
SELECT name, bonus FROM employees
WHERE bonus > (SELECT AVG(bonus) FROM employees);
```

WHAT THIS QUERY DOES

The same simple aggregate-subquery comparison from Q114/137, one more time on the bonus column — by now this shape ('filter rows against one number computed by a subquery') should feel completely routine, which is exactly the point of seeing it repeated across several different columns and tables throughout this level.

RESULT ON OUR SAMPLE DATA

name	bonus
Rahul Singh	15,000
Vikram Reddy	12,000
Karthik Nair	20,000
Arjun Reddy	11,000
Suresh Babu	18,000
Ravi Teja	16,000

6 rows returned.

Level 6: Window Functions

Window functions add a computed column alongside every row, based on a 'window' of related rows -- unlike GROUP BY, no rows get collapsed. This level covers ranking (ROW_NUMBER/RANK/DENSE_RANK), row comparisons (LEAD/LAG), running totals and moving averages, PARTITION BY, and the CTE-then-filter pattern needed for 'top N per group' queries.

Q141. ROW_NUMBER() - Rank employees by salary MEDIUM

```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num
FROM employees;
```

WHAT THIS QUERY DOES

Welcome to **window functions** — a different beast from GROUP BY. GROUP BY *collapses* many rows into one summary row per group; a window function instead keeps *every original row* and just adds an extra computed column alongside it, based on a 'window' of related rows (here, the whole table, ordered by salary). `ROW_NUMBER()` simply numbers the rows 1, 2, 3... in the order specified by `OVER (ORDER BY salary DESC)` — the highest earner gets row_num 1, and every row gets a strictly unique number even if two salaries happen to tie.

RESULT ON OUR SAMPLE DATA

name	salary	row_num
Karthik Nair	95,000	1
Ravi Teja	91,000	2
Vikram Reddy	90,000	3
Suresh Babu	88,000	4
Rahul Singh	85,000	5
Arjun Reddy	82,000	6
Aarav Sharma	75,000	7
Neha Gupta	72,000	8
Rajesh Kumar	70,000	9
Meera Krishnan	68,000	10
Kavya Nair	67,000	11
Priya Patel	65,000	12
Deepa Iyer	64,000	13
Ananya Menon	60,000	14
Sneha Patel	58,000	15

15 rows returned.

Q142. RANK() - Rank with gaps MEDIUM

```
SELECT name, salary,
       RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

WHAT THIS QUERY DOES

`RANK()` is like `ROW_NUMBER()`, but it's aware of ties: two rows with the exact same salary get the exact same rank. The catch is that ranks then **skip** ahead — if two people tie for rank 2, the next distinct salary jumps straight to rank 4 (not 3), because 'rank 3' is considered already used up by the two-way tie. This mirrors how sports leaderboards usually work: two joint-2nd-place finishers mean there's no 3rd place at all.

RESULT ON OUR SAMPLE DATA

name	salary	salary_rank
Karthik Nair	95,000	1
Ravi Teja	91,000	2
Vikram Reddy	90,000	3
Suresh Babu	88,000	4
Rahul Singh	85,000	5
Arjun Reddy	82,000	6
Aarav Sharma	75,000	7
Neha Gupta	72,000	8
Rajesh Kumar	70,000	9
Meera Krishnan	68,000	10
Kavya Nair	67,000	11
Priya Patel	65,000	12
Deepa Iyer	64,000	13
Ananya Menon	60,000	14
Sneha Patel	58,000	15

15 rows returned.

Q143. DENSE_RANK() - Rank without gaps MEDIUM

```
SELECT name, salary,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank
FROM employees;
```

WHAT THIS QUERY DOES

`DENSE_RANK()` handles ties the same way as `RANK()` (equal values get equal ranks), but *without* leaving gaps afterward — after a two-way tie for rank 2, the next distinct value becomes rank 3, not rank 4. In our sample data every salary happens to be unique, so `RANK()`, `DENSE_RANK()`, and `ROW_NUMBER()` all produce identical numbers here — their differences only become visible once there are genuine ties, which is worth keeping in mind if a first look at the output makes them seem interchangeable.

RESULT ON OUR SAMPLE DATA

name	salary	dense_rank
Karthik Nair	95,000	1
Ravi Teja	91,000	2
Vikram Reddy	90,000	3
Suresh Babu	88,000	4
Rahul Singh	85,000	5
Arjun Reddy	82,000	6
Aarav Sharma	75,000	7
Neha Gupta	72,000	8
Rajesh Kumar	70,000	9
Meera Krishnan	68,000	10
Kavya Nair	67,000	11
Priya Patel	65,000	12
Deepa Iyer	64,000	13
Ananya Menon	60,000	14
Sneha Patel	58,000	15

15 rows returned.

Q144. LEAD() - Next salary MEDIUM

```
SELECT name, salary,
       LEAD(salary) OVER (ORDER BY salary) AS next_salary
FROM employees;
```

WHAT THIS QUERY DOES

LEAD() reaches *forward* to grab a value from a later row (by default, the very next one) in the specified order, without needing a self-join. Ordered ascending here, each row's next_salary shows what the *next-higher-paid* employee earns — the very last row (the highest earner) has nothing after it, so its next_salary comes back NULL.

RESULT ON OUR SAMPLE DATA

name	salary	next_salary
Sneha Patel	58,000	60,000
Ananya Menon	60,000	64,000
Deepa Iyer	64,000	65,000
Priya Patel	65,000	67,000
Kavya Nair	67,000	68,000
Meera Krishnan	68,000	70,000
Rajesh Kumar	70,000	72,000
Neha Gupta	72,000	75,000
Aarav Sharma	75,000	82,000
Arjun Reddy	82,000	85,000
Rahul Singh	85,000	88,000
Suresh Babu	88,000	90,000
Vikram Reddy	90,000	91,000
Ravi Teja	91,000	95,000
Karthik Nair	95,000	*NULL*

15 rows returned.

Q145. LAG() - Previous salary MEDIUM

```
SELECT name, salary,
       LAG(salary) OVER (ORDER BY salary) AS prev_salary
FROM employees;
```

WHAT THIS QUERY DOES

The mirror image of LEAD() — LAG() reaches *backward* to the previous row instead. The very first row (lowest salary) has nothing before it, so its prev_salary is NULL. Together, LEAD and LAG are what let you compare each row to its neighbours without writing a self-join or a correlated subquery.

RESULT ON OUR SAMPLE DATA

name	salary	prev_salary
Sneha Patel	58,000	*NULL*

name	salary	prev_salary
Ananya Menon	60,000	58,000
Deepa Iyer	64,000	60,000
Priya Patel	65,000	64,000
Kavya Nair	67,000	65,000
Meera Krishnan	68,000	67,000
Rajesh Kumar	70,000	68,000
Neha Gupta	72,000	70,000
Aarav Sharma	75,000	72,000
Arjun Reddy	82,000	75,000
Rahul Singh	85,000	82,000
Suresh Babu	88,000	85,000
Vikram Reddy	90,000	88,000
Ravi Teja	91,000	90,000
Karthik Nair	95,000	91,000

15 rows returned.

Q146. FIRST_VALUE() - Lowest salary in department HARD

```
SELECT name, department, salary,
       FIRST_VALUE(name) OVER (PARTITION BY department ORDER BY salary) AS
lowest_paid
FROM employees;
```

WHAT THIS QUERY DOES

`PARTITION BY` is to window functions what `GROUP BY` is to aggregates — it splits the table into per-department 'windows,' and everything in `OVER(...)` then operates *separately within each partition* instead of across the whole table. `FIRST_VALUE()` grabs the value from the very first row of each partition (as defined by its `ORDER BY`) — here, ascending salary order means 'first' means 'lowest-paid,' so every row in a department shows that department's lowest earner's name, repeated alongside each of that department's individual rows.

RESULT ON OUR SAMPLE DATA

name	department	salary	lowest_paid
Kavya Nair	Finance	67,000	Kavya Nair
Rajesh Kumar	Finance	70,000	Kavya Nair
Rahul Singh	Finance	85,000	Kavya Nair
Ananya Menon	HR	60,000	Ananya Menon
Deepa Iyer	HR	64,000	Ananya Menon

name	department	salary	lowest_paid
Priya Patel	HR	65,000	Ananya Menon
Neha Gupta	IT	72,000	Neha Gupta
Aarav Sharma	IT	75,000	Neha Gupta
Arjun Reddy	IT	82,000	Neha Gupta
Suresh Babu	IT	88,000	Neha Gupta
Vikram Reddy	IT	90,000	Neha Gupta
Ravi Teja	IT	91,000	Neha Gupta
Karthik Nair	IT	95,000	Neha Gupta
Sneha Patel	Marketing	58,000	Sneha Patel
Meera Krishnan	Marketing	68,000	Sneha Patel

15 rows returned.

Q147. LAST_VALUE() - Highest salary in department HARD

```
SELECT name, department, salary,
       LAST_VALUE(name) OVER (PARTITION BY department ORDER BY salary
                              ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
FOLLOWING) AS highest_paid
FROM employees;
```

WHAT THIS QUERY DOES

The counterpart to Q146, but with an important gotcha: by default, a window function's 'frame' only extends up to the *current row* (not the whole partition), so a naive `LAST_VALUE()` would just return each row's *own* name, not the true partition maximum. The extra clause `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` explicitly widens the frame to the *entire* partition, every time, regardless of which row is 'current' — only then does `LAST_VALUE()` correctly return the highest-paid person's name for every row in that department.

RESULT ON OUR SAMPLE DATA

name	department	salary	highest_paid
Kavya Nair	Finance	67,000	Rahul Singh
Rajesh Kumar	Finance	70,000	Rahul Singh
Rahul Singh	Finance	85,000	Rahul Singh
Ananya Menon	HR	60,000	Priya Patel
Deepa Iyer	HR	64,000	Priya Patel
Priya Patel	HR	65,000	Priya Patel
Neha Gupta	IT	72,000	Karthik Nair
Aarav Sharma	IT	75,000	Karthik Nair

name	department	salary	highest_paid
Arjun Reddy	IT	82,000	Karthik Nair
Suresh Babu	IT	88,000	Karthik Nair
Vikram Reddy	IT	90,000	Karthik Nair
Ravi Teja	IT	91,000	Karthik Nair
Karthik Nair	IT	95,000	Karthik Nair
Sneha Patel	Marketing	58,000	Meera Krishnan
Meera Krishnan	Marketing	68,000	Meera Krishnan

15 rows returned.

Q148. NTILE() - Divide into quartiles MEDIUM

```
SELECT name, salary,
       NTILE(4) OVER (ORDER BY salary) AS salary_quartile
FROM employees;
```

WHAT THIS QUERY DOES

`NTILE(4)` divides the ordered rows into 4 roughly-equal-sized buckets (quartiles) and labels each row with which bucket it landed in — bucket 1 holds the lowest-salary employees, bucket 4 the highest. With 15 employees split 4 ways, some buckets naturally end up with 4 people and others with 3, since 15 doesn't divide evenly by 4; `NTILE()` handles that automatically by giving the earlier buckets the extra rows.

RESULT ON OUR SAMPLE DATA

name	salary	salary_quartile
Sneha Patel	58,000	1
Ananya Menon	60,000	1
Deepa Iyer	64,000	1
Priya Patel	65,000	1
Kavya Nair	67,000	2
Meera Krishnan	68,000	2
Rajesh Kumar	70,000	2
Neha Gupta	72,000	2
Aarav Sharma	75,000	3
Arjun Reddy	82,000	3
Rahul Singh	85,000	3
Suresh Babu	88,000	3
Vikram Reddy	90,000	4
Ravi Teja	91,000	4

name	salary	salary_quartile
Karthik Nair	95,000	4

15 rows returned.

Q149. Running total of salaries MEDIUM

```
SELECT name, salary,
       SUM(salary) OVER (ORDER BY emp_id) AS running_total
FROM employees;
```

WHAT THIS QUERY DOES

A **running total**: ordering by `emp_id`, each row's `running_total` is the sum of every salary *up to and including* the current row. This works because, as mentioned in Q147, a window function's default frame is 'from the start of the partition up through the current row' when an `ORDER BY` is present — which is exactly the shape a running total needs, with no extra `ROWS BETWEEN` clause required.

RESULT ON OUR SAMPLE DATA

name	salary	running_total
Aarav Sharma	75,000	75,000
Priya Patel	65,000	140,000
Rahul Singh	85,000	225,000
Neha Gupta	72,000	297,000
Vikram Reddy	90,000	387,000
Ananya Menon	60,000	447,000
Karthik Nair	95,000	542,000
Meera Krishnan	68,000	610,000
Arjun Reddy	82,000	692,000
Sneha Patel	58,000	750,000
Rajesh Kumar	70,000	820,000
Deepa Iyer	64,000	884,000
Suresh Babu	88,000	972,000
Kavya Nair	67,000	1,039,000
Ravi Teja	91,000	1,130,000

15 rows returned.

Q150. Cumulative salary by department MEDIUM

```
SELECT name, department, salary,
       SUM(salary) OVER (PARTITION BY department ORDER BY emp_id) AS
dept_cumulative
FROM employees;
```

WHAT THIS QUERY DOES

Combines the running-total idea from Q149 with `PARTITION BY` from Q146 — now the running total resets separately at the start of each department, rather than accumulating across the whole table. This is the standard shape for 'running total, but broken out per category' reports.

RESULT ON OUR SAMPLE DATA

name	department	salary	dept_cumulative
Rahul Singh	Finance	85,000	85,000
Rajesh Kumar	Finance	70,000	155,000
Kavya Nair	Finance	67,000	222,000
Priya Patel	HR	65,000	65,000
Ananya Menon	HR	60,000	125,000
Deepa Iyer	HR	64,000	189,000
Aarav Sharma	IT	75,000	75,000
Neha Gupta	IT	72,000	147,000
Vikram Reddy	IT	90,000	237,000
Karthik Nair	IT	95,000	332,000
Arjun Reddy	IT	82,000	414,000
Suresh Babu	IT	88,000	502,000
Ravi Teja	IT	91,000	593,000
Meera Krishnan	Marketing	68,000	68,000
Sneha Patel	Marketing	58,000	126,000

15 rows returned.

Q151. Moving average of last 3 employees HARD

```
SELECT name, salary,
       AVG(salary) OVER (ORDER BY emp_id ROWS BETWEEN 2 PRECEDING AND CURRENT
ROW) AS moving_avg
FROM employees;
```

WHAT THIS QUERY DOES

A **moving average**: instead of the default 'from the start' frame, `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` explicitly defines a sliding 3-row window (the current row plus the 2 before it), and `AVG()` is computed fresh within that window at every position. Near the very start of the table, where fewer than 2 prior rows exist, the window simply shrinks to whatever's available rather than erroring out.

RESULT ON OUR SAMPLE DATA

name	salary	moving_avg
Aarav Sharma	75,000	75,000
Priya Patel	65,000	70,000
Rahul Singh	85,000	75,000
Neha Gupta	72,000	74,000
Vikram Reddy	90,000	82,333.33
Ananya Menon	60,000	74,000
Karthik Nair	95,000	81,666.67
Meera Krishnan	68,000	74,333.33
Arjun Reddy	82,000	81,666.67
Sneha Patel	58,000	69,333.33
Rajesh Kumar	70,000	70,000
Deepa Iyer	64,000	64,000
Suresh Babu	88,000	74,000
Kavya Nair	67,000	73,000
Ravi Teja	91,000	82,000

15 rows returned.

Q152. Top 3 salaries per department HARD

```
WITH ranked AS (
    SELECT name, department, salary,
           DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS dr
    FROM employees
)
SELECT department, name, salary
FROM ranked
WHERE dr <= 3;
```

WHAT THIS QUERY DOES

This introduces a **CTE** (Common Table Expression, the `WITH ... AS (...)` syntax) — a named, temporary result set you can reference like a table in the query that follows. It's needed here because of a strict SQL rule: you're **not allowed to filter directly on a window function's result in the same SELECT that computes it** (writing `WHERE dr <= 3` right next to `DENSE_RANK() ... AS dr` would be a syntax

error). The workaround is always the same two-step shape: compute the ranking inside a CTE first, then filter on it in an outer query that reads from the CTE. This exact pattern — rank inside a CTE, filter outside — is one of the most useful things to memorize from this whole book.

RESULT ON OUR SAMPLE DATA

department	name	salary
Finance	Rahul Singh	85,000
Finance	Rajesh Kumar	70,000
Finance	Kavya Nair	67,000
HR	Priya Patel	65,000
HR	Deepa Iyer	64,000
HR	Ananya Menon	60,000
IT	Karthik Nair	95,000
IT	Ravi Teja	91,000
IT	Vikram Reddy	90,000
Marketing	Meera Krishnan	68,000
Marketing	Sneha Patel	58,000

11 rows returned.

Q153. Lowest salary per department HARD

```
WITH ranked AS (
    SELECT name, department, salary,
           RANK() OVER (PARTITION BY department ORDER BY salary) AS rn
    FROM employees
)
SELECT department, name, salary
FROM ranked
WHERE rn = 1;
```

WHAT THIS QUERY DOES

The same CTE-then-filter shape as Q152, just ranking ascending (so rank 1 means *lowest* salary) and keeping only rank 1. This is functionally the same result as the correlated-subquery approach in Q128 — a good example of two completely different techniques (window function + CTE vs. correlated subquery) arriving at the same answer, each with its own strengths (window functions tend to be easier to extend to 'top N' as done in Q152, while correlated subqueries can be simpler for a single MIN/MAX).

RESULT ON OUR SAMPLE DATA

department	name	salary
Finance	Kavya Nair	67,000
HR	Ananya Menon	60,000
IT	Neha Gupta	72,000
Marketing	Sneha Patel	58,000

4 rows returned.

Q154. Previous salary comparison MEDIUM

```
SELECT name, salary,
       LAG(salary) OVER (ORDER BY salary) AS prev_salary,
       salary - LAG(salary) OVER (ORDER BY salary) AS diff_from_prev
FROM employees;
```

WHAT THIS QUERY DOES

Builds directly on Q145 by using the same LAG() value twice — once to display it, and once inside a subtraction to compute the gap between each employee's salary and the person just below them in the ranking. The very first row's difference is NULL, since LAG() itself is NULL there and any arithmetic involving NULL also produces NULL.

RESULT ON OUR SAMPLE DATA

name	salary	prev_salary	diff_from_prev
Sneha Patel	58,000	*NULL*	*NULL*
Ananya Menon	60,000	58,000	2,000
Deepa Iyer	64,000	60,000	4,000
Priya Patel	65,000	64,000	1,000
Kavya Nair	67,000	65,000	2,000
Meera Krishnan	68,000	67,000	1,000
Rajesh Kumar	70,000	68,000	2,000
Neha Gupta	72,000	70,000	2,000
Aarav Sharma	75,000	72,000	3,000
Arjun Reddy	82,000	75,000	7,000
Rahul Singh	85,000	82,000	3,000
Suresh Babu	88,000	85,000	3,000
Vikram Reddy	90,000	88,000	2,000
Ravi Teja	91,000	90,000	1,000
Karthik Nair	95,000	91,000	4,000

15 rows returned.

Q155. Next salary comparison MEDIUM

```
SELECT name, salary,
       LEAD(salary) OVER (ORDER BY salary) AS next_salary,
       LEAD(salary) OVER (ORDER BY salary) - salary AS diff_to_next
FROM employees;
```

WHAT THIS QUERY DOES

The LEAD() counterpart to Q154 — shows the gap up to the *next*-higher salary instead of down to the previous one. Comparing the two diff columns across both queries is a good way to see that 'gap to the next person up' and 'gap from the previous person' are just the same set of gaps, viewed from opposite directions.

RESULT ON OUR SAMPLE DATA

name	salary	next_salary	diff_to_next
Sneha Patel	58,000	60,000	2,000
Ananya Menon	60,000	64,000	4,000
Deepa Iyer	64,000	65,000	1,000
Priya Patel	65,000	67,000	2,000
Kavya Nair	67,000	68,000	1,000
Meera Krishnan	68,000	70,000	2,000
Rajesh Kumar	70,000	72,000	2,000
Neha Gupta	72,000	75,000	3,000
Aarav Sharma	75,000	82,000	7,000
Arjun Reddy	82,000	85,000	3,000
Rahul Singh	85,000	88,000	3,000
Suresh Babu	88,000	90,000	2,000
Vikram Reddy	90,000	91,000	1,000
Ravi Teja	91,000	95,000	4,000
Karthik Nair	95,000	*NULL*	*NULL*

15 rows returned.

Q156. Month-over-month sales HARD

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
       SUM(amount) AS total_sales,
       LAG(SUM(amount)) OVER (ORDER BY DATE_FORMAT(order_date, '%Y-%m')) AS
prev_month_sales,
       (SUM(amount) - LAG(SUM(amount)) OVER (ORDER BY DATE_FORMAT(order_date,
'%Y-%m'))) /
        LAG(SUM(amount)) OVER (ORDER BY DATE_FORMAT(order_date, '%Y-%m')) * 100
```

```
AS growth_percent
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m');
```

WHAT THIS QUERY DOES

This shows that `LAG()/LEAD()` can be applied *on top of an aggregate*, not just on raw columns: the query first groups orders by month and sums each month's revenue, and only then does `LAG(SUM(amount))` reach back to grab the *previous month's already-summed total* — letting you compute a month-over-month growth percentage in a single query, without a self-join or a separate 'previous month' subquery. *(`DATE_FORMAT()` is a MySQL function; the live example uses SQLite's `strftime('%Y-%m', order_date)` to the same effect.)*

RESULT ON OUR SAMPLE DATA

month	total_sales	prev_month_sales	growth_percent
2024-01	50,000	*NULL*	*NULL*
2024-02	145,000	50,000	190
2024-03	150,000	145,000	3.45
2024-04	119,000	150,000	-20.67

4 rows returned.

Q157. Daily revenue with running total MEDIUM

```
SELECT order_date,
       SUM(amount) AS daily_revenue,
       SUM(SUM(amount)) OVER (ORDER BY order_date) AS cumulative_revenue
FROM orders
GROUP BY order_date
ORDER BY order_date;
```

WHAT THIS QUERY DOES

Notice the **nested SUMs**: the inner `SUM(amount)` is a normal GROUP BY aggregate producing one total per day, while the outer `SUM(...) OVER (ORDER BY order_date)` is a window function running a cumulative total *across those already-aggregated daily totals*. Aggregate functions and window functions can combine like this — it's a slightly advanced-looking pattern, but it's really just 'Q149's running total, applied to grouped daily sums instead of individual raw rows.'

RESULT ON OUR SAMPLE DATA

order_date	daily_revenue	cumulative_revenue
2024-01-10	15,000	15,000
2024-01-15	23,000	38,000
2024-01-20	12,000	50,000

order_date	daily_revenue	cumulative_revenue
2024-02-01	50,000	100,000
2024-02-05	18,000	118,000
2024-02-10	32,000	150,000
2024-02-15	45,000	195,000
2024-03-01	21,000	216,000
2024-03-05	67,000	283,000
2024-03-10	34,000	317,000
2024-03-15	28,000	345,000
2024-04-01	42,000	387,000
2024-04-05	15,000	402,000
2024-04-10	33,000	435,000
2024-04-15	29,000	464,000

15 rows returned.

Q158. Customer ranking by total spent MEDIUM

```
SELECT c.name,
       SUM(o.amount) AS total_spent,
       RANK() OVER (ORDER BY SUM(o.amount) DESC) AS customer_rank
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

WHAT THIS QUERY DOES

JOIN + GROUP BY (to compute each customer's total spend) combined with a window function (to rank those totals) — all three major query-building techniques from this book working together in one query. This is a very realistic 'top customers' report shape.

RESULT ON OUR SAMPLE DATA

name	total_spent	customer_rank
InnovationLabs	97,000	1
DeltaCorp	67,000	2
TechCorp	53,000	3
GlobalSolutions	45,000	4
BetaSystems	45,000	4
OmegaLabs	42,000	6
EpsilonSoft	34,000	7
AlphaInc	32,000	8

name	total_spent	customer_rank
ThetaSolutions	28,000	9
GammaTech	21,000	10

10 rows returned.

Q159. Product ranking by sales MEDIUM

```
SELECT p.product_name,
       SUM(oi.quantity) AS total_quantity_sold,
       RANK() OVER (ORDER BY SUM(oi.quantity) DESC) AS product_rank
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name;
```

WHAT THIS QUERY DOES

The same JOIN + GROUP BY + RANK shape as Q158, applied to product sales volume instead of customer spend. LEFT JOIN (rather than a plain JOIN) ensures a product that's never been ordered still shows up ranked last, with a quantity of 0, instead of being silently excluded — the same reasoning as Q104.

RESULT ON OUR SAMPLE DATA

product_name	total_quantity_sold	product_rank
Laptop Pro	9	1
External SSD	4	2
27" Monitor	3	3
Enterprise Software	3	3
Wireless Mouse	1	5
Mechanical Keyboard	1	5
Cloud Storage Plan	1	5
Webcam HD	*NULL*	8
Printer	*NULL*	8
USB Hub	*NULL*	8

10 rows returned.

Q160. Department ranking by total salary EASY

```
SELECT department,
       SUM(salary) AS total_salary,
       RANK() OVER (ORDER BY SUM(salary) DESC) AS dept_rank
```

```
FROM employees
GROUP BY department;
```

WHAT THIS QUERY DOES

Same GROUP BY + RANK combination once more, this time needing no JOIN at all since department and salary already live in the same table — a simpler variant of the Q158/159 shape.

RESULT ON OUR SAMPLE DATA

department	total_salary	dept_rank
IT	593,000	1
Finance	222,000	2
HR	189,000	3
Marketing	126,000	4

4 rows returned.

Q161. Percentage of total salary by employee MEDIUM

```
SELECT name, salary,
       ROUND(salary * 100.0 / SUM(salary) OVER (), 2) AS pct_of_total
FROM employees
ORDER BY pct_of_total DESC;
```

WHAT THIS QUERY DOES

An empty `OVER ()` — with no `PARTITION BY` and no `ORDER BY` — means 'treat the entire table as one single window.' `SUM(salary) OVER ()` therefore computes the grand total salary *once*, and repeats that same total alongside every row, which is exactly what you need as the denominator to calculate what percentage of total payroll each individual employee represents.

RESULT ON OUR SAMPLE DATA

name	salary	pct_of_total
Karthik Nair	95,000	8.41
Ravi Teja	91,000	8.05
Vikram Reddy	90,000	7.96
Suresh Babu	88,000	7.79
Rahul Singh	85,000	7.52
Arjun Reddy	82,000	7.26
Aarav Sharma	75,000	6.64
Neha Gupta	72,000	6.37
Rajesh Kumar	70,000	6.19

name	salary	pct_of_total
Meera Krishnan	68,000	6.02
Kavya Nair	67,000	5.93
Priya Patel	65,000	5.75
Deepa Iyer	64,000	5.66
Ananya Menon	60,000	5.31
Sneha Patel	58,000	5.13

15 rows returned.

Q162. Percentage within department MEDIUM

```
SELECT name, department, salary,
       ROUND(salary * 100.0 / SUM(salary) OVER (PARTITION BY department), 2) AS
pct_of_dept
FROM employees
ORDER BY department, pct_of_dept DESC;
```

WHAT THIS QUERY DOES

The same percentage-of-total idea as Q161, but adding `PARTITION BY department` so the denominator becomes each department's own total instead of the company-wide total — answering 'what share of my *department*'s payroll do I represent' rather than 'what share of the whole company.'

RESULT ON OUR SAMPLE DATA

name	department	salary	pct_of_dept
Rahul Singh	Finance	85,000	38.29
Rajesh Kumar	Finance	70,000	31.53
Kavya Nair	Finance	67,000	30.18
Priya Patel	HR	65,000	34.39
Deepa Iyer	HR	64,000	33.86
Ananya Menon	HR	60,000	31.75
Karthik Nair	IT	95,000	16.02
Ravi Teja	IT	91,000	15.35
Vikram Reddy	IT	90,000	15.18
Suresh Babu	IT	88,000	14.84
Arjun Reddy	IT	82,000	13.83
Aarav Sharma	IT	75,000	12.65
Neha Gupta	IT	72,000	12.14
Meera Krishnan	Marketing	68,000	53.97
Sneha Patel	Marketing	58,000	46.03

15 rows returned.

Q163. Salary percentile rank HARD

```
SELECT name, salary,
       PERCENT_RANK() OVER (ORDER BY salary) AS percentile_rank
FROM employees;
```

WHAT THIS QUERY DOES

PERCENT_RANK() expresses each row's rank as a fraction between 0 and 1, using the formula (rank - 1) / (total rows - 1) — the lowest value always scores exactly 0, the highest always scores exactly 1, and everyone else falls proportionally in between. This is the standard way to answer 'what percentile is this salary in' without manually computing rank-to-percentage conversions yourself.

RESULT ON OUR SAMPLE DATA

name	salary	percentile_rank
Sneha Patel	58,000	0
Ananya Menon	60,000	0.07
Deepa Iyer	64,000	0.14
Priya Patel	65,000	0.21
Kavya Nair	67,000	0.29
Meera Krishnan	68,000	0.36
Rajesh Kumar	70,000	0.43
Neha Gupta	72,000	0.50
Aarav Sharma	75,000	0.57
Arjun Reddy	82,000	0.64
Rahul Singh	85,000	0.71
Suresh Babu	88,000	0.79
Vikram Reddy	90,000	0.86
Ravi Teja	91,000	0.93
Karthik Nair	95,000	1

15 rows returned.

Q164. Cumulative distribution of salaries HARD

```
SELECT name, salary,
       CUME_DIST() OVER (ORDER BY salary) AS cume_distribution
FROM employees;
```

WHAT THIS QUERY DOES

Closely related to `PERCENT_RANK()` but subtly different: `CUME_DIST()` (cumulative distribution) reports, for each row, the fraction of *all* rows whose value is less than or equal to the current one. Unlike `PERCENT_RANK()`, the lowest value here doesn't score 0 — it scores 1/15 (itself counts as 'less than or equal to itself'), and the highest value always scores exactly 1.

RESULT ON OUR SAMPLE DATA

name	salary	cume_distribution
Sneha Patel	58,000	0.07
Ananya Menon	60,000	0.13
Deepa Iyer	64,000	0.20
Priya Patel	65,000	0.27
Kavya Nair	67,000	0.33
Meera Krishnan	68,000	0.40
Rajesh Kumar	70,000	0.47
Neha Gupta	72,000	0.53
Aarav Sharma	75,000	0.60
Arjun Reddy	82,000	0.67
Rahul Singh	85,000	0.73
Suresh Babu	88,000	0.80
Vikram Reddy	90,000	0.87
Ravi Teja	91,000	0.93
Karthik Nair	95,000	1

15 rows returned.

Q165. Employee tenure ranking MEDIUM

```
SELECT name,
       DATEDIFF(CURDATE(), hire_date) AS tenure_days,
       ROW_NUMBER() OVER (ORDER BY DATEDIFF(CURDATE(), hire_date) DESC) AS
tenure_rank
FROM employees;
```

WHAT THIS QUERY DOES

`DATEDIFF()` computes the number of days between two dates, and `CURDATE()` returns today's date — together giving each employee's tenure in days, which `ROW_NUMBER()` then ranks from longest-serving to newest. *Since 'today' would make this book's printed numbers go stale the moment you read it, the live example below fixes the reference date at 2024-12-31 (using SQLite's `julianday()` date arithmetic) instead of the real current date, so the tenure figures shown stay consistent and reproducible.*

RESULT ON OUR SAMPLE DATA

name	tenure_days	tenure_rank
Suresh Babu	3,958	1
Karthik Nair	3,380	2
Ravi Teja	3,091	3
Arjun Reddy	2,952	4
Vikram Reddy	2,593	5
Rahul Singh	2,366	6
Priya Patel	2,113	7
Rajesh Kumar	2,031	8
Aarav Sharma	1,812	9
Meera Krishnan	1,602	10
Deepa Iyer	1,507	11
Neha Gupta	1,429	12
Kavya Nair	1,229	13
Ananya Menon	930	14
Sneha Patel	711	15

15 rows returned.

Q166. Moving 7-day average of orders HARD

```
WITH daily_orders AS (
    SELECT order_date, COUNT(*) AS order_count
    FROM orders
    GROUP BY order_date
)
SELECT order_date, order_count,
       AVG(order_count) OVER (ORDER BY order_date ROWS BETWEEN 6 PRECEDING AND
CURRENT ROW) AS moving_7day_avg
FROM daily_orders
ORDER BY order_date;
```

WHAT THIS QUERY DOES

Combines a CTE (first collapse orders down to one row per day with its order count) and a sliding-window moving average (same `ROWS BETWEEN ... PRECEDING AND CURRENT ROW` idea as Q151, just widened from 3 rows to 7). Since our sample data only has one order per distinct date, the '7-day' window here mostly just averages however many daily-order-count rows came before it — but on denser real-world data, this is exactly how you'd smooth out day-to-day noise in an orders-per-day chart.

RESULT ON OUR SAMPLE DATA

order_date	order_count	moving_7day_avg
2024-01-10	1	1
2024-01-15	1	1
2024-01-20	1	1
2024-02-01	1	1
2024-02-05	1	1
2024-02-10	1	1
2024-02-15	1	1
2024-03-01	1	1
2024-03-05	1	1
2024-03-10	1	1
2024-03-15	1	1
2024-04-01	1	1
2024-04-05	1	1
2024-04-10	1	1
2024-04-15	1	1

15 rows returned.

Q167. Salary difference from department average MEDIUM

```
SELECT name, department, salary,
       AVG(salary) OVER (PARTITION BY department) AS dept_avg,
       salary - AVG(salary) OVER (PARTITION BY department) AS diff_from_dept_avg
FROM employees;
```

WHAT THIS QUERY DOES

A window-function alternative to the correlated subquery from Q123/116: `AVG(salary) OVER (PARTITION BY department)` computes each department's average salary and attaches it directly alongside every one of that department's rows, letting you compute the difference in the very same SELECT — generally considered more readable (and often faster) than the equivalent correlated subquery.

RESULT ON OUR SAMPLE DATA

name	department	salary	dept_avg	diff_from_dept_avg
Rahul Singh	Finance	85,000	74,000	11,000
Rajesh Kumar	Finance	70,000	74,000	-4,000
Kavya Nair	Finance	67,000	74,000	-7,000
Priya Patel	HR	65,000	63,000	2,000
Ananya Menon	HR	60,000	63,000	-3,000
Deepa Iyer	HR	64,000	63,000	1,000
Aarav Sharma	IT	75,000	84,714.29	-9,714.29
Neha Gupta	IT	72,000	84,714.29	-12,714.29
Vikram Reddy	IT	90,000	84,714.29	5,285.71
Karthik Nair	IT	95,000	84,714.29	10,285.71
Arjun Reddy	IT	82,000	84,714.29	-2,714.29
Suresh Babu	IT	88,000	84,714.29	3,285.71
Ravi Teja	IT	91,000	84,714.29	6,285.71
Meera Krishnan	Marketing	68,000	63,000	5,000
Sneha Patel	Marketing	58,000	63,000	-5,000

15 rows returned.

Q168. Running total per month MEDIUM

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
       SUM(amount) AS monthly_sales,
       SUM(SUM(amount)) OVER (ORDER BY DATE_FORMAT(order_date, '%Y-%m')) AS
cumulative_ytd
FROM orders
GROUP BY DATE_FORMAT(order_date, '%Y-%m');
```

WHAT THIS QUERY DOES

The same 'nested SUM' running-total idea from Q157, now grouped by month instead of by day — producing a year-to-date cumulative revenue figure that grows month over month. *(Again using SQLite's `strftime('%Y-%m', ...)` in place of MySQL's `DATE_FORMAT()`.)*

RESULT ON OUR SAMPLE DATA

month	monthly_sales	cumulative_ytd
2024-01	50,000	50,000
2024-02	145,000	195,000
2024-03	150,000	345,000
2024-04	119,000	464,000

4 rows returned.

Q169. First order for each customer MEDIUM

```
SELECT customer_id, order_date, amount,
       FIRST_VALUE(order_date) OVER (PARTITION BY customer_id ORDER BY
order_date) AS first_order_date
FROM orders;
```

WHAT THIS QUERY DOES

The window-function alternative to Q129/130's correlated-subquery approach: `FIRST_VALUE()` with `PARTITION BY customer_id` attaches each customer's earliest order date to every one of their order rows directly, without needing a separate subquery per row.

RESULT ON OUR SAMPLE DATA

customer_id	order_date	amount	first_order_date
1	2024-01-10	15,000	2024-01-10
1	2024-01-15	23,000	2024-01-10
1	2024-04-05	15,000	2024-01-10
2	2024-01-20	12,000	2024-01-20
2	2024-04-10	33,000	2024-01-20
3	2024-02-01	50,000	2024-02-01
3	2024-02-05	18,000	2024-02-01
3	2024-04-15	29,000	2024-02-01
4	2024-02-10	32,000	2024-02-10
5	2024-02-15	45,000	2024-02-15
6	2024-03-01	21,000	2024-03-01
7	2024-03-05	67,000	2024-03-05
8	2024-03-10	34,000	2024-03-10
9	2024-03-15	28,000	2024-03-15
10	2024-04-01	42,000	2024-04-01

15 rows returned.

Q170. Last order for each customer MEDIUM

```
SELECT customer_id, order_date, amount,
       LAST_VALUE(order_date) OVER (PARTITION BY customer_id ORDER BY order_date
                                    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
```

```
FOLLOWING) AS last_order_date
FROM orders;
```

WHAT THIS QUERY DOES

The `LAST_VALUE()` counterpart to Q169 — and just like Q147, it needs the explicit `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` frame to correctly look at the *entire* partition rather than defaulting to 'up through the current row,' which would otherwise make every row just echo its own date back.

RESULT ON OUR SAMPLE DATA

customer_id	order_date	amount	last_order_date
1	2024-01-10	15,000	2024-04-05
1	2024-01-15	23,000	2024-04-05
1	2024-04-05	15,000	2024-04-05
2	2024-01-20	12,000	2024-04-10
2	2024-04-10	33,000	2024-04-10
3	2024-02-01	50,000	2024-04-15
3	2024-02-05	18,000	2024-04-15
3	2024-04-15	29,000	2024-04-15
4	2024-02-10	32,000	2024-02-10
5	2024-02-15	45,000	2024-02-15
6	2024-03-01	21,000	2024-03-01
7	2024-03-05	67,000	2024-03-05
8	2024-03-10	34,000	2024-03-10
9	2024-03-15	28,000	2024-03-15
10	2024-04-01	42,000	2024-04-01

15 rows returned.

Level 7: CTE & Recursive Queries

CTEs (WITH ... AS ...) give a subquery a readable name you can reuse, and chain multiple named steps together. Recursive CTEs go further, letting a query call itself to walk hierarchies of unknown depth -- the single technique behind org charts, category trees, folder structures, and bill-of-materials explosions.

Q171. Simple CTE - High earners EASY

```
WITH high_earners AS (
    SELECT name, salary, department
    FROM employees
    WHERE salary > 80000
)
SELECT * FROM high_earners;
```

WHAT THIS QUERY DOES

You've already seen CTEs used as a workaround in Level 6 (Q152/153) — here's the pattern in its simplest form. `WITH high_earners AS (...)` defines a named, temporary result set, and the query below it can then reference `high_earners` exactly as if it were a real table. For a query this simple, a CTE isn't strictly necessary (you could just filter `employees` directly) — its real value shows up once a query gets complex enough that giving an intermediate step its own name makes the whole thing easier to read, as the next few questions demonstrate.

RESULT ON OUR SAMPLE DATA

name	salary	department
Rahul Singh	85,000	Finance
Vikram Reddy	90,000	IT
Karthik Nair	95,000	IT
Arjun Reddy	82,000	IT
Suresh Babu	88,000	IT
Ravi Teja	91,000	IT

6 rows returned.

Q172. Multiple CTEs - Department stats MEDIUM

```
WITH dept_avg AS (
    SELECT department, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department
```

```
),
dept_count AS (
    SELECT department, COUNT(*) AS emp_count
    FROM employees
    GROUP BY department
)
SELECT da.department, da.avg_salary, dc.emp_count
FROM dept_avg da
JOIN dept_count dc ON da.department = dc.department;
```

WHAT THIS QUERY DOES

You can define *multiple* CTEs in one `WITH` clause, separated by commas, and even join them together afterward as if they were ordinary tables. This is often clearer than nesting subqueries directly inside a JOIN's `ON` clause (as Q105 did) — each CTE gets a readable name and can be understood on its own, one piece at a time, before seeing how they're combined.

RESULT ON OUR SAMPLE DATA

department	avg_salary	emp_count
Finance	74,000	3
HR	63,000	3
IT	84,714.29	7
Marketing	63,000	2

4 rows returned.

Q173. Recursive employee hierarchy HARD

```
WITH RECURSIVE emp_hierarchy AS (
    -- Base case: top-level employees
    SELECT emp_id, name, manager_id, 0 AS level, CAST(name AS CHAR(500)) AS path
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    -- Recursive case: direct reports
    SELECT e.emp_id, e.name, e.manager_id, h.level + 1,
           CONCAT(h.path, ' -> ', e.name)
    FROM employees e
    JOIN emp_hierarchy h ON e.manager_id = h.emp_id
)
SELECT * FROM emp_hierarchy ORDER BY level, emp_id;
```

WHAT THIS QUERY DOES

Your first **recursive CTE** — a CTE that references *itself*, used whenever data has an unknown, variable depth (an org chart can be 2 levels deep or 10; you don't know in

advance). It always has two parts joined by `UNION ALL`: a **base case** (here, employees with no manager — the top of the tree, at level 0) and a **recursive case** (employees whose manager was found in the *previous* round, each one level deeper than their manager). SQL keeps re-running the recursive part, feeding each round's results back in as input to the next, until a round produces no new rows — at which point it stops automatically. The `path` column, built up with string concatenation at each step, is a nice bonus that shows the full management chain down to each employee.

RESULT ON OUR SAMPLE DATA

emp_id	name	manager_id	level	path
1	Aarav Sharma	*NULL*	0	Aarav Sharma
2	Priya Patel	1	1	Aarav Sharma -> Priya Patel
3	Rahul Singh	1	1	Aarav Sharma -> Rahul Singh
4	Neha Gupta	1	1	Aarav Sharma -> Neha Gupta
5	Vikram Reddy	1	1	Aarav Sharma -> Vikram Reddy
7	Karthik Nair	1	1	Aarav Sharma -> Karthik Nair
15	Ravi Teja	1	1	Aarav Sharma -> Ravi Teja
6	Ananya Menon	2	2	Aarav Sharma -> Priya Patel -> Ananya Menon
8	Meera Krishnan	5	2	Aarav Sharma -> Vikram Reddy -> Meera Krishnan
9	Arjun Reddy	5	2	Aarav Sharma -> Vikram Reddy -> Arjun Reddy
11	Rajesh Kumar	3	2	Aarav Sharma -> Rahul Singh -> Rajesh Kumar
12	Deepa Iyer	2	2	Aarav Sharma -> Priya Patel -> Deepa Iyer
13	Suresh Babu	5	2	Aarav Sharma -> Vikram Reddy -> Suresh Babu
14	Kavya Nair	3	2	Aarav Sharma -> Rahul Singh -> Kavya Nair
10	Sneha Patel	8	3	Aarav Sharma -> Vikram Reddy -> Meera Krishnan -> Sneha Patel

15 rows returned.

Q174. Recursive category hierarchy MEDIUM

```
-- Example: Product categories with parent-child
CREATE TABLE categories (
    cat_id INT PRIMARY KEY,
    cat_name VARCHAR(100),
    parent_id INT
);

WITH RECURSIVE category_tree AS (
    SELECT cat_id, cat_name, parent_id, 0 AS level, cat_name AS path
    FROM categories
```

```
    WHERE parent_id IS NULL

    UNION ALL

    SELECT c.cat_id, c.cat_name, c.parent_id, ct.level + 1,
           CONCAT(ct.path, ' > ', c.cat_name)
    FROM categories c
    JOIN category_tree ct ON c.parent_id = ct.cat_id
)
SELECT * FROM category_tree ORDER BY level, cat_id;
```

WHAT THIS QUERY DOES

An illustrative `categories` table has been added (Electronics > Computers > Laptops > Gaming Laptops, plus a separate Accessories and Software branch) to demonstrate this concretely. Structurally, this is the exact same recursive-CTE template as Q173's employee hierarchy — a self-referencing `parent_id` column, a base case for top-level rows, and a recursive case joining each round's results back to their children. Once you recognize this shape, it applies to *any* parent-child hierarchy: org charts, product categories, folder trees (Q180), threaded comments, and more — the table changes, but the recursive-CTE skeleton stays identical.

RESULT ON OUR SAMPLE DATA

cat_id	cat_name	parent_id	level	path
1	Electronics	*NULL*	0	Electronics
6	Software	*NULL*	0	Software
2	Computers	1	1	Electronics > Computers
5	Accessories	1	1	Electronics > Accessories
3	Laptops	2	2	Electronics > Computers > Laptops
4	Gaming Laptops	3	3	Electronics > Computers > Laptops > Gaming Laptops

6 rows returned.

Q175. Generate numbers 1 to 100 MEDIUM

```
WITH RECURSIVE numbers AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM numbers WHERE n < 100
)
SELECT * FROM numbers;
```

WHAT THIS QUERY DOES

A recursive CTE doesn't have to reference a real table at all — here the base case is just the literal number 1, and the recursive case adds 1 each round, stopping once n reaches 100. This 'number generator' pattern is genuinely useful in practice: it's a common way to fill in gaps in a report (e.g. showing '0 orders' for days that had no

orders at all, which a plain GROUP BY would simply omit) by generating every possible number/date first and then LEFT JOINing your real data onto it.

RESULT ON OUR SAMPLE DATA

n
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20

Showing first 20 of 100 rows.

Q176. Calendar generation for 2024 MEDIUM

```
WITH RECURSIVE calendar AS (
    SELECT '2024-01-01' AS date
    UNION ALL
    SELECT DATE_ADD(date, INTERVAL 1 DAY)
    FROM calendar
    WHERE date < '2024-12-31'
)
SELECT date, DAYNAME(date) AS day_of_week FROM calendar;
```

WHAT THIS QUERY DOES

The same number-generator idea from Q175, applied to dates instead of integers: start at Jan 1st, and each recursive round adds one more day, until the end of the

year is reached. Generating a full calendar table like this is the standard trick for reports that need every date to appear (even ones with zero activity) rather than only the dates that happen to already exist in your data. *(The live example below generates just the first 10 days rather than all 365, purely to keep the printed table a reasonable size — the technique is identical either way. `DATE_ADD`/`DAYNAME` are MySQL functions; SQLite's equivalents are `date(d, '+1 day')` and `strftime('%w', d)`, the latter returning a weekday number 0-6 rather than a name.)*

RESULT ON OUR SAMPLE DATA

date	day_of_week
2024-01-01	1
2024-01-02	2
2024-01-03	3
2024-01-04	4
2024-01-05	5
2024-01-06	6
2024-01-07	0
2024-01-08	1
2024-01-09	2
2024-01-10	3

10 rows returned.

Q177. Running balance with CTE MEDIUM

```
WITH transactions AS (
    SELECT 1 AS txn_id, '2024-01-01' AS txn_date, 1000 AS amount, 'credit' AS
type
    UNION ALL SELECT 2, '2024-01-02', 200, 'debit'
    UNION ALL SELECT 3, '2024-01-03', 500, 'credit'
),
balance AS (
    SELECT txn_id, txn_date, amount, type,
           SUM(CASE WHEN type = 'credit' THEN amount ELSE -amount END)
               OVER (ORDER BY txn_date) AS running_balance
    FROM transactions
)
SELECT * FROM balance;
```

WHAT THIS QUERY DOES

Two things worth noticing here. First, the `transactions` CTE isn't built from a real table at all — it's hand-typed sample rows using `SELECT ... UNION ALL SELECT ...`, a handy way to mock up a small dataset for testing a query idea without needing to actually create and populate a table. Second, `CASE WHEN` inside

the `SUM() OVER (...)` turns debits into negative numbers before summing, so the running total correctly goes up on credits and down on debits — combining conditional logic with a window function's running-total pattern from Q149.

RESULT ON OUR SAMPLE DATA

txn_id	txn_date	amount	type	running_balance
1	2024-01-01	1,000	credit	1,000
2	2024-01-02	200	debit	800
3	2024-01-03	500	credit	1,300

3 rows returned.

Q178. Bill of materials explosion HARD

```
-- Example: Manufacturing BOM
CREATE TABLE bom (
    parent_id INT,
    component_id INT,
    quantity INT
);

WITH RECURSIVE bom_explosion AS (
    SELECT parent_id, component_id, quantity, 1 AS level
    FROM bom
    WHERE parent_id = 1  -- Start with product

    UNION ALL

    SELECT b.parent_id, b.component_id, b.quantity, be.level + 1
    FROM bom b
    JOIN bom_explosion be ON b.parent_id = be.component_id
)
SELECT * FROM bom_explosion;
```

WHAT THIS QUERY DOES

An illustrative `bom` (Bill of Materials) table has been added: product 1 is built from 1× component 2 and 4× component 3, and those components are themselves built from smaller sub-components. This is a recursive CTE with a manufacturing twist — instead of a hierarchy of *people* or *categories*, it's a hierarchy of *parts*, where each round of recursion 'explodes' one level further down into sub-components. This is exactly how manufacturing and inventory systems compute 'to build 10 finished products, how many of every individual screw, bolt, and sub-assembly do I actually need' — the same recursive-CTE skeleton as every other hierarchy question in this level, just applied to a bill of materials instead of an org chart.

RESULT ON OUR SAMPLE DATA

parent_id	component_id	quantity	level
1	2	1	1
1	3	4	1
2	4	2	2
3	5	1	2

4 rows returned.

Q179. Organization tree with depth HARD

```
WITH RECURSIVE org_tree AS (
    SELECT emp_id, name, manager_id, 0 AS depth,
           CAST(name AS CHAR(500)) AS org_path
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    SELECT e.emp_id, e.name, e.manager_id, ot.depth + 1,
           CONCAT(ot.org_path, ' > ', e.name)
    FROM employees e
    JOIN org_tree ot ON e.manager_id = ot.emp_id
)
SELECT depth, name, org_path
FROM org_tree
ORDER BY depth, name;
```

WHAT THIS QUERY DOES

Functionally identical to Q173's employee hierarchy — same base case, same recursive case, same path-building — just renaming `level` to `depth` and finishing with an `ORDER BY` that groups the output by organizational depth rather than by employee ID. Included again here by the original cheat sheet as a second worked example of the recursive-hierarchy pattern, since it's important enough to see twice.

RESULT ON OUR SAMPLE DATA

depth	name	org_path
0	Aarav Sharma	Aarav Sharma
1	Karthik Nair	Aarav Sharma > Karthik Nair
1	Neha Gupta	Aarav Sharma > Neha Gupta
1	Priya Patel	Aarav Sharma > Priya Patel
1	Rahul Singh	Aarav Sharma > Rahul Singh
1	Ravi Teja	Aarav Sharma > Ravi Teja
1	Vikram Reddy	Aarav Sharma > Vikram Reddy

depth	name	org_path
2	Ananya Menon	Aarav Sharma > Priya Patel > Ananya Menon
2	Arjun Reddy	Aarav Sharma > Vikram Reddy > Arjun Reddy
2	Deepa Iyer	Aarav Sharma > Priya Patel > Deepa Iyer
2	Kavya Nair	Aarav Sharma > Rahul Singh > Kavya Nair
2	Meera Krishnan	Aarav Sharma > Vikram Reddy > Meera Krishnan
2	Rajesh Kumar	Aarav Sharma > Rahul Singh > Rajesh Kumar
2	Suresh Babu	Aarav Sharma > Vikram Reddy > Suresh Babu
3	Sneha Patel	Aarav Sharma > Vikram Reddy > Meera Krishnan > Sneha Patel

15 rows returned.

Q180. Folder hierarchy MEDIUM

```
-- Example: File system folders
CREATE TABLE folders (
    folder_id INT PRIMARY KEY,
    folder_name VARCHAR(100),
    parent_id INT
);

WITH RECURSIVE folder_tree AS (
    SELECT folder_id, folder_name, parent_id, 0 AS level,
           folder_name AS full_path
    FROM folders
    WHERE parent_id IS NULL

    UNION ALL

    SELECT f.folder_id, f.folder_name, f.parent_id, ft.level + 1,
           CONCAT(ft.full_path, '/', f.folder_name)
    FROM folders f
    JOIN folder_tree ft ON f.parent_id = ft.folder_id
)
SELECT * FROM folder_tree ORDER BY full_path;
```

WHAT THIS QUERY DOES

An illustrative `folders` table has been added (`Documents/Projects/SQL-Book` and a separate `Photos/2024` branch). The final worked example of this level's recursive-hierarchy family, applied to a file system: exactly the same base case / recursive case / path-building skeleton as employee hierarchies (Q173/179) and category trees (Q174), just joined with '/' instead of ' > ' to produce familiar-looking file paths. By this point in the level, recognizing 'self-referencing parent_id column + need the full ancestry path' as the signal to reach for a recursive CTE should feel like second nature.

RESULT ON OUR SAMPLE DATA

folder_id	folder_name	parent_id	level	full_path
1	Documents	*NULL*	0	Documents
2	Projects	1	1	Documents/Projects
3	SQL-Book	2	2	Documents/Projects/SQL-Book
4	Photos	*NULL*	0	Photos
5	2024	4	1	Photos/2024

5 rows returned.

Level 8: Real Interview Problems (Questions 181–200)

These twenty questions are styled after real "company round" SQL interview questions. Each one states a business problem first, then the query that solves it, then a short breakdown of how the query actually works.

Q181: Amazon orders analysis

PROBLEM

Find products ordered in the last 30 days but with zero sales in the previous 90 days

```
WITH recent_sales AS (
    SELECT DISTINCT product_id
    FROM order_items oi
    JOIN orders o ON oi.order_id = o.order_id
    WHERE o.order_date >= CURDATE() - INTERVAL 30 DAY
),
past_sales AS (
    SELECT DISTINCT product_id
    FROM order_items oi
    JOIN orders o ON oi.order_id = o.order_id
    WHERE o.order_date BETWEEN CURDATE() - INTERVAL 120 DAY AND CURDATE() - INTERVAL 31 DAY
)
SELECT p.product_name, p.price
FROM products p
WHERE p.product_id IN (SELECT product_id FROM recent_sales)
AND p.product_id NOT IN (SELECT product_id FROM past_sales);
```

HOW THIS WORKS

Two CTEs build two separate ID lists – products sold in the last 30 days, and products sold in the 90-day window before that – and the final query keeps only IDs that appear in the first list but not the second. This "IN one set, NOT IN another set" shape is the standard way to spot products that suddenly came back to life after a quiet spell, without needing a join at all.

Q182: Flipkart sales analysis

PROBLEM

Top 3 sellers by month

```
WITH monthly_sales AS (
    SELECT
        e.name AS seller,
        DATE_FORMAT(o.order_date, '%Y-%m') AS month,
        SUM(o.amount) AS total_sales
    FROM employees e
    JOIN orders o ON e.emp_id = o.customer_id
    GROUP BY e.name, DATE_FORMAT(o.order_date, '%Y-%m')
),
ranked_sales AS (
    SELECT *,
           RANK() OVER (PARTITION BY month ORDER BY total_sales DESC) AS rn
    FROM monthly_sales
)
SELECT month, seller, total_sales
FROM ranked_sales
WHERE rn <= 3
ORDER BY month DESC, rn;
```

HOW THIS WORKS

This is the “top-N per group” pattern from Level 6, just wrapped in CTEs for readability: the first CTE rolls sales up by seller and month, the second ranks sellers within each month with RANK() OVER (PARTITION BY month …), and the outer query keeps only rn <= 3. Splitting it into two named steps makes it obvious that ranking happens after aggregation, not before.

Q183: Swiggy delivery analysis

PROBLEM

Average delivery time by city and delivery partner

```
CREATE TABLE deliveries (
    delivery_id INT,
    restaurant_id INT,
    delivery_partner_id INT,
    order_time DATETIME,
    delivery_time DATETIME,
    city VARCHAR(50)
);

SELECT
    city,
    delivery_partner_id,
    AVG(TIMESTAMPDIFF(MINUTE, order_time, delivery_time)) AS avg_delivery_time,
    COUNT(*) AS total_deliveries
FROM deliveries
WHERE delivery_time IS NOT NULL
GROUP BY city, delivery_partner_id
HAVING total_deliveries >= 10
ORDER BY city, avg_delivery_time;
```

HOW THIS WORKS

A plain GROUP BY – no CTE needed here. TIMESTAMPDIFF(MINUTE, order_time, delivery_time) turns two timestamps into a single “minutes taken” number per row, AVG() rolls that up per city/partner pair, and HAVING total_deliveries >= 10 filters out partners with too few deliveries to trust the average. Filtering delivery_time IS NOT NULL first excludes orders that haven’t been delivered yet.

Q184: Zomato customer segmentation

PROBLEM

Segment customers based on order frequency and amount

```
WITH customer_metrics AS (
    SELECT
        c.customer_id,
        c.name,
        COUNT(o.order_id) AS order_count,
        SUM(o.amount) AS total_spent,
        AVG(o.amount) AS avg_order_value,
        DATEDIFF(CURDATE(), MAX(o.order_date)) AS days_since_last_order
    FROM customers c
    LEFT JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.customer_id, c.name
)
SELECT
    name,
    order_count,
    total_spent,
    CASE
        WHEN order_count >= 10 AND total_spent > 100000 THEN 'Platinum'
        WHEN order_count >= 5 AND total_spent > 50000 THEN 'Gold'
        WHEN order_count >= 2 AND total_spent > 10000 THEN 'Silver'
        WHEN days_since_last_order > 90 THEN 'Churn Risk'
        ELSE 'Bronze'
    END AS customer_segment,
    CASE
        WHEN days_since_last_order < 7 THEN 'Very Active'
        WHEN days_since_last_order < 30 THEN 'Active'
        WHEN days_since_last_order < 90 THEN 'Dormant'
        ELSE 'Inactive'
    END AS activity_status
FROM customer_metrics
ORDER BY total_spent DESC;
```

HOW THIS WORKS

This is a classic RFM-style segmentation: one CTE computes each customer’s order count, total spend, and recency (days_since_last_order) in a single pass, then two separate CASE expressions turn those raw numbers into human-readable labels. Notice the CASE branches are checked top-to-bottom – a customer is only “Gold” if they fail the “Platinum” test first – so branch order matters as much as the conditions themselves.

Q185: Uber trip analysis

PROBLEM

Find drivers with highest acceptance rate and earnings

```
CREATE TABLE trips (
    trip_id INT,
    driver_id INT,
    rider_id INT,
    request_time DATETIME,
    pickup_time DATETIME,
    dropoff_time DATETIME,
    status VARCHAR(20),
    fare DECIMAL(10,2),
    distance_km DECIMAL(5,2)
);

WITH driver_metrics AS (
    SELECT
        driver_id,
        COUNT(*) AS total_requests,
        SUM(CASE WHEN status = 'Completed' THEN 1 ELSE 0 END) AS completed_trips,
        SUM(CASE WHEN status = 'Completed' THEN fare ELSE 0 END) AS total_earnings,
        AVG(CASE WHEN status = 'Completed' THEN
            TIMESTAMPDIFF(MINUTE, pickup_time, dropoff_time)
        END) AS avg_trip_time
    FROM trips
    GROUP BY driver_id
)
SELECT
    driver_id,
    completed_trips,
    total_earnings,
    ROUND(completed_trips * 100.0 / total_requests, 2) AS acceptance_rate,
    RANK() OVER (ORDER BY total_earnings DESC) AS earnings_rank
FROM driver_metrics
WHERE total_requests > 50
ORDER BY acceptance_rate DESC, total_earnings DESC;
```

HOW THIS WORKS

SUM(CASE WHEN status = ‘Completed’ THEN 1 ELSE 0 END) is the standard trick for “counting only rows that match a condition” inside a single GROUP BY, used twice here – once to count completed trips, once to total their fares. total_requests (every row, completed or not) stays as a plain COUNT(*) so the acceptance-rate division afterward compares completed trips against all requests, not just completed ones.

Q186: Ola ride cancellation analysis

PROBLEM

Find reasons for high cancellation rate by time of day

```
WITH trip_cancellations AS (
    SELECT
        CASE
            WHEN HOUR(request_time) BETWEEN 6 AND 11 THEN 'Morning'
            WHEN HOUR(request_time) BETWEEN 12 AND 17 THEN 'Afternoon'
            WHEN HOUR(request_time) BETWEEN 18 AND 23 THEN 'Evening'
            ELSE 'Night'
        END AS time_of_day,
        status,
        COUNT(*) AS trip_count
    FROM trips
    WHERE status = 'Cancelled'
    GROUP BY time_of_day, status
),
total_by_time AS (
    SELECT
        CASE
            WHEN HOUR(request_time) BETWEEN 6 AND 11 THEN 'Morning'
            WHEN HOUR(request_time) BETWEEN 12 AND 17 THEN 'Afternoon'
            WHEN HOUR(request_time) BETWEEN 18 AND 23 THEN 'Evening'
            ELSE 'Night'
        END AS time_of_day,
        COUNT(*) AS total_trips
    FROM trips
    GROUP BY time_of_day
)
SELECT
    tc.time_of_day,
    tc.trip_count AS cancellations,
    tt.total_trips,
    ROUND(tc.trip_count * 100.0 / tt.total_trips, 2) AS cancellation_rate
FROM trip_cancellations tc
JOIN total_by_time tt ON tc.time_of_day = tt.time_of_day
ORDER BY cancellation_rate DESC;
```

HOW THIS WORKS

The same CASE expression buckets HOUR(request_time) into four labels in both CTEs – one counts only cancelled trips per bucket, the other counts all trips per bucket – and the join lines the two counts up so the rate can be divided out. Whenever you see the identical CASE logic repeated across two CTEs like this, it's because one side needs a filtered count and the other needs the unfiltered total.

Q187: Netflix user watch patterns

PROBLEM

Find binge-watching sessions (3+ episodes in 24 hours)

```
CREATE TABLE viewing_history (
    user_id INT,
    episode_id INT,
    series_id INT,
    watch_date DATETIME,
    duration INT
);

WITH binge_patterns AS (
    SELECT
        user_id,
        series_id,
        watch_date,
        LAG(watch_date) OVER (PARTITION BY user_id, series_id ORDER BY watch_date) AS prev_watch,
        COUNT(*) OVER (PARTITION BY user_id, series_id
                       ORDER BY watch_date
                       RANGE BETWEEN INTERVAL 24 HOUR PRECEDING AND CURRENT ROW) AS episodes_in_24h
    FROM viewing_history
)
SELECT
    user_id,
    series_id,
    MIN(watch_date) AS binge_start,
    MAX(watch_date) AS binge_end,
    episodes_in_24h
FROM binge_patterns
WHERE episodes_in_24h >= 3
GROUP BY user_id, series_id, episodes_in_24h
ORDER BY episodes_in_24h DESC;
```

HOW THIS WORKS

The interesting part is the window frame: RANGE BETWEEN INTERVAL 24 HOUR PRECEDING AND CURRENT ROW is a time-based window rather than a row-count-based one – instead of “the previous 2 rows” it means “every row whose watch_date falls within the last 24 hours of this row’s watch_date”. COUNT(*) over that frame gives a rolling episode count per user/series, and keeping only episodes_in_24h >= 3 surfaces genuine binge sessions. (LAG here is computed but unused in the final SELECT – in practice you’d drop it or use it to also report the gap between episodes.)

Q188: Spotify song popularity analysis

PROBLEM

Find songs that increased in popularity month-over-month

```
CREATE TABLE song_plays (
    song_id INT,
    play_date DATE,
    plays_count INT,
    unique_listeners INT
);

WITH monthly_plays AS (
    SELECT
        song_id,
        DATE_FORMAT(play_date, '%Y-%m') AS month,
        SUM(plays_count) AS total_plays,
        SUM(unique_listeners) AS total_listeners
    FROM song_plays
    GROUP BY song_id, DATE_FORMAT(play_date, '%Y-%m')
),
play_growth AS (
    SELECT
        song_id,
        month,
        total_plays,
        LAG(total_plays) OVER (PARTITION BY song_id ORDER BY month) AS prev_month_plays,
        LAG(total_listeners) OVER (PARTITION BY song_id ORDER BY month) AS prev_month_listeners
    FROM monthly_plays
)
SELECT
    song_id,
    month,
    total_plays,
    prev_month_plays,
    ROUND((total_plays - prev_month_plays) * 100.0 / prev_month_plays, 2) AS growth_percent
FROM play_growth
WHERE prev_month_plays IS NOT NULL
AND total_plays > prev_month_plays
ORDER BY growth_percent DESC
LIMIT 10;
```

HOW THIS WORKS

LAG(total_plays) OVER (PARTITION BY song_id ORDER BY month) fetches “last month’s total_plays for this same song” onto the current row, without any self-join. Once that value sits alongside this month’s number, a plain percentage-change formula does the rest. prev_month_plays IS NOT NULL drops each song’s first month, which has no prior month to compare against.

Q189: YouTube video performance

PROBLEM

Identify videos with high engagement ratio (likes + comments / views)

```
CREATE TABLE videos (
    video_id INT,
    upload_date DATE,
    views INT,
    likes INT,
    comments INT,
    shares INT,
    category VARCHAR(50)
);

SELECT
    video_id,
    category,
    views,
    likes,
    comments,
    shares,
    ROUND((likes + comments + shares) * 100.0 / views, 2) AS engagement_rate,
    CASE
        WHEN views > 1000000 THEN 'Viral'
        WHEN views > 100000 THEN 'Popular'
        WHEN views > 10000 THEN 'Rising'
        ELSE 'New'
    END AS video_tier
FROM videos
WHERE views > 1000
ORDER BY engagement_rate DESC
LIMIT 20;
```

HOW THIS WORKS

No CTE or window function needed – just row-level arithmetic. engagement_rate is computed directly per row as (likes + comments + shares) / views, and video_tier buckets the raw view count with a CASE expression. Multiplying by 100.0 (not 100) forces the division into floating-point math before ROUND(), which is what keeps the result from truncating to a whole number.

Q190: Instagram follower growth

PROBLEM

Calculate follower growth rate by month

```
CREATE TABLE follower_history (
    account_id INT,
    date DATE,
    followers INT,
    following INT,
    posts_count INT
);

WITH monthly_growth AS (
    SELECT
        account_id,
        DATE_FORMAT(date, '%Y-%m') AS month,
        MIN(followers) AS start_followers,
        MAX(followers) AS end_followers,
        MIN(posts_count) AS start_posts,
        MAX(posts_count) AS end_posts
    FROM follower_history
    GROUP BY account_id, DATE_FORMAT(date, '%Y-%m')
)
SELECT
    account_id,
    month,
    start_followers,
    end_followers,
    end_followers - start_followers AS net_gain,
    ROUND((end_followers - start_followers) * 100.0 / start_followers, 2) AS growth_rate,
    (end_posts - start_posts) AS new_posts
FROM monthly_growth
WHERE start_followers > 0
ORDER BY growth_rate DESC;
```

HOW THIS WORKS

Rather than LAG-ing a previous row, this squeezes “start of month” and “end of month” out of a single GROUP BY: since follower_history logs one row per day, MIN(followers) within a month is (assuming followers only grow) the earliest reading and MAX(followers) the latest. It’s a lighter-weight alternative to LAG() that works whenever a metric only moves in one direction across the days being grouped.

Q191: Banking transaction analysis

PROBLEM

Identify suspicious transactions (> 3x average, > 5 transactions/day)

```
CREATE TABLE bank_transactions (
    transaction_id INT,
    customer_id INT,
    transaction_date DATETIME,
    amount DECIMAL(10,2),
    type VARCHAR(20),
    status VARCHAR(20)
);

WITH customer_avg AS (
    SELECT
        customer_id,
        AVG(amount) AS avg_transaction,
        STDDEV(amount) AS stddev_transaction
    FROM bank_transactions
    WHERE status = 'Completed'
    GROUP BY customer_id
),
daily_counts AS (
    SELECT
        customer_id,
        DATE(transaction_date) AS day,
        COUNT(*) AS daily_count
    FROM bank_transactions
    GROUP BY customer_id, DATE(transaction_date)
)
SELECT
    t.transaction_id,
    t.customer_id,
    t.amount,
    c.avg_transaction,
    t.amount / c.avg_transaction AS multiplier,
    CASE
        WHEN t.amount > c.avg_transaction + 3 * c.stddev_transaction THEN 'Extreme'
        WHEN t.amount > c.avg_transaction * 2 THEN 'High'
        ELSE 'Normal'
    END AS risk_level
FROM bank_transactions t
JOIN customer_avg c ON t.customer_id = c.customer_id
WHERE t.amount > c.avg_transaction * 2
OR (t.customer_id, DATE(t.transaction_date)) IN (
    SELECT customer_id, day FROM daily_counts WHERE daily_count > 5
)
ORDER BY t.amount DESC;
```

HOW THIS WORKS

Two independent CTEs each answer one half of “what does normal look like for this customer” – customer_avg gives a per-customer baseline (mean and standard deviation), daily_counts gives a per-day transaction frequency. The WHERE clause then flags a transaction if either condition trips: its amount is more than double that customer’s average, OR it belongs to a (customer, day) pair with more than 5 transactions. The tuple form (t.customer_id, DATE(t.transaction_date)) IN (...) is a compact way to check two columns against a subquery at once, instead of writing a second join.

Q192: ATM withdrawal patterns

PROBLEM

Find ATMs with most withdrawals and average amount by time

```
CREATE TABLE atm_transactions (
    atm_id INT,
    transaction_time DATETIME,
    amount DECIMAL(10,2),
    customer_id INT,
    status VARCHAR(20)
);

SELECT
    atm_id,
    DATE(transaction_time) AS day,
    HOUR(transaction_time) AS hour,
    COUNT(*) AS withdrawals,
    AVG(amount) AS avg_amount,
    SUM(amount) AS total_amount,
    RANK() OVER (PARTITION BY DATE(transaction_time) ORDER BY COUNT(*) DESC) AS withdrawal_rank
FROM atm_transactions
WHERE status = 'Successful'
AND transaction_time >= CURDATE() - INTERVAL 30 DAY
GROUP BY atm_id, DATE(transaction_time), HOUR(transaction_time)
HAVING withdrawals >= 10
ORDER BY day DESC, withdrawal_rank;
```

HOW THIS WORKS

This mixes an aggregate window function directly into a GROUP BY query: COUNT*() OVER (PARTITION BY DATE(…) …) is allowed to reference the same COUNT()* that GROUP BY just computed, because window functions always run after grouping. The result is each atm/day/hour bucket ranked against every other bucket from that same day, so you can see which ATM was busiest on any given day without a separate query.

Q193: Hospital patient readmission analysis

PROBLEM

Calculate 30-day readmission rate by department

```
CREATE TABLE admissions (
    admission_id INT,
    patient_id INT,
    admission_date DATE,
    discharge_date DATE,
    department VARCHAR(50),
    diagnosis VARCHAR(100)
);

WITH readmissions AS (
    SELECT
        a1.patient_id,
        a1.department,
        a1.admission_date,
        a1.discharge_date,
        a2.admission_date AS readmission_date,
        DATEDIFF(a2.admission_date, a1.discharge_date) AS days_between
    FROM admissions a1
    LEFT JOIN admissions a2 ON a1.patient_id = a2.patient_id
        AND a2.admission_date > a1.admission_date
        AND DATEDIFF(a2.admission_date, a1.discharge_date) <= 30
),
department_stats AS (
    SELECT
        department,
        COUNT(DISTINCT patient_id) AS unique_patients,
        COUNT(DISTINCT CASE WHEN readmission_date IS NOT NULL THEN patient_id END) AS readmitted_patients
    FROM readmissions
    GROUP BY department
)
SELECT
    department,
    unique_patients,
    readmitted_patients,
    ROUND(readmitted_patients * 100.0 / unique_patients, 2) AS readmission_rate
FROM department_stats
WHERE unique_patients >= 10
ORDER BY readmission_rate DESC;
```

HOW THIS WORKS

The readmissions CTE is a self-join on the same admissions table (a1 = the original stay, a2 = any later stay by the same patient within 30 days of discharge) – structurally similar to the self-referencing joins used for hierarchies earlier, but joined by patient_id and a date condition instead of a manager_id. LEFT JOIN keeps patients with no readmission too (their readmission_date is NULL), which is exactly what COUNT(DISTINCT CASE WHEN readmission_date IS NOT NULL THEN patient_id END) needs in order to count only the patients who really did come back.

Q194: Student marks analysis

PROBLEM

Find top 3 students in each subject with average > 80

```
CREATE TABLE marks (
    student_id INT,
    subject VARCHAR(50),
    marks INT,
    exam_date DATE
);

WITH student_subject_scores AS (
    SELECT
        student_id,
        subject,
        AVG(marks) AS avg_marks,
        COUNT(*) AS exams_taken
    FROM marks
    GROUP BY student_id, subject
    HAVING avg_marks > 80
),
ranked_students AS (
    SELECT
        student_id,
        subject,
        avg_marks,
        RANK() OVER (PARTITION BY subject ORDER BY avg_marks DESC) AS subject_rank
    FROM student_subject_scores
)
SELECT
    student_id,
    subject,
    ROUND(avg_marks, 2) AS avg_score,
    subject_rank
FROM ranked_students
WHERE subject_rank <= 3
ORDER BY subject, subject_rank;
```

HOW THIS WORKS

Two filters happen at two different stages, which is why this needs two CTEs rather than one: HAVING avg_marks > 80 filters out weak students before ranking even starts, then RANK() OVER (PARTITION BY subject ...) ranks only the students who survived that filter, and subject_rank <= 3 keeps the podium. Doing the HAVING filter and the rank filter in the same WHERE clause wouldn't work, since a WHERE clause can't reference a window function's own output.

Q195: E-commerce revenue analysis by product category

PROBLEM

Find categories with highest growth in revenue

```
WITH monthly_category_revenue AS (
    SELECT
        p.category,
        DATE_FORMAT(o.order_date, '%Y-%m') AS month,
        SUM(oi.quantity * oi.price) AS revenue
    FROM products p
    JOIN order_items oi ON p.product_id = oi.product_id
    JOIN orders o ON oi.order_id = o.order_id
    WHERE o.status = 'Completed'
    GROUP BY p.category, DATE_FORMAT(o.order_date, '%Y-%m')
),
category_growth AS (
    SELECT
        category,
        month,
        revenue,
        LAG(revenue) OVER (PARTITION BY category ORDER BY month) AS prev_month_revenue
    FROM monthly_category_revenue
)
SELECT
    category,
    month,
    revenue,
    prev_month_revenue,
    ROUND((revenue - prev_month_revenue) * 100.0 / prev_month_revenue, 2) AS growth_percent,
    CASE
        WHEN revenue > prev_month_revenue THEN 'Growing'
        WHEN revenue < prev_month_revenue THEN 'Declining'
        ELSE 'Stable'
    END AS trend
FROM category_growth
WHERE prev_month_revenue IS NOT NULL
ORDER BY growth_percent DESC;
```

HOW THIS WORKS

Same LAG-based month-over-month shape as Q188's Spotify question and Q190's Instagram question – by this point in the level the pattern should be recognizable on sight: aggregate to one row per group-per-month, LAG() that revenue within each group, then compare current vs. previous. The only new piece here is the three-way join (products, order_items, orders) needed just to get category and revenue onto the same row in the first place.

Q196: Warehouse inventory optimization

PROBLEM

Identify products with low turnover (aged inventory)

```
CREATE TABLE inventory_items (
    item_id INT,
    product_id INT,
    warehouse_id INT,
    quantity INT,
    last_restock_date DATE,
    sales_last_30_days INT
);

WITH inventory_metrics AS (
    SELECT
        product_id,
        warehouse_id,
        quantity,
        DATEDIFF(CURDATE(), last_restock_date) AS days_since_restock,
        sales_last_30_days,
        CASE
            WHEN sales_last_30_days = 0 AND DATEDIFF(CURDATE(), last_restock_date) > 90 THEN 'Critical'
            WHEN sales_last_30_days < 5 AND DATEDIFF(CURDATE(), last_restock_date) > 60 THEN 'Excess'
            WHEN quantity < 10 AND sales_last_30_days > 0 THEN 'Low Stock'
            ELSE 'Normal'
        END AS inventory_status
    FROM inventory_items
)
SELECT
    product_id,
    warehouse_id,
    quantity,
    days_since_restock,
    sales_last_30_days,
    inventory_status,
    ROUND(quantity * 1.0 / NULLIF(sales_last_30_days, 0), 2) AS days_of_inventory
FROM inventory_metrics
WHERE inventory_status != 'Normal'
ORDER BY FIELD(inventory_status, 'Critical', 'Low Stock', 'Excess'), days_since_restock DESC;
```

HOW THIS WORKS

NULLIF(sales_last_30_days, 0) swaps a 0 for NULL right before the division, which is what stops quantity / 0 from throwing a divide-by-zero error – dividing by NULL just quietly produces NULL instead. FIELD(inventory_status, ‘Critical’, ‘Low Stock’, ‘Excess’) is a handy way to sort text values into a custom order (not alphabetical) by giving each string a position number to sort by, so the most urgent status prints first.

Q197: Flight booking analysis

PROBLEM

Find optimal pricing by analyzing booking patterns

```
CREATE TABLE flight_bookings (
    booking_id INT,
    flight_id INT,
    booking_date DATE,
    travel_date DATE,
```

```
    price DECIMAL(10,2),
    seats_booked INT,
    customer_id INT
);

WITH booking_metrics AS (
    SELECT
        flight_id,
        booking_date,
        travel_date,
        price,
        seats_booked,
        DATEDIFF(travel_date, booking_date) AS days_in_advance,
        CASE
            WHEN DATEDIFF(travel_date, booking_date) > 60 THEN 'Very Early'
            WHEN DATEDIFF(travel_date, booking_date) > 30 THEN 'Early'
            WHEN DATEDIFF(travel_date, booking_date) > 14 THEN 'Mid'
            WHEN DATEDIFF(travel_date, booking_date) > 7 THEN 'Late'
            ELSE 'Last Minute'
        END AS booking_window
    FROM flight_bookings
),
price_analysis AS (
    SELECT
        flight_id,
        booking_window,
        AVG(price) AS avg_price,
        SUM(seats_booked) AS total_seats,
        COUNT(*) AS bookings_count
    FROM booking_metrics
    GROUP BY flight_id, booking_window
)
SELECT
    flight_id,
    booking_window,
    ROUND(avg_price, 2) AS avg_price,
    total_seats,
    bookings_count,
    ROUND(AVG(avg_price) OVER (PARTITION BY flight_id), 2) AS flight_avg_price,
    ROUND((avg_price - AVG(avg_price) OVER (PARTITION BY flight_id)) /
          AVG(avg_price) OVER (PARTITION BY flight_id) * 100, 2) AS price_variance
FROM price_analysis
ORDER BY flight_id, FIELD(booking_window, 'Very Early', 'Early', 'Mid', 'Late', 'Last Minute');
```

HOW THIS WORKS

Three layers stack here: booking_metrics buckets each individual booking into a “how far in advance” window, price_analysis aggregates those into one row per flight/window, and the outer SELECT adds AVG(avg_price) OVER (PARTITION BY flight_id) – a window function computed on top of already-aggregated data – to get each flight’s overall average price for comparison. price_variance then shows, as a percentage, how much pricier or cheaper each booking window is versus that flight’s own average, which is exactly the “how much does booking early actually save” answer the question is really asking for.

Q198: Hotel reservation analysis

PROBLEM

Find peak occupancy periods and predict demand

```
CREATE TABLE reservations (
    reservation_id INT,
    hotel_id INT,
    check_in_date DATE,
    check_out_date DATE,
    rooms_booked INT,
    total_guests INT,
    total_amount DECIMAL(10,2),
    status VARCHAR(20)
);

WITH daily_occupancy AS (
    SELECT hotel_id, check_in_date AS date, rooms_booked, 'Check-in' AS event_type
    FROM reservations WHERE status = 'Confirmed'
    UNION ALL
    SELECT hotel_id, check_out_date AS date, -rooms_booked, 'Check-out' AS event_type
    FROM reservations WHERE status = 'Confirmed'
),
occupancy_by_date AS (
    SELECT
        hotel_id,
        date,
        SUM(rooms_booked) AS net_rooms_change,
        SUM(SUM(rooms_booked)) OVER (PARTITION BY hotel_id ORDER BY date) AS occupied_rooms
    FROM daily_occupancy
    GROUP BY hotel_id, date
)
SELECT
    hotel_id, date, occupied_rooms,
    LAG(occupied_rooms) OVER (PARTITION BY hotel_id ORDER BY date) AS prev_day_occupancy,
    occupied_rooms - LAG(occupied_rooms) OVER (PARTITION BY hotel_id ORDER BY date) AS daily_change,
    CASE
        WHEN occupied_rooms > (SELECT AVG(occupied_rooms) FROM occupancy_by_date WHERE hotel_id = o.hotel_id) * 1.2
        THEN 'Peak'
        WHEN occupied_rooms < (SELECT AVG(occupied_rooms) FROM occupancy_by_date WHERE hotel_id = o.hotel_id) * 0.8
        THEN 'Low'
        ELSE 'Normal'
    END AS occupancy_tier
FROM occupancy_by_date o
WHERE date >= CURDATE()
ORDER BY hotel_id, date;
```

HOW THIS WORKS

The clever setup is daily_occupancy: instead of computing “how many rooms are occupied on date X” directly, it turns every reservation into two signed events – a +rooms_booked row on check-in and a -rooms_booked row on check-out – stacked with UNION ALL. A running total of those signed events (SUM(SUM(...)) OVER (ORDER BY date), summing an already-summed daily total) then naturally produces the occupied-room count for every date, rising on check-ins and falling on check-outs, the same running-balance idea as Q177’s credits/debits but applied to room-nights instead of money.

Q199: Stock market transaction analysis

PROBLEM

Calculate portfolio performance with buy/sell patterns

```
CREATE TABLE stock_trades (
    trade_id INT,
    user_id INT,
    stock_symbol VARCHAR(10),
    trade_date DATE,
    trade_type VARCHAR(4), -- 'BUY', 'SELL'
    shares INT,
    price DECIMAL(10,2)
);

WITH position_calculations AS (
    SELECT
        user_id, stock_symbol, trade_date, trade_type, shares, price,
        CASE WHEN trade_type = 'BUY' THEN shares ELSE -shares END AS net_shares,
        CASE WHEN trade_type = 'BUY' THEN shares * price ELSE -shares * price END AS net_value
    FROM stock_trades
),
daily_positions AS (
    SELECT
        user_id, stock_symbol, trade_date, shares, price, net_shares, net_value,
        SUM(net_shares) OVER (PARTITION BY user_id, stock_symbol ORDER BY trade_date) AS cumulative_shares,
        SUM(net_value) OVER (PARTITION BY user_id, stock_symbol ORDER BY trade_date) AS cost_basis
    FROM position_calculations
)
SELECT
    user_id, stock_symbol, trade_date, trade_type, shares, price, cumulative_shares, cost_basis,
    CASE WHEN cumulative_shares > 0 THEN ROUND(cost_basis / cumulative_shares, 2) ELSE 0 END AS avg_cost_per_share,
    RANK() OVER (PARTITION BY user_id ORDER BY trade_date DESC) AS recent_trade_rank
FROM daily_positions
WHERE trade_type = 'SELL'
ORDER BY user_id, trade_date DESC;
```

HOW THIS WORKS

BUY and SELL rows are first converted into signed net_shares and net_value (a SELL becomes negative), and then two running totals – SUM(net_shares) OVER (...) and SUM(net_value) OVER (...), both ordered by trade_date – turn that trade-by-trade history into a live position size and running cost basis per user/stock, the same signed-running-total idea as Q177 and Q198 applied a third time to a different domain. avg_cost_per_share = cost_basis / cumulative_shares at that point in time is then just one more column computed from the two running totals.

Q200: End-to-End Employee Management System SQL Challenge

```
-- Complete employee management dashboard
WITH employee_details AS (
    SELECT
        e.emp_id, e.name, e.age, e.gender, e.salary, e.department, e.designation,
        e.hire_date, e.experience_years, e.bonus,
        m.name AS manager_name, d.location AS dept_location, d.budget AS dept_budget
    FROM employees e
    LEFT JOIN employees m ON e.manager_id = m.emp_id
    LEFT JOIN departments d ON e.department = d.dept_name
),
department_stats AS (
    SELECT
        department, COUNT(*) AS employee_count, AVG(salary) AS avg_salary,
        SUM(salary) AS total_salary, AVG(experience_years) AS avg_experience,
        MAX(salary) AS max_salary, MIN(salary) AS min_salary
    FROM employee_details
    GROUP BY department
),
project_stats AS (
    SELECT department, COUNT(*) AS active_projects, SUM(budget) AS project_budget
    FROM projects WHERE status = 'Active'
    GROUP BY department
),
order_stats AS (
    SELECT e.department, COUNT(o.order_id) AS total_orders, SUM(o.amount) AS total_revenue,
           AVG(o.amount) AS avg_order_value
    FROM employees e
    LEFT JOIN orders o ON e.emp_id = o.customer_id
    GROUP BY e.department
)
SELECT
    ed.department, ed.employee_count, ROUND(ed.avg_salary, 2) AS avg_salary,
    ROUND(ed.total_salary, 2) AS total_salary, ROUND(ed.avg_experience, 1) AS avg_experience_years,
    COALESCE(ps.active_projects, 0) AS active_projects,
    COALESCE(ROUND(os.total_revenue, 2), 0) AS total_revenue,
    ROUND(COALESCE(os.total_revenue, 0) / NULLIF(ed.total_salary, 0) * 100, 2) AS revenue_to_salary_ratio,
    RANK() OVER (ORDER BY COALESCE(os.total_revenue, 0) DESC) AS revenue_rank
FROM department_stats ed
LEFT JOIN project_stats ps ON ed.department = ps.department
LEFT JOIN order_stats os ON ed.department = os.department
ORDER BY revenue_rank;
```

HOW THIS WORKS

The capstone question, and deliberately the longest query in the book – but every individual piece has already appeared earlier in this level or the ones before it. Four CTEs each aggregate one business area (employee details, department stats, project stats, order stats) completely independently, and only the final SELECT joins all four together with LEFT JOIN, so a department with zero active projects or zero orders still shows up instead of disappearing. COALESCE(…, 0) papers over the NULLs a LEFT JOIN produces for departments with no matching projects/orders, and NULLIF(ed.total_salary, 0) guards the final ratio against a divide-by-zero the same way Q196 did. If you can trace what each CTE contributes on its own before looking at the JOINs, you've fully internalized the pattern this whole level was building toward.